Plataforma de Simulação GenESyS

Guia prático do usuário e do desenvolvedor GenESyS

Prof. Rafael Luiz Cancian, Dr. Eng.

www.ine.ufsc.br/rafael.cancian

July 2026

Contents

II Developer Manual

Preface

This book presents the *GenESyS* simulator as both a tool for practical use and a development platform. Its purpose is twofold: it serves the reader who wants to install the system, understand how it is used, build models, execute them, and interpret the results; it also serves the developer who wants to understand the internal architecture and extend the simulator with new components, data definitions, language functions, tools, and applications.

Unlike a broad text on systems modeling and simulation, this volume is deliberately centered on software. General concepts appear only when they are needed to explain how *GenESyS* works, how it is organized, or how its resources should be used. The emphasis remains on concrete repository-backed material: operational descriptions, documented commands, architectural decisions, and reproducible procedures.

The book is organized to support progressive reading. The early chapters focus on direct use: installation, first execution, model creation, simulation, and result inspection. The middle chapters explain the user-facing language, shell, worker boundary, and standalone tools. The later chapters move into architecture, implementation, testing, packaging, and governance.

The front matter below makes the reading strategy and evidence model explicit. It records the repository snapshot used for this revision, the main paths through the book, the current safety and scientific boundaries, and the conventions used throughout the manual.

Repository Revision and Evidence

This manual revision is anchored to the current repository snapshot used for this update:

- Repository: `rlcancian/Genesys-Simulator`
- Branch: `WorkInProgress`
- Commit: `f3214dfbde92b28a292416f2abbb033af023d25d`
- Manual revision date: July 23, 2026
- Build environment: Ubuntu 24.04.4 LTS
- Compiler: g++ 13.3.0
- CMake: 3.28.3
- Qt: 6.4.2

All source-level claims in this book should be read against that snapshot unless a later chapter explicitly states a newer validated commit.

Reading Paths

The manual is written for several entry points:

- first contact;
- GUI modeling;
- terminal execution;
- component development;
- architecture;
- tests;
- scientific simulation;
- biological extensions.

Figure 1 shows the recommended progression through the book.

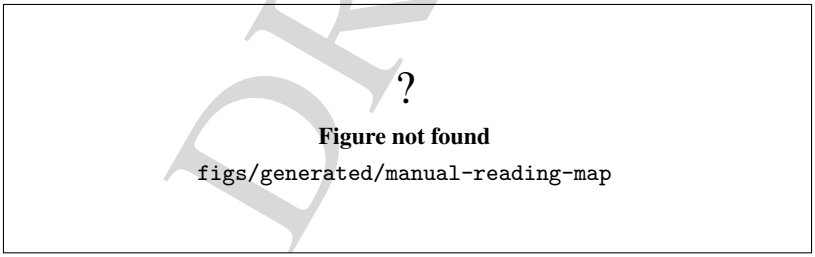


Figure 1: Recommended progression through the manual from first contact to advanced topics.

Figure 2 maps audiences to the chapters that matter most to them.

Safety and Scientific Validity

The manual uses a bounded evidence ladder: build evidence, startup evidence, functional evidence, numerical evidence, scientific evidence, security evidence, and release readiness.

Figure 3 summarizes that ladder and the distinctions between the evidence levels.

?

Figure not found

`figs/generated/manual-audience-matrix`

Figure 2: Audience-to-chapter matrix for the current manual structure.

?

Figure not found

`figs/generated/manual-evidence-ladder`

Figure 3: Evidence ladder used by the manual.

Figure 4 summarizes the current application scope and the maturity level claimed for each family of tools.

?

Figure not found

`figs/generated/manual-application-scope`

Figure 4: Current application scope and maturity snapshot.

This book does not treat build success as scientific validation, and local-only or private-only services remain documented with that boundary intact.

Conventions

The canonical prose is written in English. Commands, file names, targets, labels, and code symbols are preserved as they appear in the repository.

The manual uses the following maturity words in a narrow, technical sense:

- supported;
- partially validated;
- experimental;
- planned.

Figure blocks are documented with ‘FIGURE-SPEC‘ comments and a placeholder asset until a final image is available. That rule keeps the source explicit even when the artwork is still pending.

This manual remains a draft and should be treated as a work in progress. It is intentionally conservative about claims: build and startup success are reported as build or startup evidence only, and stronger claims require the corresponding validation level. The *Worker* service remains documented with its local or private boundary intact.

The front matter sections are organized in Sections , , , and .

Florianopolis, July 2026.

User Manual

1. User Installation

1.1 Introduction

This chapter explains how to install and start GenESyS as a user-facing application, without requiring knowledge of the source tree or the internal build layout. The focus is on supported installation paths, the runtime files that appear after installation, the first launch of the GUI, and the checks that confirm the program is ready for use.

The documented baseline for the repository is Ubuntu 24.04. The current packaging and build surface are Debian-based and Qt6-based, so this chapter is written around that environment. Other systems may work if they satisfy the same dependencies, but they are not the baseline validated by this repository.

For a user, the important distinction is simple:

- installation puts a runnable GenESyS application on the system;
- development build instructions are for contributors and validators.

The chapter therefore keeps the user path separate from the source-build path, even though both can produce the same GUI executable name.

1.2 Supported installation paths

The repository currently documents two practical ways to get a runnable GenESyS GUI on the machine.

?

Figure not found

`figs/generated/user-installation-flow`

Figure 1.1: Supported installation and first-start flow for GenESyS.

1.2.1 Preferred path: Debian package

The packaged desktop application is named `genesys-gui`. The Debian metadata installs the executable to `/usr/bin/genesys-gui` and also ships the desktop integration files under the standard XDG locations:

- `/usr/share/applications/io.github.rlcancian.genesys.desktop`;
- `/usr/share/metainfo/io.github.rlcancian.genesys.metainfo.xml`;
- `/usr/share/icons/hicolor/scalable/apps/io.github.rlcancian.genesys.svg`.

This is the clearest end-user route when a package artifact is available. It creates the expected desktop launcher and provides a direct command-line entry point.

If the package is available as a local file, install it with `apt` so dependency resolution happens automatically:

```
sudo apt install ./genesys-gui_*.deb
```

Listing 1.1: Installing the local GenESyS Debian package

If the package is already published in a configured repository, the same binary package can be installed by name:

```
sudo apt install genesys-gui
```

Listing 1.2: Installing GenESyS from a repository

1.2.2 Manual artifact installation

When a binary artifact is delivered outside an apt repository, the same package format should still be preferred. In practice, that means a user or maintainer can download the `.deb` file and install it locally with the command above.

If the artifact is provided as a staged filesystem tree rather than a package, the repository does not define a separate user-facing installer for it. In that case, the artifact should be treated as a validation or distribution artifact that must already match the expected layout before it is copied into place. This chapter does not define a new custom installer.

1.3 System baseline and runtime dependencies

The current supported baseline is the Qt6 desktop stack on Ubuntu 24.04. The Debian package metadata confirms the build-time requirements used by the project:

- `cmake` 3.24 or newer;
- `ninja-build`;
- `g++`;
- `pkgconf`;
- `qt6-base-dev`;
- `qt6-base-dev-tools`;
- `libgl-dev`.

For a packaged install, the user does not usually install those build-time packages directly. They matter here because they define the toolchain that produces the user package and the GUI runtime.

At runtime, the GUI expects a normal graphical desktop session, access to the display server, and the standard desktop integration files provided by the package. The application is intended to launch as a desktop GUI, not as a headless background service.

?

Figure not found

`figs/generated/user-installed-structure`

Figure 1.2: Installed GenESyS desktop files and launcher locations.

1.4 First launch of the GUI

After installation, start the GUI from the desktop launcher or from a shell. The installed binary name is `genesys-gui`. A terminal launch looks like this:

```
genesys-gui
```

Listing 1.3: Starting the installed GenESyS GUI

If you are validating a local build rather than a package install, the current source tree can produce the same executable through the `gui-app` preset:

```
cmake --preset gui-app
cmake --build --preset gui-app --parallel "$(nproc)"
./build/gui-app/source/applications/gui/genesys/genesys-gui
```

Listing 1.4: Local build-and-run validation path

The local build path is useful for validation, but it is not the same thing as the user-install path. In the book, the package install is the primary user path.

?

Figure not found

`figs/generated/user-first-start`

Figure 1.3: First successful start of the GenESyS GUI.

1.4.1 What to confirm on the first run

The first launch should confirm only the basics:

- the window opens without an immediate failure;
- the menus and toolbars are visible;
- the main workspace loads normally;
- the application can be closed cleanly.

The user does not need to build a model yet. The goal is only to prove that the installed application starts and displays the main GUI.

1.4.2 Version confirmation

The GUI includes an *About* menu with an *About Genesys* action. That action is the most direct way to confirm the application identity and version information from inside the running program.

?

Figure not found

`figs/generated/user-version-confirmation`

Figure 1.4: Version confirmation through the GenESyS About dialog.

1.5 Where the files live after installation

From a user point of view, the important installed locations are the launcher binary and the desktop integration files. For package-based installs, the key paths are:

- `/usr/bin/genesys-gui`;
- `/usr/share/applications/io.github.rlcancian.genesys.desktop`;
- `/usr/share/metainfo/io.github.rlcancian.genesys.metainfo.xml`;
- `/usr/share/icons/hicolor/scalable/apps/io.github.rlcancian.genesys.svg`.

For local validation from source, the matching build-tree executable is:

- `build/gui-app/source/applications/gui/genesys/genesys-gui`.

?

Figure not found

`figs/generated/user-executable-locations`

Figure 1.5: Installed and build-tree executable locations for GenESyS.

1.6 Updating the installation

If GenESyS was installed from a Debian package, update it by installing the new package version over the old one. On a configured package repository, the standard flow is:

```
sudo apt update
sudo apt install --only-upgrade genesys-gui
```

Listing 1.5: Updating an apt-based GenESyS installation

If you are using a local `.deb` file, replace the package from the new artifact:

```
sudo apt install ./genesys-gui_*.deb
```

Listing 1.6: Replacing a local package artifact

This keeps the package manager in control of file ownership and dependency tracking. It is preferable to copying files by hand into system directories.

1.7 Removing the installation

To remove the package, use the package manager instead of deleting files manually. A plain remove keeps configuration if the package has any:

```
sudo apt remove genesys-gui
```

Listing 1.7: Removing the GenESyS package

If a full purge is desired, remove configuration files as well:

```
sudo apt purge genesys-gui
```

Listing 1.8: Purging the GenESyS package

Because the Debian package also declares `genesys-web` as a runtime dependency, a system that installed both packages explicitly may need both names in the removal command. The safe habit is to inspect the package list after the first uninstall if the goal is to clean the machine completely.

1.8 Initial troubleshooting

If the GUI does not start, the first checks should be practical rather than architectural:

- verify that the package installed successfully;
- verify that `genesys-gui` is on the path or run from the full installed location;
- verify that the desktop session can open Qt applications;
- verify that the package dependencies were resolved;
- verify that the display session is available if the command is started from a terminal.

If the application starts but the About dialog or launcher does not look correct, recheck the package version and the desktop integration files. If the local build path is being used, confirm that the `gui-app` preset was selected and that the build completed successfully before starting the binary.

1.9 Limitations

This chapter does not claim support for every operating system or every distribution packaging style. It documents the Debian-based package path and the local build-tree validation path that are currently confirmed in the repository.

It also does not redefine the installer format. If a future release adds a different distribution artifact, that artifact should be documented with the same discipline as the Debian package: package name, installed paths, launch command, update path, removal path, and first-run verification.

?

Figure not found

`figs/generated/user-installation-structure-summary`

Figure 1.6: Lifecycle summary for a user installation of GenESyS.

1.10 Final considerations

The user installation path for GenESyS is intentionally narrow: install the desktop package when available, or use the local build-tree validation path only when you are checking the runtime from source. In both cases, the outcome that matters is the same: the `genesys-gui` application starts, displays its main window, and exposes the expected About dialog and desktop launcher.

Once that works, the reader can move to the next chapter and start learning how the application is organized and what the main GUI areas mean in practice.

Test your understanding of this content by answering the following questions:

1. Why is the Debian package the preferred user-installation path in this chapter?
2. Which files and paths confirm that GenESyS was installed correctly on the system?
3. How does the chapter distinguish a user installation from a local build-tree validation?

2. Application Overview

This chapter maps the current GenESyS application family and explains which executable a reader should open for each common workflow. It is intentionally grounded in the current repository state, not in historical names or planned future layouts.

The overview is split into four groups:

- the main interactive entry points;
- the standalone Qt frontends;
- the model-specific application route;
- the planned-but-not-implemented DOE route.

Figure 2.1 summarizes that family tree. Chapter 2 then introduces the entry-point map in Section 2.1, expands the core entry points in Section 2.2, covers the standalone Qt frontends in Section 2.3, describes the model-specific route in Section 2.4, and closes with the final choice flow in Section 2.5. The following sections spell out the target, executable, purpose, dependencies, public, status, evidence level, limitations, and the chapter where each application is documented in more detail.

2.1 Choosing the right entry point

If the goal is interactive model work, start with the main GUI. If the goal is scripted or terminal-driven execution, start with the shell. If the goal is to serve or control jobs through the worker runtime, use the worker. If the goal is a specialized Qt helper, use one of the standalone frontends. If the goal is to

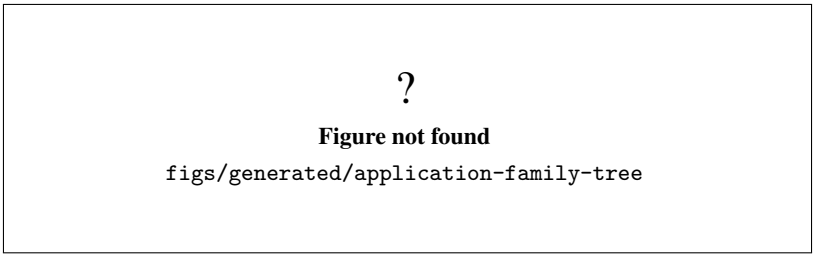


Figure 2.1: Current GenESyS application family and intended usage groups.

run a selected example from `source/applications/modelSpecific`, use the model-specific application route. If the goal is DOE functionality, no application exists yet and the plan remains intentionally unimplemented.

Figure 2.4 condenses that choice into a simple decision path for readers who want the shortest route to the right executable.

2.2 Core entry points

2.2.1 Main GUI

The main GUI is the canonical interactive entry point for model editing and simulation work. It is the first application a new user should open when the goal is to inspect, edit, or execute a model visually.

Target. `genesys_gui_application`, exposed through the aggregate `genesys_gui` target.

Executable. `genesys-gui`.

Purpose. Primary Qt editor and simulation front end for day-to-day model work.

Dependencies. Qt6 Core, Gui, Widgets, PrintSupport, and Network, with Qt5 fallback in the current CMake code, plus the shared kernel, parser, plugin, simulator, worker-support, and tools libraries linked by the GUI target.

Public. General users, model authors, and reviewers who need the main interactive workspace.

Status. Partially validated.

Evidence. The installed launcher, build-tree startup, and About-dialog version check are documented in Chapter 1; the current status ledger records focused GMDD diagnostics but not a full end-to-end interaction certification.

Limitations. Startup and focused GUI checks do not prove complete workflows or scientific correctness.

Chapter. Chapter 3.

2.2.2 Shell

The shell is the command-line entry point for scripted or terminal-based work. It is the right choice when the user wants direct control over model loading, commands, and repeatable text-driven execution.

Target. `genesys_shell`.

Executable. `genesys_shell`.

Purpose. Command-line front end for scripted model loading, command routing, and interactive terminal usage.

Dependencies. Kernel utility and statistics libraries, parser, plugins, and simulator support/runtime libraries.

Public. Advanced users, automation scripts, and developers who prefer a terminal workflow.

Status. Partially validated.

Evidence. The current status ledger records preset/build coverage, scripted command handling, plugin count reporting, and exit behavior.

Limitations. This chapter does not claim GUI workflows, file-based plugin autoload, or broader deployment behavior for the shell.

Chapter. Chapter 9.

2.2.3 Worker

The worker is the HTTP/background runtime that serves simulation-related requests. It is not a GUI helper and it is not approved for public Internet exposure by default.

Target. `genesys_worker_app`.

Executable. `genesys-worker`.

Purpose. Service runtime for controlled worker jobs, session handling, and backend request processing.

Dependencies. `genesys_worker_core` plus the shared kernel, parser, plugins, simulator support/runtime libraries, and the worker HTTP/session/auth stack.

Public. Operators and integration code in controlled academic or intranet-style deployments.

Status. Partially validated.

Evidence. The current status ledger records preset/build coverage, loopback health checks, exact JSON responses, and bounded exit behavior.

Limitations. Public bind, authentication, TLS, quotas, isolation, and audit controls are still separate security requirements, so startup evidence does not establish deployment readiness.

Chapter. Chapter 10.

2.3 Standalone Qt frontends

Figure 2.2 groups the executable targets with the shared backend libraries they rely on. This makes the current build surface easier to scan without pretending that the helper windows are a single monolithic executable.

2.3.1 HTTP Worker GUI

The HTTP Worker GUI is the control dialog for the worker runtime. It exists to support the worker-focused deployment path and to provide a graphical front end around the worker service layer.

Target. `genesys_httpworker_gui_application`.

Executable. `genesys-httpworker-gui`.

?

Figure not found

`figs/generated/application-executable-matrix`

Figure 2.2: Executable targets and shared backend library families for the current GUI applications.

Purpose. Qt control window for worker-related configuration and interaction.

Dependencies. Qt6 or Qt5 Core, Gui, and Widgets, plus `genesys_worker_core`.

Public. Operators and developers who need a graphical worker control surface.

Status. Defined in CMake, but not independently validated beyond the current target and preset evidence.

Evidence. The current build files and presets define the application route and executable name.

Limitations. There is no separate startup or interaction certification yet, so this application should be treated as present but lightly evidenced.

Chapter. Chapter 11.

2.3.2 Data Analyser

The Data Analyser is a specialized Qt window for data-analysis workflows. It is useful when the user wants a dedicated analysis surface instead of the main model editor.

Target. `genesys_dataanalyser_gui_application`.

Executable. `genesys-dataanalyser-gui`.

Purpose. Dedicated analysis frontend for imported or prepared data.

Dependencies. Qt6 or Qt5 Core, Gui, and Widgets, plus the shared kernel, parser, plugins, simulator support/runtime, and tools libraries.

Public. Users who need a dedicated analysis workspace.

Status. Startup validated.

Evidence. The current status ledger records Xvfb window creation, liveness, and controlled teardown.

Limitations. Startup does not establish analysis correctness, workflow completeness, or Level 3 scientific maturity.

Chapter. Chapter 11.

2.3.3 Optimizer

The Optimizer is the optimization-oriented Qt frontend. It is the GUI entry point for optimization work when the user wants a standalone workspace instead of the main editor.

Target. `genesys_optimizer_gui_application`.

Executable. `genesys-optimizer-gui`.

Purpose. Dedicated optimization frontend for the current optimizer scaffold.

Dependencies. Qt6 or Qt5 Core, Gui, and Widgets, plus the shared kernel, parser, plugins, simulator support/runtime, and tools libraries.

Public. Advanced users and developers experimenting with optimization workflows.

Status. Startup validated.

Evidence. The current status ledger records Xvfb window creation, liveness, and controlled teardown.

Limitations. Startup does not prove a selected optimization algorithm, full workflow maturity, or validated scientific output.

Chapter. Chapter 11.

2.3.4 AI Assistant

The AI Assistant is the dedicated Qt frontend for assistant-oriented interaction. It is separated from the main GUI so the assistant workflow can be handled independently.

Target. `genesys_ai_assistant_gui_application`.

Executable. `genesys-ai-assistant-gui`.

Purpose. Dedicated assistant UI for prompt, tool, and response workflows.

Dependencies. Qt6 or Qt5 Core, Gui, and Widgets, plus the shared kernel, parser, plugins, simulator support/runtime, and tools libraries.

Public. Users and developers who need a separate assistant-oriented application surface.

Status. Startup validated.

Evidence. The current status ledger records no-credential Xvfb startup and teardown.

Limitations. Startup does not establish provider integration, credential handling, redaction, or failure workflow maturity.

Chapter. Chapter 11.

2.4 Model-specific applications

Figure 2.3 places the current families on a simple maturity ladder. The intent is to prevent overclaiming: different applications are at different evidence levels, and startup alone is not the same as full workflow validation.

The model-specific application route builds one selected example from `source/applications/modelSpecific`. The current CMake logic treats the selection as a build-time input, so the exact executable contents depend on the selected source file.

Target. `genesys_modelspecific_app`. The selected source file is provided by `GENESYS_MODELSPECIFIC_APP`.

Executable. `genesys_modelspecific_app`. A legacy compatibility copy named `genesys_terminal_application` is created after the build.

Purpose. Build and run one selected model-specific example or teaching application from the model-

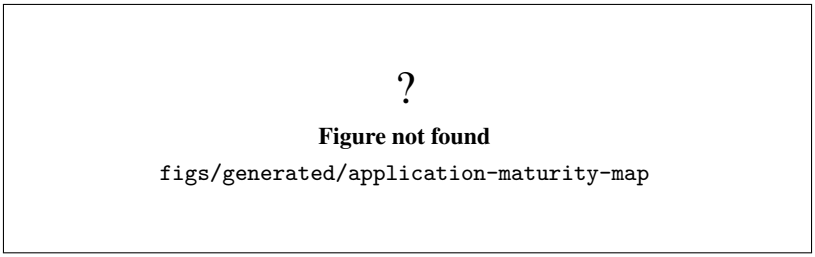


Figure 2.3: Current application maturity and evidence levels.

specific tree.

Dependencies. The shared kernel, parser, plugins, simulator support/runtime libraries, and the selected model-specific source and header files.

Public. Advanced users, instructors, and maintainers who need concrete example flows.

Status. Snapshot only.

Evidence. The current status ledger treats the family as a historical bounded sweep that still needs current revalidation.

Limitations. There is no single fixed executable payload; the selected example determines the runtime behavior, and each selection must be rechecked on the current branch.

Chapter. Chapter 12.

2.4.1 Do Experiments

The DOE route is intentionally not implemented yet. The CMake umbrella knows that the feature is planned, but the repository does not provide a real empty application for it.

Target. None yet.

Executable. None yet.

Purpose. Future design-of-experiments functionality, once a bounded backend and product contract exist.

Dependencies. No implemented application route yet.

Public. None yet.

Status. Not started; intentionally unavailable.

Evidence. The GUI umbrella emits a fatal configuration error when the feature flag is enabled, which confirms that the repository does not ship a placeholder executable.

Limitations. Readers should not expect a launcher, preset, or runnable window for this route until the backend/product definition is completed.

Chapter. No dedicated user chapter yet.

2.5 How to choose

Figure 2.4 turns the application map into a decision tree. The reader should use it as a quick answer to the question "which application should I open first?" without needing to inspect the full source tree. For direct lookup, Section 2.2.1 covers the main GUI, Section 2.2.2 covers the shell, Section 2.2.3 covers the worker, Section 2.3.1 covers the HTTP Worker GUI, Section 2.3.2 covers the Data Analyser, Section 2.3.3 covers the Optimizer, Section 2.3.4 covers the AI Assistant, Section 2.4 covers the model-specific route, and Section 2.4.1 covers the intentionally absent DOE path.

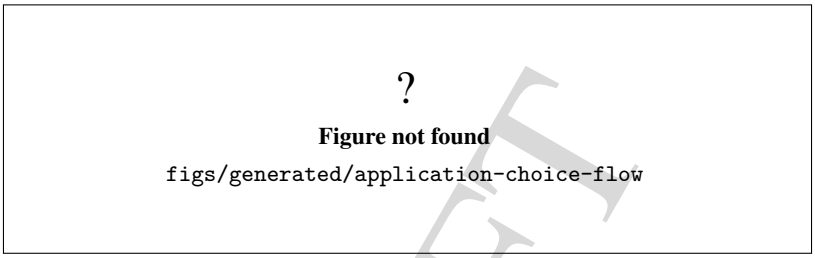


Figure 2.4: Decision flow for selecting the right GenESyS application.

In practical terms, the choice is simple:

- edit or inspect a model visually: open the main GUI;
- run scripts or terminal commands: open the shell;
- serve controlled jobs: open the worker;
- use a helper window for analysis, optimization, or assistant tasks: open the matching standalone frontend;
- run a selected teaching or example model: use the model-specific application route;
- look for DOE: do not expect an application yet.

The rest of the user manual expands each of those choices in the dedicated chapters listed above.

3. The Main Graphical Application

This chapter documents the current `genesys-gui` surface as it exists in the `WorkInProgress` branch at commit `7cc6458da93c5099d7f1f5dda7535f5a761999ea`. It is grounded in the present Qt window layout and in the controllers that now split the main window into narrower responsibilities.

Objective. Describe the real top-level GUI, the visible panels, and the user-facing actions that anchor day-to-day model work.

Audience. Users who start with the graphical interface and need to understand where to load models, edit them, inspect traces, and launch simulation runs.

Scope. Cover the main window layout, menus, toolbars, model trees, property editor, console, result panes, simulation controls, tool actions, and the safe shutdown path.

Status. Partially validated by code inspection and by the current branch structure; visual screenshots are intentionally represented by compilable placeholders until the final images are produced.

The GUI is the main composition root for the desktop workflow. It wires the model lifecycle, scene interaction, trace rendering, property editing, plugin catalog updates, and simulation event handling through dedicated controllers. That makes the window easier to document in slices: the frame itself, the navigation widgets, the editable scene, the trace and results panes, and the simulation configuration path.

The rest of this chapter follows that order through Section 3.1, Section 3.2, Section 3.3, Section 3.4, Section 3.5, Section 3.6, Section 3.7, and Section 3.8.

3.1 Main window layout

The central window is built by `mainwindow.ui` and initialized by `mainwindow.cpp`. The root widget owns the standard Qt menus, the toolbars, the central tab sets, the dock widgets, and the scene/view pair that renders the current model. The central surface is split across:

- `tabWidgetCentral`;
- `tabWidgetModel`;
- `tabWidgetSimulation`;
- `tabWidgetReports`;
- `dockWidgetPropertyEditor`;
- `dockWidgetConsole`;
- the graphical view backed by `ModelGraphicsScene`.

The layout is intentionally split into a model-editing center and supporting inspection panes. The editor area keeps the scene, text model, simulation trace tabs, and report tabs visible as separate work surfaces instead of collapsing everything into one monolithic canvas. That separation matches the current controller split and makes it easier to explain which actions are always available and which are disabled while a simulation is running.

?

Figure not found

`figs/generated/main-graphical-window-annotated`

Figure 3.1: Annotated main GUI window showing the primary workspace and supporting panes.

Figure 3.1 is the anchor for the rest of this chapter. It should be read together with the menus in Figure 3.2 and the scene/editor panes in Figures 3.3, 3.4, and 3.5.

3.2 Menus and toolbars

The top-level menus are the normal entry points for model lifecycle, editing, simulation, tools, and about/help actions. The current menu surface includes Model, Edit, View, Simulation, Tools, and About. The Model menu owns new, open, recent, save, close, information, check, and model-navigation actions. The Edit menu exposes undo, redo, find, replace, clipboard, grouping, and ungrouping actions. The View and drawing toolbars control zoom, grid/ruler visibility, guides, snapping, alignment, and graphical selection behavior.

The toolbar set is arranged as a compact action strip around the main window. The current code adds the model, edit, view, arrange, draw, simulation, graphical-model, animate, and about toolbars to the top and left areas, then locks them down to the top area at runtime so the workspace stays predictable. That is the correct reading of the GUI: the toolbars are a command surface, not independent windows.

Several actions are intentionally state-gated. When no model is open, the GUI disables most editing and simulation controls. When a simulation is active, it locks the edit, view, drawing, and property-editing

paths so the runtime state cannot be damaged by concurrent edits.

?

Figure not found

`figs/generated/main-graphical-menus`

Figure 3.2: Menu bar and toolbar grouping in the main GUI.

3.3 Model trees and property editor

The left-hand navigation and the property editor are where the current model becomes inspectable. The component tree lists model components and reflects the scene selection. The data-definition tree lists the current model data definitions. The plugin tree shows loaded plugins and their catalog entries. The property editor on the right mirrors the selected model object and is kept in sync with scene selection changes and tree selection changes.

The important behavior here is synchronization. Selecting an item in the tree or in the scene should drive the property editor to the same model object. The code keeps that linkage in the composition root and in the property-editor controller rather than leaving it implicit. That is why the chapter should describe the trees as coordinated views of the model, not as independent widgets.

?

Figure not found

`figs/generated/main-graphical-component-tree`

Figure 3.3: Model browsing area with the component and data-definition trees.

3.4 Scene and graphical editing

The graphics view is the primary editing surface for model diagrams. The backing scene is `ModelGraphicsScene`. That scene receives the draw, arrange, zoom, grouping, connection, and visibility commands exposed by the menus and toolbars. The visible editing modes include the normal selection flow and the drawing and animation actions for lines, rectangles, ellipses, polygons, text, counters, variables, queues, stations, entities, events, attributes, statistics, plots, and simulated time.

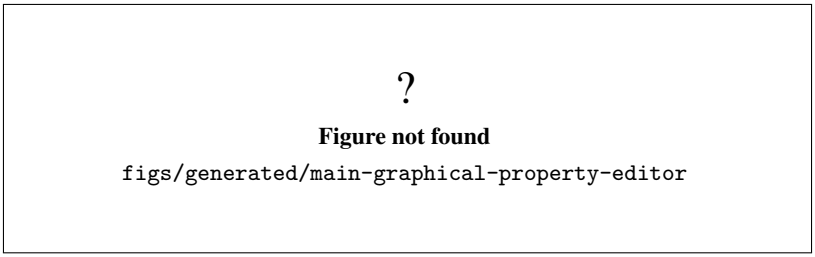


Figure 3.4: Docked property editor synchronized with the current selection.

The chapter should also make clear that the scene is not just a canvas. It is the coordination point for undo, graphical connections, selection, visibility filters, and graphical simulation overlays. That is why the current GUI maintains separate actions for grid, ruler, guides, snap, recursive visibility, attached elements, and editable elements. Those options do not merely change appearance; they change how the model is navigated and edited.

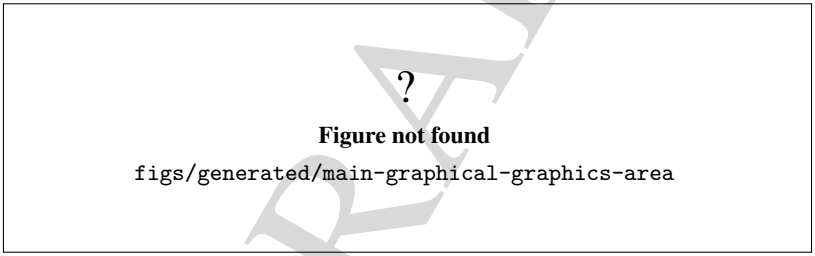


Figure 3.5: Graphical editing area and scene-level interaction aids.

3.5 Trace, console, and reports

The lower and side output panes are where the GUI exposes runtime feedback. The main console receives normal traces and errors. The simulation pane shows simulation-specific traces. The reports pane shows report-oriented output and the summary results that are emitted when the model or runtime suppresses a full automatic report dialog.

This is implemented through the trace bridge controller instead of through ad hoc widget writes in the window class. The practical effect for the reader is that the console, simulation output, and reports output are separate sinks with different meanings. A general trace does not prove a simulation completed, and a report pane does not mean the model was scientifically validated. The chapter should say that explicitly so the interface is not overread.

Figure 3.6 captures the trace-oriented panes, while Figure 3.7 captures the report and result-oriented panes that appear after a run.

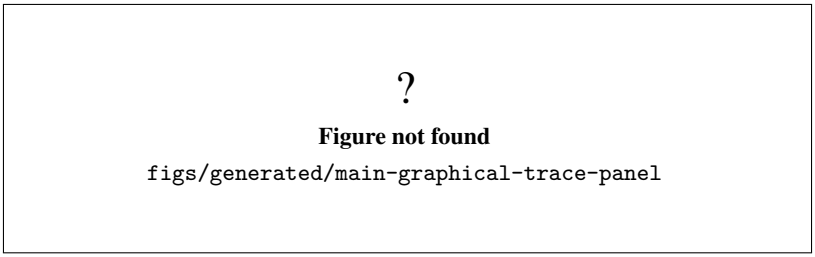


Figure 3.6: Console and trace panes used for runtime feedback.

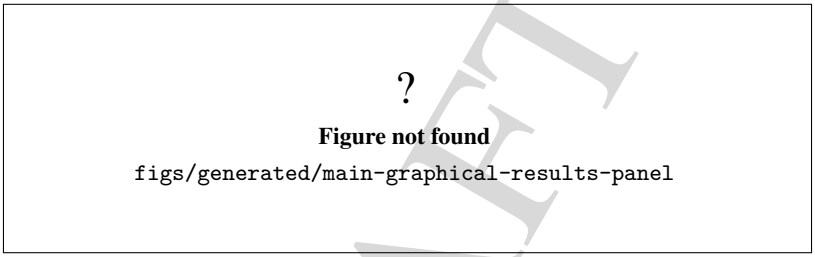


Figure 3.7: Results and reports area after execution.

3.6 Simulation controls and configuration

The simulation menu and the dedicated toolbar expose start, stop, pause, resume, step, and configuration actions. The configuration action opens the simulation configuration dialog, which mirrors the live `ModelSimulation` state and lets the user edit replication counts, time units, warm-up period, termination condition, step-by-step execution, pause behavior, and local parallelization settings. The dialog also carries a distributed-parallelization placeholder button, which is explicitly documented in the code as a future extension rather than as a finished backend integration.

This is the right place to mention the state gating again. The GUI enables the simulation actions only when the current model and runtime state permit them. That means the chapter should describe the controls as available commands with preconditions, not as always-active buttons.

Figure 3.8 shows the dialog surface the user reaches from this command.

3.7 Available, incomplete, and experimental actions

The current GUI exposes a small number of actions that should be documented as present but not necessarily mature. The visible example is the legacy `ToolsExperimentation` slot, which remains in the class but does not currently map to a live QAction in `mainwindow.ui`. That is a compatibility artifact, not a user-facing feature. The chapter should call such items out plainly so readers can distinguish implemented commands from historical wiring and future scaffolding.

More generally, the GUI uses explicit state checks to keep incomplete or dangerous actions disabled when they should not run. The documentation should reflect that contract instead of implying that every

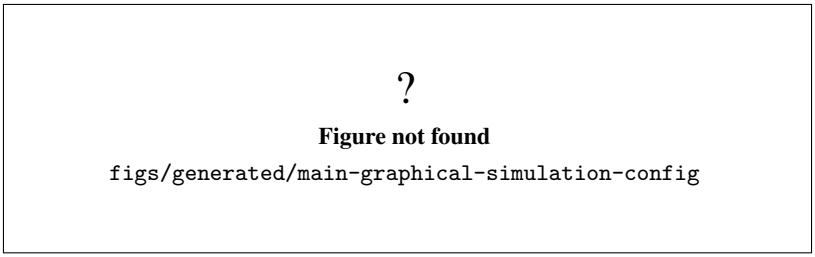


Figure 3.8: Simulation configuration dialog opened from the main GUI.

listed slot is a polished feature. This is especially important for the simulation toolbar, the plugin manager entry point, and the graphical simulation toggle.

3.8 Safe closure

Closing the main window is not a trivial hide-and-exit operation. The current window class coordinates pending-change checks, scene cleanup, trace shutdown, signal disconnection, and model persistence prompts before allowing the application to exit. In practice, that means the user can close the window without losing the current model accidentally, but only after the current dirty state has been reviewed.

The chapter should end by telling the user that the GUI is a coordinated workspace, not a thin shell around a single canvas. The model tree, property editor, scene, trace panes, and simulation dialog all participate in that workspace and are kept consistent by the controller layer.

4. The First Model

This chapter is grounded in the current `WorkInProgress` branch at commit `f6dbc29866dba3e8a7021277229b82e96fb54d52`.

The worked example comes from the checked-in model file `models/AirportSecurityExample.gen`. Its generator is the model-specific source file `AirportSecurityExample.cpp`. Chapter 4 is organized around Section 4.1, Section 4.2, Section 4.3, Section 4.4, Section 4.5, and Section 4.6.

Objective. Walk through a small but complete model from loading to a first comparison run, using only elements that already exist in the repository.

Audience. Readers who have launched the GUI, opened a model before, and now need one reproducible path that can be inspected, executed, and slightly modified.

Scope. Cover the source of the tutorial model, its main components, the initial simulation settings, the first execution, a single controlled change, and the limits of what the example proves.

The model choice is intentionally conservative. It uses the familiar flow of arrival, processing, decision, and disposal. It does not depend on optional subsystems, and it already exists as a repository artifact. That makes it a useful bridge between the GUI overview chapter and the later chapters on model editing and simulation execution.

The chapter follows this sequence: Figure 4.1, Figure 4.2, Figure 4.3, Figure 4.4, Figure 4.5, Figure 4.6, Figure 4.7, and Figure 4.8.

4.1 Why this example

`AirportSecurityExample.gen` is a good first model for four reasons.

- It exists in the repository as a checked-in file.
- Its structure is small enough to read in one session.
- It uses the basic modeling primitives that appear throughout the rest of the user manual.
- It produces a clear before/after comparison when one parameter is changed.

The generated model contains these main elements:

- `Create_1` for passenger arrivals;
- `Process_1` for the security check;
- `Decide_1` for the pass/fail branch;
- `Dispose_1` for cleared passengers;
- `Dispose_2` for denied passengers;
- `Transportation_Security_Officer` as the seized resource;
- `Check_for_Proper_Identification.Queue` as the waiting queue.

The source file that generates the model, shown above, confirms that the example is not invented for the manual. The model is built in code, saved to `models/AirportSecurityExample.gen`, and then executed by the model- specific application.

?

Figure not found

`figs/generated/first-model-conceptual-flow`

Figure 4.1: Conceptual flow of the first tutorial model.

Figure 4.1 is the reference view for the rest of the chapter. Every later property, run setting, and output should be read against this flow.

4.2 Opening the model in the GUI

Open `models/AirportSecurityExample.gen` in the main GUI. After the file loads, the component tree should expose the model objects listed above, and the scene should show a short linear process with one branch to each disposal.

The point of this first load is not to edit anything yet. It is to confirm that the repository model and the GUI agree on the same structure. If the loaded scene does not resemble the conceptual flow in Figure 4.1, stop and recheck the file being opened.

The loaded model should be readable without zoom tricks. The main flow starts at `Create_1`, reaches `Process_1`, and then reaches `Decide_1`. The two terminal branches represent the cleared and denied paths.

?

Figure not found

`figs/generated/first-model-gui-load`

Figure 4.2: Loaded tutorial model in the main GUI.

4.3 Initial properties and simulation settings

The initial model properties are the ones saved in the repository example. They are the values the reader should see after loading the file for the first time:

- arrival distribution: `expo(2)` minutes;
- security delay: `tria(0.75, 1.5, 3)` minutes;
- pass condition: `unif(0, 1) < 0.96`;
- replication length: 12 hours;
- number of replications: 2;
- warm-up period: 0.5 hour;
- trace level: event level.

These are the properties that make the example reproducible as a tutorial. The manual does not invent a separate seed field for this model file. Instead, the reproducible record for a run must include the exact random seed used by the current execution environment, alongside the file name and the simulation settings above.

?

Figure not found

`figs/generated/first-model-properties`

Figure 4.3: Baseline properties for the tutorial model.

Before the first execution, confirm two things. First, the model file name is the one above, not a different saved copy. Second, the simulation settings in the dialog match the saved model values. If either one is different, the run is no longer the baseline version described in this chapter.

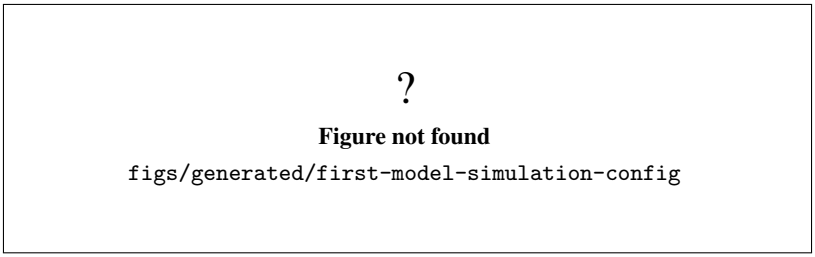


Figure 4.4: Simulation configuration for the baseline tutorial run.

4.4 First execution

Run the model once without changing anything. The expected behavior is simple: passengers arrive, enter the process, seize the officer resource, wait in the queue if needed, and leave through one of the two disposal paths.

Because the tutorial uses stochastic distributions, the exact event order and the exact counts depend on the seed used in the run. That does not weaken the tutorial. It only means that the reproducible record must keep the seed with the model file and the simulation settings.

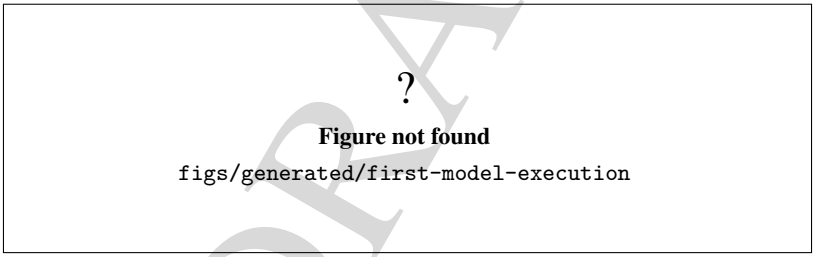


Figure 4.5: Baseline simulation running in the GUI.

At this stage, the main thing to verify is not a specific number but a correct qualitative path: arrivals happen, the security step consumes time, and the branching decision sends some entities to the cleared path and some to the denied path.

The trace and the report together are enough to establish that the example is working. A successful first run proves that the GUI can load the model, the kernel can execute it, and the reporting path can summarize it.

4.5 One controlled change

The simplest useful change is to make the security branch stricter. Change the decision condition from `unif(0, 1) < 0.96` to `unif(0, 1) < 0.90`. That keeps the structure intact but increases the number of entities that fall into the denied path.

The change is deliberately small:

?

Figure not found

`figs/generated/first-model-trace`

Figure 4.6: Trace view for the first baseline execution.

?

Figure not found

`figs/generated/first-model-report`

Figure 4.7: Report summary from the baseline execution.

- no new components are added;
- no connections are rewired;
- only one numeric threshold changes;
- the effect should be visible in the trace and report.

Save the modified model under a new name before rerunning it. That preserves the baseline example and makes the comparison explicit.

?

Figure not found

`figs/generated/first-model-comparison`

Figure 4.8: Comparison between the baseline and the stricter branch threshold.

The comparison should answer one question clearly: did the changed threshold affect the outcome in the expected direction? If the answer is yes, the user has completed a full first-model loop: load, read, run, change, rerun, compare.

4.6 What this example proves

This tutorial proves that the checked-in example model can be loaded and understood as a small simulation system. It also proves that a single parameter change can produce an observable difference in the result path.

What it does not prove is broader than that:

- it does not certify the statistical quality of the chosen distributions;
- it does not establish a universal seed policy for all models;
- it does not replace the later chapters on editing, execution, and results;
- it does not claim the example is a final product model.

That limitation is intentional. A first model should be concrete, readable, and repeatable, not exhaustive. The later chapters build on the same logic with more detailed workflows and richer example structures.

5. Model Editing in the GUI

This chapter turns the GUI from a passive viewer into an editing workspace. The goal is to show a reproducible path for creating or loading a model, changing it, checking it, saving it, and opening it again without losing the structure that was just edited.

The safest way to follow the chapter is to start from a checked-in example model such as `models/AirportSecurityExample.gen`, save a working copy, and make the edits in that copy. That keeps the original file intact and makes the round-trip easier to verify.

The chapter is organized around these steps: Figure 5.1, Figure 5.2, Figure 5.3, Figure 5.4, and Figure 5.5.

Objective. Show how a user can create, connect, edit, validate, save, and reopen a model through the graphical interface.

Audience. Readers who already know the main GUI layout and now need to perform real editing work on a model.

Scope. Cover the component insertion path, connection flow, property editing, duplicate/remove behavior, validation, and the persistence round-trip back to disk.

The implementation used by the GUI is intentionally conservative. The current code exposes model creation, open, save, check, undo, redo, cut, copy, paste, delete, group, ungroup, and replace actions. There is no separate dedicated “duplicate” command in the current GUI path, so this chapter treats duplication as the documented copy-paste workflow.

5.1 Starting from a reproducible baseline

Editing should begin from a stable model state. In practice, that means one of two things:

- create a new model from the GUI template;
- open an existing checked-in model and save it under a new name before editing.

For the purposes of this chapter, the second option is the better baseline. A checked-in example gives the reader a known starting point, a known structure, and a known file to compare before and after the edits.

The relevant GUI entry points are the model lifecycle actions wired in the main window:

- `Model New`;
- `Model Open`;
- `Model Save`;
- `Model Check`.

The same workspace also exposes the property editor, the component tree, the graphical scene, and the current model tabs. That combination is what makes editing possible without leaving the application.

?

Figure not found

`figs/generated/model-editing-workflow`

Figure 5.1: Workspace ready for model editing.

Figure 5.1 is the anchor for the rest of the chapter. Every later edit, connection, validation, and persistence step should be read against the same loaded model.

5.2 Creating components and placing them in the model

Once a model is open, editing begins by inserting or selecting components in the graphical scene. The GUI keeps the scene, the property editor, and the tree views synchronized so that selecting an element immediately exposes its editable fields.

The practical sequence is simple:

1. choose the model or component source from the GUI;
2. place a component in the scene;
3. select the new component;
4. confirm that the property editor follows the selection;
5. repeat until the desired flow exists.

If the user starts from a blank model, the same cycle applies, only with more insertions. If the user starts from the checked-in tutorial model, the cycle becomes a controlled edit to an existing flow.

The main point is that creation is not separate from inspection. A component should be created

and then immediately checked in the property editor, because most modeling errors start as a mismatch between the intended role of the component and its configured properties.

?

Figure not found

`figs/generated/model-editing-connection`

Figure 5.2: Connection between model components.

Figure 5.2 should show the connection workflow as it exists in the scene, not as an abstract diagram. The user should be able to recognize source and destination elements and see that the connection belongs to the current model.

5.3 Editing properties and provoking one controlled error

Property editing is where the model becomes concrete. In the GUI, selecting a component or data definition updates the property editor so the user can change names, numeric parameters, expressions, and other configuration fields.

The recommended sequence is:

1. select one component;
2. change one safe property first;
3. deliberately introduce one invalid property value;
4. run `Model Check`;
5. read the error feedback in the console or dialog;
6. correct the value and check again.

This is not a decorative exercise. The point is to prove that the GUI does more than store the visual layout: it also validates the model before execution. The code path behind `Model Check` calls the model's own `check()` logic, refreshes the graphical data definitions when needed, and marks the model as valid or invalid based on the result.

The chapter does not invent a separate validation mechanism. It uses the same check action that the GUI already exposes.

Figure 5.3 shows the bad state. Figure 5.4 shows the result of asking the application to validate it. Together they prove that the editor and the checker are coupled in the expected way.

5.4 Undo, redo, copy, paste, delete, group, and ungroup

Once a model is editable, the GUI needs to support routine editing operations. The current code exposes them through the edit menu and the scene commands:

- undo;
- redo;

?

Figure not found

`figs/generated/model-editing-invalid-property`

Figure 5.3: A deliberately invalid property value before validation.

?

Figure not found

`figs/generated/model-editing-validation-message`

Figure 5.4: Model validation feedback after a check.

- cut;
- copy;
- paste;
- delete;
- group;
- ungroup;
- replace.

The most important practical point is that copy and paste are the current conservative path for duplication. There is no separate dedicated duplicate action in the code path inspected for this chapter. If a reader wants to duplicate a model element, the supported route is to copy it and then paste it into the same model or a working copy.

Deletion is equally direct. The scene filters the selected items to the ones that can actually be manipulated, and the delete command removes only those items. Grouping and ungrouping are available for layout management, but they do not replace the underlying model relationships.

These commands matter because editing a model is usually a sequence of small corrections rather than one big creation step. A user adds, repositions, duplicates, and removes elements until the flow matches the intended logic.

5.5 Saving and reopening the edited model

After the model is in the desired state, save it and reopen it immediately. That is the fastest way to prove the persistence round-trip.

The GUI accepts a graphical save target with the `.gui` extension and also supports saving a text model to `.gen`. For an editing workflow, `.gui` is the conservative choice because it preserves the graphical state that was just changed. If the user needs a kernel-oriented artifact, the `.gen` route remains available.

The practical verification steps are:

1. save the working copy under a new base name;
2. close or switch away from the model;
3. reopen the saved file;
4. confirm that the scene and properties match the saved state;
5. verify that the model is no longer marked as dirty.

This is the point where the chapter proves persistence rather than just editing. If the model opens with the same structure and the same properties, the GUI round-trip worked.

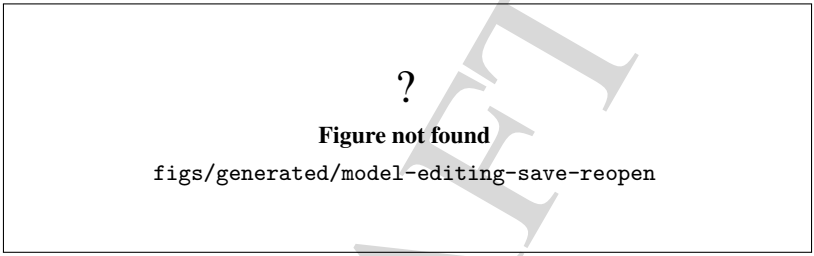


Figure 5.5: Edited model after save and reopen.

Figure 5.5 is the final proof that the editing session was not temporary. If the saved file can be reopened and the change remains visible, the workflow is complete.

5.6 What this chapter proves

This chapter establishes a narrow but useful claim:

- the GUI can create or open a model;
- the model can be edited through the scene and property editor;
- the GUI can expose and report validation problems;
- changes can be saved and reopened;
- the same working copy survives the persistence round-trip.

The chapter does not claim that every optional export path, advanced plugin, or experimental workflow is fully documented here. It focuses on the editing path that is already visible in the current GUI and on the behavior that can be confirmed directly from the repository.

That is enough to move to the next chapters, where the manual starts explaining execution, results, and the more specialized model workflows that build on this editing foundation.

Test your understanding of this content by answering the following questions:

1. Why is it safer to start from a checked-in model and save a working copy before editing?
2. Which GUI actions form the core edit loop in this chapter?
3. Why does the chapter treat duplication as copy and paste instead of a separate command?
4. What proves that the edited model was really saved and reopened?

6. Running Simulations

This chapter documents the current simulation-execution surface of `genesys-gui` as it exists in the repository. The goal is not to invent a new execution model, but to explain the commands, states, and breakpoint surfaces already wired into the GUI and kernel.

Objective. Document the visible steps required to launch, pause, resume, step through, and stop a simulation run.

Audience. Users who already have a model loaded and want to execute it safely.

Scope. Cover the simulation configuration dialog, the run commands, the derived execution states, breakpoint handling, the standard start-to-end flow, and the current limitations of the local execution path.

Status. Partially validated by code inspection. The chapter matches the current GUI labels and kernel methods, but the visual figures are still placeholders until the final screenshots are produced.

The chapter follows this sequence: Figure 6.1, Figure 6.2, Figure 6.4, Figure 6.3, and Figure 6.5.

6.1 Configuring a simulation run

The simulation dialog is opened through `actionSimulationConfigure` and is titled *Configure Simulation*. Its current content is split into three tabs: *Model Simulation*, *Simulation Report*, and *Parallelization*. The first tab contains the operational controls that matter for a local run:

- *Number of Replications*;

- *Replication Base Time Unit*;
- *Replication Length*;
- *Warmup Period*;
- *Terminating Condition*;
- *Initialize system*;
- *Initialize statistics between replications*;
- *Step by step*;
- *Pause on event*;
- *Pause on replication*.

The dialog mirrors the live `ModelSimulation` object rather than a static copy. That matters because the current kernel stores the same execution parameters that the GUI exposes: replication count, replication length, time-unit metadata, warm-up period, terminating condition, and the boolean pause/initialization flags.

The dialog also contains a *Simulation Report* tab. In the current code that tab is read-only from the GUI point of view: it reports whether a `SimulationReporter_if` instance is loaded, but it does not expose editable reporter controls yet.

The *Parallelization* tab should be read conservatively. The local parallelization fields are real GUI state, but the distributed button is a placeholder that only explains the intended future integration path. For this chapter, the supported execution path is the local one.

The chapter does not invent a dedicated seed field here. The current dialog and kernel surface do not expose a direct seed control in this path, so any run record that needs reproducibility must keep the seed from the surrounding execution environment or from the model-generation context.

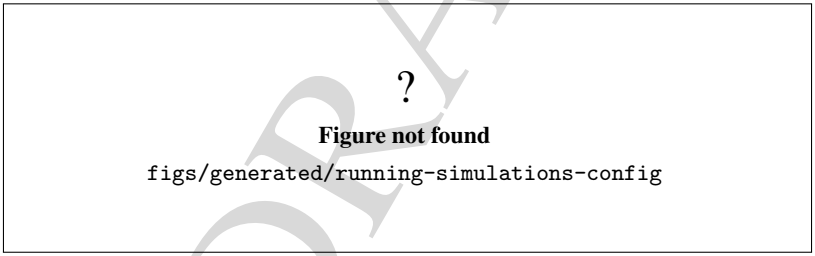


Figure 6.1: Simulation configuration dialog used before execution.

6.2 Simulation states and commands

The current implementation does not expose a dedicated enum for the user-facing simulation state. Instead, the GUI derives the visible state from `ModelSimulation::isRunning()` and `ModelSimulation::isPaused()`, while the kernel transitions are driven by `start()`, `pause()`, `step()`, and `stop()`.

The practical state machine is:

- *Idle or ready*: no run is active yet;
- *Running*: the simulation loop is processing events and replications;
- *Paused*: the run was interrupted by a pause request, a step request, or a breakpoint;
- *Ended*: the simulation completed or was stopped.

The command behavior is conservative and direct:

- `Start` checks the model, initializes the simulation if needed, and then runs replications until the end condition is met;
- `Pause` requests a pause at the next safe point;
- `Resume` reuses the start path while the simulation is paused;
- `Step` requests one event step and then returns to the paused state if the run remains open;
- `Stop` requests termination and clears the paused state when the simulation is already suspended.

The GUI menu and toolbar reflect those same states. While the simulation is running or paused, the code disables structural model editing, most view manipulation, and property editing. That lock is deliberate: the runtime state must not be mutated by concurrent model edits.

?

Figure not found

`figs/generated/running-simulations-state-machine`

Figure 6.2: Effective simulation state machine exposed by the GUI and kernel.

6.3 Breakpoints and pause policies

Breakpoints are exposed separately from the pause buttons. The current kernel supports breakpoints on:

- components;
- entities;
- simulation time.

Those breakpoints are listed in the Breakpoints tab through the `tableWidget_Breakpoints` surface. The table is populated from the live `ModelSimulation` breakpoint lists exposed by `getBreakpointsOnComponent()`, `getBreakpointsOnEntity()`, and `getBreakpointsOnTime()`.

Breakpoint handling is event-driven. When an event matches a component, entity, or time breakpoint, the simulation emits the breakpoint notification, traces a pause message, and suspends the run. The implementation also guards against retriggering the same time breakpoint back-to-back.

The pause-related checkboxes in the configuration dialog are different from breakpoints:

- *Step by step* makes the simulation behave like a one-event loop;
- *Pause on event* requests a pause after event handling;
- *Pause on replication* requests a pause between replications.

That distinction matters. Breakpoints are selective stop conditions. The pause flags are execution policies. The user can combine them, but they are not the same feature.

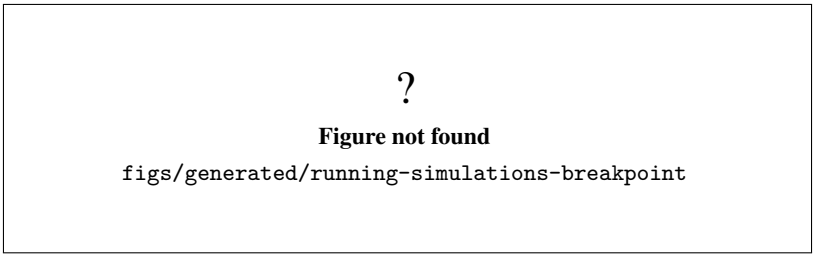


Figure 6.3: Breakpoint list and paused execution.

6.4 Start-to-end execution flow

A clean execution sequence in the current GUI is:

1. load a model;
2. open *Configure Simulation*;
3. confirm the number of replications, replication length, warm-up period, and terminating condition;
4. decide whether step-by-step, pause-on-event, or pause-on-replication should be enabled;
5. start the simulation;
6. observe the trace and runtime feedback;
7. pause, step, or resume if a breakpoint or manual pause is needed;
8. stop the run when the result is sufficient.

The main guarantee here is that the command sequence matches the runtime locking rules. When the run is active, editing surfaces are disabled. When the run is paused, the simulation can still be resumed or stepped. When the run is done, the GUI returns to the normal editing state.

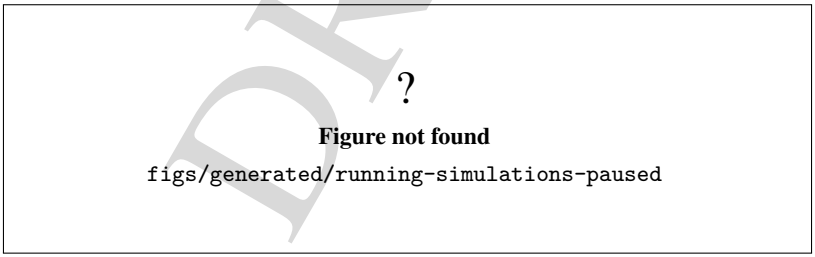


Figure 6.4: Paused simulation with step and resume available.

The chapter's working rule is simple: do not treat a run as correct merely because it started. A correct run is one where the configuration dialog, command behavior, state transitions, and breakpoint behavior all line up with the kernel implementation.

6.5 Current limitations

This chapter intentionally stays within the currently supported local execution surface.

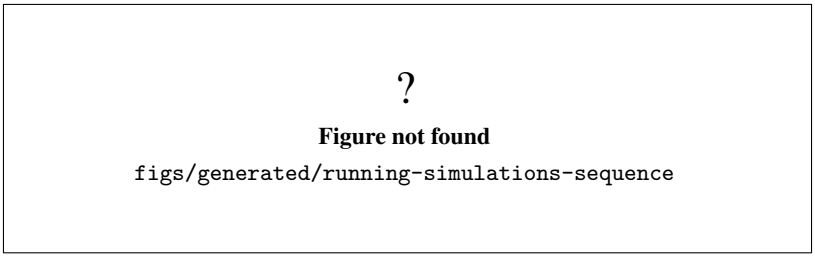


Figure 6.5: Start-to-end simulation execution sequence.

- The distributed parallelization controls are present as GUI structure, but the dialog still treats them as a future path rather than a finished runtime.
- The simulation seed is not exposed as a dedicated control in this dialog.
- The chapter does not claim a universal execution policy for all future experiment abstractions.
- The visual figures remain placeholders until the final screenshots are produced.

Those limitations are not defects in the chapter. They are the boundary between what the current repository actually supports and what the manual should not overstate.

7. Results, Traces, and Reports

This chapter documents the result path that already exists in the repository. It does not invent a new analysis workflow. Instead, it explains how runtime traces, report text, aggregated statistics, and grouped plots are produced by the current GUI and kernel code.

Objective. Explain how users inspect traces, simulation reports, aggregated statistics, and run-to-run comparisons after a simulation.

Audience. Users who need to read simulation output, compare runs, and understand what the reported numbers do and do not prove.

Scope. Cover the trace console, the simulation and reports panes, the aggregated results table, the grouped result plots, the replication reporting flow, and the current limits of interpretation and export.

Status. Partially validated by code inspection. The chapter matches the current widget names, reporter calls, and statistics columns, but the figures are still structural placeholders until the final screenshots are produced.

The chapter follows this sequence: Figure 7.1, Figure 7.2, Figure 7.3, Figure 7.4, Figure 7.5, and Figure 7.6.

7.1 Trace surfaces and report routing

The GUI currently separates three textual destinations for runtime output. The trace controller routes generic traces and fatal or recoverable errors to `textEdit_Console`, simulation-event traces to `textEdit_Simulation`, and report traces to `textEdit_Reports`. That split is implemented in `TraceConsoleController` and wired from the main window.

The report path is explicit in the kernel. The default simulation reporter emits replication and full-simulation summaries. Those report methods write lines with `TraceManager::Level::L2_results`, so the GUI can display them as report output instead of ordinary console chatter.

The simulation pane is for live execution feedback. It shows event-level text, including the lines emitted while a replication is running or advancing through events. The reports pane is for structured summary text, not for free-form debugging.

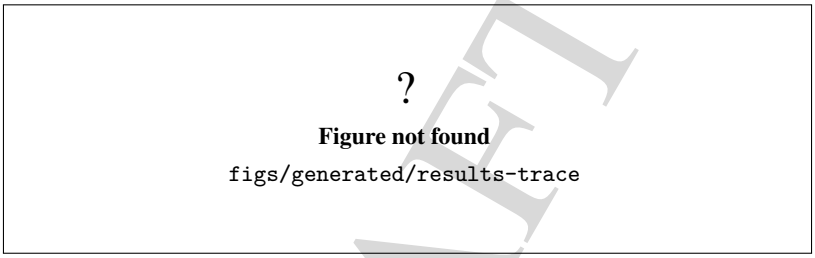


Figure 7.1: Live trace surfaces used while a simulation is running.

As shown in Figure 7.1, the runtime trace surfaces are separated by purpose. That separation matters because a trace line that explains a simulation event should not be read as a final statistical result.

7.2 Replication and simulation reports

The current reporter emits two related textual summaries:

- a replication report, produced after each replication when reporting is enabled;
- a simulation report, produced after the full run finishes.

The replication report starts and ends with explicit markers that identify the replication number. Inside that block, the reporter groups statistics and counters by parent type and parent name, then prints the columns currently used by the kernel: name, element count, minimum, maximum, average, variance, standard deviation, variation coefficient, confidence interval half-width, and confidence level. Counters are printed as counts only.

The simulation report reuses the same grouping logic across the complete run. It shows the same statistic columns, but the numbers are aggregated across the full simulation rather than a single replication. The current code can also show simulation controls and, when enabled, a list of simulation responses.

The chapter stays conservative about the word *export*. In the current GUI, the primary artifact is the rendered report text in the reports pane. For offline result-file analysis, the repository also provides `SimulationResultsDatasetParser` in `source/tools`, which reads text datasets and Genesys `Record` files for analysis outside the main simulation window.

As illustrated in Figure 7.2, the report text is not a free-form log. It is a structured summary, so the surrounding text should be read as statistical reporting rather than event narration.

?

Figure not found

`figs/generated/results-report`

Figure 7.2: Structured replication or full-simulation report in the reports pane.

7.3 Aggregated results table

The aggregated table in the reports area is populated from the current simulation aggregate list returned by the kernel. The GUI configures the table with these columns:

- Type;
- ParentType;
- ParentName;
- Name;
- NumElements;
- Min;
- Max;
- Average;
- Variance;
- StdDev;
- VarCoef;
- HalfWidthCI;
- ConfidenceLevel.

The table is filled from the current simulation's aggregate data definitions. When a row corresponds to a `StatisticsCollector`, the GUI reads the statistics object and prints the full descriptive set. When a row corresponds to a `Counter`, the GUI prints the counter value and leaves the descriptive statistics cells blank. Parent type and parent name are derived from the owning model element so the table can be read in the same hierarchy used by the report text.

?

Figure not found

`figs/generated/results-table`

Figure 7.3: Aggregated simulation results table.

Figure 7.3 is the quickest way to inspect the numeric summary for a run. It should be read together

with the report text, because the table makes the values compact but does not explain the model context by itself.

7.4 Grouped result plots

The plots area is built dynamically inside the `tabReportsPlots` tab. The GUI wraps the content in a scroll area and creates one chart per result category. The charting widget is a custom `ResultsBoxPlotWidget`, not a generic plotting component from an external tool.

The chart grouping is conservative. Items are grouped by parent type. Each item is labeled with either the result name itself or a *parent.name* form when a parent exists. The chart draws the minimum and maximum as whiskers and the average as a central horizontal line. The filled box is a presentation aid; it should not be read as a strict quartile box plot unless the underlying code changes to compute quartiles explicitly.

The result plots are intentionally summary plots. They are useful for spotting wide ranges, missing stability, or obviously skewed responses, but they do not replace a statistical test or a design-of-experiments analysis.

?

Figure not found

`figs/generated/results-plots`

Figure 7.4: Grouped result charts rendered from simulation aggregates.

As illustrated in Figure 7.4, the plot view complements the table view. The table is exact and compact; the plot is visual and comparative.

7.5 Comparing replications

Replication comparison in the current code is primarily textual and summary based. The replication report makes each replication boundary explicit, which lets the user compare the same statistic across consecutive replications when the model and settings are held constant.

If the user needs a file-based comparison, the repository also contains the dataset parser used by the analysis tools. That parser understands Genesys `Record` files and other text datasets with replication metadata, so the results can be inspected outside the simulation run itself.

The important limit is methodological: a replication-by-replication comparison is only meaningful when the runs are comparable. Different seeds, different input data, or different model settings change the interpretation. The chapter therefore treats replication comparison as a descriptive aid, not as proof of equivalence.

Figure 7.5 should be read as a comparison aid only. It does not claim that the reported mean or interval alone proves a global model property.

?

Figure not found

`figs/generated/results-replication-comparison`

Figure 7.5: Comparison of results across replications.

7.6 Data to report pipeline and limits

The current data path is:

1. the simulation kernel updates statistics, counters, and responses;
2. the simulation reporter renders replication or full-simulation summaries;
3. the trace controller routes report text into the reports pane;
4. the GUI fills the results table and grouped plots from aggregate data;
5. the analysis tools can load text datasets and `Record` files for offline inspection.

The pipeline is complete enough to read a real run, but it is not a promise that every metric is automatically exported in a user-friendly file format. The chapter therefore distinguishes three layers:

- live trace output;
- structured report text and aggregate tables;
- offline dataset loading for analysis.

The current limits are equally important:

- the plots are summary visuals, not a statistical inference engine;
- the report text is generated from the current kernel aggregates, not from an external audited data warehouse;
- the chapter does not claim a dedicated one-click export path that is absent from the current GUI.

?

Figure not found

`figs/generated/results-data-flow`

Figure 7.6: Data flow from simulation aggregates to GUI and offline analysis.

As shown in Figure 7.6, the reporting path is layered. That layering is useful because it keeps live runtime feedback separate from the aggregated numbers used for reading and comparison.

8. Model Language for Users

This chapter documents the current user-facing textual representation of a GenESyS model. It does not try to replace the graphical workflow or the later developer-level language reference. Instead, it explains how the textual model view fits into the GUI, how to read a saved model file, and how to keep the text synchronized with the current kernel model.

Objective. Show how the textual model language relates to the graphical model and how a user can read it as a second view of the same system.

Audience. Users who already know the GUI and want to inspect or edit the textual model representation with confidence.

Scope. Cover the current text editor surface, the current line-oriented file structure, the relationship between model text and GUI elements, and the practical limits of this chapter.

Status. Partially validated by code inspection and example model files. The chapter matches the current GUI services (`ModelLanguageSynchronizer` and `GraphicalModelSerializer`) and the current `.gen` file shape, but it intentionally stops short of a full grammar reference.

The chapter follows this sequence: Figure 8.1, Table 8.1, and Figure 8.2.

8.1 What the model language is

The user-facing model language is the textual form of a GenESyS model. It is the same model that appears in the GUI, but serialized as text so it can be read, saved, copied, compared, and in some cases edited directly.

In the current code, the GUI keeps this representation in the `TextCodeEditor` widget inside the model-language tab set. The `ModelLanguageSynchronizer` service rebuilds that text from the current `Simulator` state, and it can also apply textual edits back into the model. The persistence path is handled by `GraphicalModelSerializer`, which saves the textual representation using the established compatibility format.

That means the text is not a separate abstraction with its own independent truth. It is a synchronized view of the current model. If the model changes in the GUI, the text view must be refreshed. If the text is edited, the model must be revalidated before it can be used again.

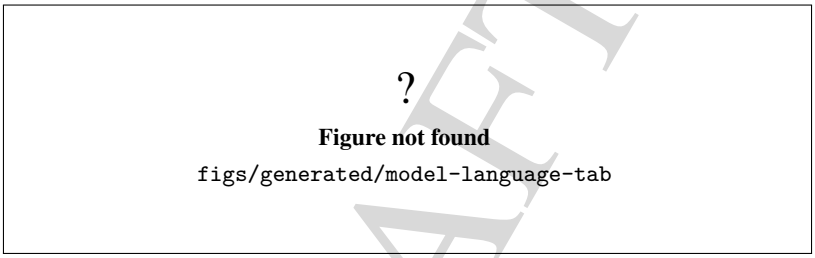


Figure 8.1: Textual model view used by the GUI to represent the current model.

Figure 8.1 should be read as a direct companion to the graphical model, not as a replacement for it.

8.2 How a saved model is organized

A current saved model file is line-oriented and sectioned. One current example is the airport security example model under `models/`. Its `.gen` file starts with comment headers, then lists the simulator and model metadata, and then continues with data definitions and model components.

The exact naming of sections can vary slightly between models, but the current shape is easy to recognize:

- comment lines at the top identify the file as a GenESyS model;
- a header section records simulator, model, and simulation metadata;
- model data definitions appear next;
- model components follow after that.

The format is intentionally explicit. A line typically begins with a numeric prefix, then a type name, then a quoted object name, and then a list of property assignments. For example, the example model file contains entries such as `Simulator`, `ModelInfo`, `ModelSimulation`, `EntityType`, `Resource`, `Queue`, `Create`, `Process`, `Seize`, `Delay`, `Release`, `Decide`, and `Dispose`.

The model text in Table 8.1 is useful because it maps directly to the same conceptual objects the user already sees in the GUI. That one-to-one relation is the main reason the textual representation is worth learning.

Table 8.1: Typical sections of a current GenESyS model text file.

Section	What the user reads there
Comment header	File identity and simple human-readable reminders
Simulator and model metadata	Names, descriptions, version metadata, and simulation defaults
Model data definitions	Entity types, attributes, queues, resources, and other supporting data
Model components	The connected flow elements that form the actual model behavior

8.3 How to read the text as a user

The best reading strategy is to start from the same questions used in the GUI. Instead of trying to understand the whole file at once, ask what each section describes in the graphical model.

In practice:

- the header tells you which model is open and which simulation defaults it uses;
- data definitions tell you what reusable objects the model depends on;
- component lines tell you how the flow is wired;
- expressions tell you what behavior is parameterized rather than hard-coded.

That same example model is a useful reading aid because it is small enough to inspect and still contains all the major current categories. Its first component block shows how text expresses the same model that the GUI diagram represents: the source component, the next component, the delay expression, the queue relation, the resource request, the decision condition, and the alternative exits are all present in text.

For a user, the important habit is to match names and links:

- component names in the file should match the labels seen in the GUI;
- property values should match the properties edited in the property panel;
- connection fields such as `nextId` should line up with the visible flow;
- captions and descriptions should remain readable as plain text.

?

Figure not found

`figs/generated/model-language-flow`

Figure 8.2: How the textual model view maps to the graphical model and back.

Figure 8.2 summarizes the practical reading rule: learn the graphical model first, then use the textual view to confirm and compare what the GUI already showed.

8.4 Editing and synchronization

The current implementation supports a synchronization workflow, not a free-form text editor detached from the simulator. That distinction matters.

When the text is changed, the GUI must apply the text back into the current model and then refresh the rest of the interface. When the model changes in the GUI, the text must be regenerated so the editor does not go stale. The code paths involved in this cycle are already explicit in the repository:

- The sync service rebuilds the text from model state.
- The sync service applies text edits back to the model.
- The serializer writes the current text representation to disk.
- The serializer restores persisted model state.

For the user, the conservative rule is simple: after a direct text change, reload or re-sync the model before relying on the GUI again. After a GUI change, confirm that the text view has been refreshed before assuming the file on disk already reflects the new state.

This chapter does not introduce a separate command line or a new syntax for manual editing. It documents the existing synchronized surface only.

8.5 What this chapter does not cover

This chapter is intentionally not the full language reference. It does not try to define the complete grammar, every expression function, or every parser production. Those topics belong to the later developer-level language chapter.

It also does not claim that every old or future file variant is supported forever. The only safe claim here is about the current repository state: the GUI stores a textual model view, saves it in the established format, and keeps that view synchronized with the kernel model.

If a text excerpt and the GUI appear to disagree, the user should treat that as a synchronization problem to resolve, not as a signal that the text is a better authority than the model or vice versa.

8.6 Final considerations

The main value of the model language for users is readability. It gives the same model a second, formal surface that can be inspected alongside the GUI. That helps with understanding, comparison, review, and troubleshooting, but it only works well when the user remembers that the text and the diagram are two views of one synchronized model.

In the reading sequence used by this manual, the textual model view comes after the GUI, the first model, editing, execution, and results. That order is intentional. By the time the reader reaches this chapter, the model language is no longer an opaque notation. It is a familiar model written in another form.

The next chapter returns to command-line usage for users who need the terminal workflow or want a more scripted interaction style.

9. Shell Usage

This chapter documents the current terminal shell as implemented in `source/applications/shell/`. It is based on code inspection of:

- `GenesysShell.cpp`;
- `GenesysShell.h`;
- `main.cpp`;
- `BaseGenesysTerminalApplication.cpp`;
- the current `genesys_shell` CMake target.

Objective. Document the supported terminal workflow for users who want to inspect a model, load a file, run a simulation, or automate a small sequence of shell commands.

Audience. Readers who are comfortable with a terminal and want a lightweight command surface instead of the GUI.

Scope. Cover build and launch, startup behavior, the current command families, model loading and validation, simulation control, scripted automation, and the current limits of the shell.

Status. Confirmed by code inspection and aligned with the current shell target. The figure blocks are structural placeholders until the final screenshots are produced.

The chapter follows this sequence: Figure 9.1, Figure 9.2, Figure 9.3, and Figure 9.4.

9.1 Build and launch

The shell target is named `genesys_shell`. The dedicated preset is `genesys_shell`, and its build directory is configured under `build/genesys_shell`.

The current build entry points are:

- `cmake -preset genesys_shell`;
- `cmake -build -preset genesys_shell -parallel "$(nproc)"`.

The executable starts by constructing `GenesysShell`, setting the default trace handlers, and then invoking `main(argc, argv)`. During startup, the shell temporarily lowers the trace level, tries to autoload plugins from `autoloadplugins.txt`, restores the internal trace level, and then enters the prompt loop.

If `autoloadplugins.txt` is missing or incomplete, the shell prints an error and continues. The shell still starts; plugin loading failure does not abort the process.

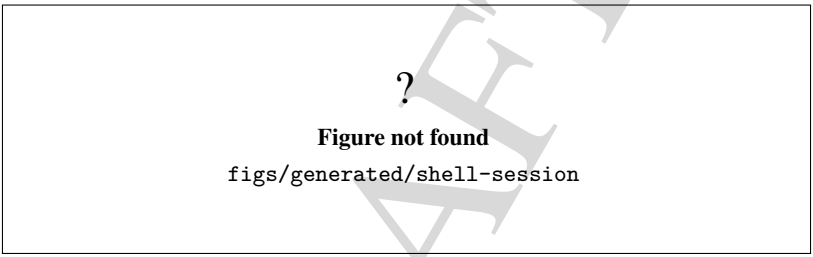


Figure 9.1: Complete terminal session for the Genesys shell.

Figure 9.1 is the reference view for the rest of the chapter. It should show the prompt `$genesys>` and a normal shutdown path.

9.2 Command surface

The shell accepts a single command word followed by optional arguments. The first word is matched case-insensitively; the remaining words are preserved as arguments and rejoined only by the commands that explicitly do so.

The current command families are:

- `help` and `exit`;
- metadata and identity: `facade`, `info`, `licence`, and `trace`;
- plugin management: `plugin`;
- model lifecycle: `model`, `load`, and `save`;
- simulation control: `sim`, `simul`, and `config`;
- inspection helpers: `data`, `component`, and `experiment`;
- automation helpers: `execute-script` and `bash`.

The command families are intentionally conservative. The shell does not invent new semantics on top of the underlying simulator facade; it forwards to the current code paths and prints textual diagnostics when a request cannot be fulfilled.

?

Figure not found

`figs/generated/shell-command-flow`

Figure 9.2: Current Genesys shell command families.

9.2.1 Help and exit

`help` without arguments prints the main command list, split into ordinary commands and the grouped commands that have topic help. The grouped topics currently include `sim`, `config`, `plugin`, `model`, `trace`, `execute-script`, `load`, `save`, `facade`, `licence`, `data`, `component`, and `experiment`.

`help <topic>` expands the selected topic and shows the current subcommand vocabulary. This is the safest way to discover the command surface without reading the source.

`exit` sets the exit flag and returns control to the outer loop. The current implementation prints a quit message and stops the interactive loop.

9.2.2 Metadata, identity, and trace

`facade` reports the simulator name and version metadata through `SimulatorFacade`. The subcommands `get-version`, `get-version-number`, and `get-name` are confirmed in the code.

`info` exposes the current model metadata. The supported actions include reading and updating the model name, analyst name, description, project title, version, and changed flag.

`licence` prints licence details and the current model or runtime limits. The shell supports show, limits, activation-code, the activation-code search and insertion/removal actions, and the model/entity/host/thread limit queries.

`trace` reads or changes the current trace level. The shell accepts `show` and `level <1-9>` and stores the chosen level in the underlying trace manager.

9.2.3 Plugin management

`plugin` inspects installed plugins, prints their count, shows detailed information for a selected plugin, autoloads a plugin list file, displays a language template, validates a candidate library, checks system dependencies, discovers plugin filenames, and navigates the plugin list.

The current shell has a safety gate around dependency installation. When a plugin load path requires system packages, the shell only offers automatic installation when every missing dependency can be installed automatically and stdin is interactive. In non-interactive mode, it refuses to run the install command automatically and asks the user to retry manually.

9.2.4 Model and simulation

The model commands are split between grouped and convenience forms. `model` supports creating, removing, checking, showing, loading, saving, and navigating the current model list. The top-level `load` and `save` commands are convenience aliases for the current model file operations.

`sim` is the main simulation inspection and control command. It can show the current state, start, pause, step, stop, read counters and timing values, change the replication configuration, toggle pause behavior, and read or update reporting options.

`config` is the configuration-oriented command family. It focuses on replication count, replication length, warm-up, base time unit, terminating condition, pause behavior, and the report toggles.

`simul` remains as a compact control alias in the current code. The shell also keeps the lower-level report and breakpoint helpers exposed by `sim` for users who need fine-grained control during debugging.

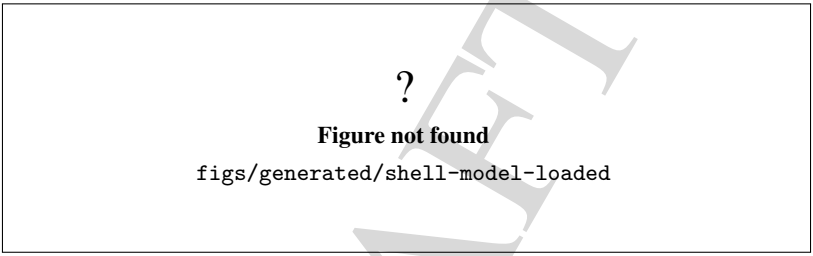


Figure 9.3: Shell session after loading a model file.

Figure 9.3 should be read as the minimum proof that the shell can move from startup into a loaded-model workflow.

9.2.5 Inspection helpers

`data` inspects the current model data definitions. The current code can count definitions, list them by class name, retrieve individual entries, clear the set, and read or update the changed flag.

`component` inspects the current model components. It can count, show, list, clear, search by id or name, move through the list, and update the changed flag.

`experiment` inspects simulation experiments. It can count, show the current experiment, create a new one, save or load an experiment file, and navigate the experiment list.

9.3 Loading, validation, and execution

There are two model-loading paths in the current shell:

- `model load <filename>`;
- `load <filename>`.

Both paths call the current facade loader and print either `Model loaded` or an error message. The file path is used as provided, so the command should be run from a directory where that path resolves correctly.

`model check` validates the loaded model through the facade. The shell prints a notice if the check

reports errors. This is the current validation step for the terminal workflow; it confirms that the loaded model is accepted by the code, not that the scenario is scientifically sufficient.

`sim start`, `sim pause`, `sim step`, and `sim stop` control execution. The `sim show` command prints the current simulation state, including the configured parameters and runtime flags.

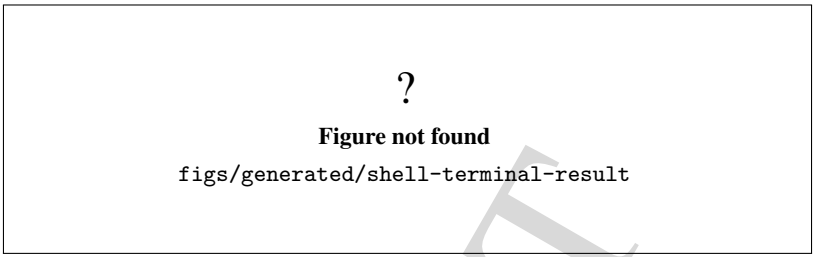


Figure 9.4: Terminal result after a real shell command.

The terminal result in Figure 9.4 should be read as evidence of the current command path. It must come from the real shell output, not from an edited mockup.

9.4 Output, return, and automation

The shell prints its own diagnostics directly to standard output. There is no separate structured return object for command success or failure, so the user must read the text output to understand what happened.

The current process exit path is conservative: `main` returns `0` after the shell loop ends. In practice, this means that command failures are communicated through text rather than by a rich exit-status protocol.

Automation is supported in three ways:

- additional command strings passed on the argv list;
- `execute-script <filename>` for line-oriented shell scripts;
- `bash <command>` for raw host-shell execution.

The argv path executes each argument string as a command string in order. This is useful for compact command sequences, but it is not a full shell parser. When a command line needs spaces, quote the entire string at the outer shell layer or prefer `execute-script`.

`execute-script` ignores blank lines and comment lines that begin with `#`. Each executed line is echoed with the `$execute-script>` prefix before it is routed through the same command parser used by the interactive shell.

`bash` forwards the collected argument text to `system()`. That makes it powerful but also unsafe for untrusted input. It is intended for local automation, not for a security boundary.

Figure 9.5 is the right place to document the scripted path because it proves the shell can be used for lightweight automation without opening the GUI.

9.5 Current limits

The shell is intentionally narrow and has several limits that should be stated explicitly.

?

Figure not found

`figs/generated/shell-automation`

Figure 9.5: Automated shell execution through argv or script input.

- Command parsing is whitespace based; there is no general quoted-argument parser inside the shell itself.
- Many commands expect exactly one path token, so file names with spaces are not a supported pattern.
- Failures are mostly textual and do not map to a detailed exit-status protocol.
- `bash` executes host commands directly and should only be used on trusted local input.
- `plugin` autoload can prompt for dependency installation only when the input is interactive and all missing dependencies are eligible for automatic installation.
- The shell is a command surface over the current simulator code; it does not add a separate policy layer for security, scientific validation, or release approval.

These limits are deliberate. They keep the shell small, predictable, and easy to audit, while leaving stronger policy decisions to the surrounding documentation and governance process.

10. Worker Usage and Security

This chapter documents the current worker application as it exists in `source/applications/worker/`, with supporting evidence from `source/applications/mcp/` and the worker GUI wrapper in `source/applications/gui/httpworker/`. The chapter is based on code inspection of:

- `source/applications/worker/main.cpp`;
- `source/applications/worker/BaseGenesysWebApplication.cpp`;
- `source/applications/worker/http/SimpleHttpServer.cpp`;
- `source/applications/worker/api/ApiRouter.cpp`;
- `source/applications/worker/service/SimulatorSessionService.cpp`;
- `source/applications/worker/service/SimulatorSessionService.h`;
- `source/applications/worker/session/SessionManager.cpp`;
- `source/applications/worker/auth/TokenService.cpp`;
- `source/applications/worker/worker/WorkerJobManager.cpp`;
- `source/applications/worker/README.md`;
- `source/applications/mcp/README.md`.

Objective. Explain how the worker is started, how clients create sessions and use bearer tokens, how model and simulation requests are scoped, how worker jobs behave, and why the current runtime must remain in a trusted local or private-network boundary.

Audience. Operators, integrators, and developers who need to use the worker directly or connect an automation client such as the MCP connector.

Scope. Cover build and launch, public discovery endpoints, session and token handling, model import and

persistence, synchronous worker jobs, transport limits, and the security boundary that currently applies.

Status. Confirmed by code inspection and aligned with the current worker README. The chapter uses structural figure placeholders until final screenshots and diagrams are produced.

The chapter follows this sequence: Figure 10.1, Figure 10.2, Figure 10.3, Figure 10.4, and Figure 10.5.

10.1 Build and launch

The worker target is named `genesys_worker_app`. The executable is `genesys-worker`. The current build entry points are:

- `cmake -preset genesys_worker_app`;
- `cmake -build -preset genesys_worker_app -parallel "$(nproc)"`.

The runtime entry point in `main.cpp` is intentionally thin. It creates `BaseGenesysWebApplication` and forwards `argc/argv` to the wrapper. The wrapper reads two command-line parameters:

- `-port <N>` selects the listening port;
- `-max-requests <N>` stops the server after `N` requests.

The current default port is `8080`. A `-max-requests` value of `0` means unlimited requests.

The worker can be started with:

```
./build/worker-debug/source/applications/worker/genesys-worker --port 8080
```

For smoke testing, the bounded form is convenient:

```
./build/worker-debug/source/applications/worker/genesys-worker \
  --port 8080 --max-requests 1
```

The current worker README documents the same target and executable names. The MCP connector README expects the worker to be reachable through `GENESYS_URL`, typically `http://localhost:8080` during local development.

?

Figure not found

`figs/generated/worker-runtime-topology`

Figure 10.1: Current worker runtime topology.

Figure 10.1 should show the worker as a single HTTP process that owns the transport, routing, session, and simulator layers.

10.2 Endpoint surface

The current router in `ApiRouter.cpp` exposes a conservative endpoint surface:

- `GET /health`;
- `GET /api/v1/worker/info`;
- `GET /api/v1/worker/capabilities`;
- `POST /api/v1/auth/session`;
- `GET /api/v1/simulator/info`;
- `POST /api/v1/models`;
- `GET /api/v1/models/current`;
- `POST /api/v1/models/save`;
- `POST /api/v1/models/load`;
- `GET /api/v1/simulation/status`;
- `POST /api/v1/simulation/config`;
- `POST /api/v1/simulation/run`;
- `POST /api/v1/simulation/step`;
- `GET /api/v1/worker/jobs/{jobId}`;
- `POST /api/v1/worker/jobs/{jobId}/run`;
- `GET /api/v1/worker/jobs/{jobId}/result`;
- `POST /api/v1/worker/models/import-language`.

Only `/health`, `/api/v1/worker/info`, and `/api/v1/worker/capabilities` are public discovery endpoints. The remaining operations require `Authorization: Bearer <token>` and are session-scoped.

The worker capability payload is useful because it makes the current staged contract explicit. The implementation currently advertises:

- session API support;
- session-scoped simulator ownership;
- model creation and persistence;
- simulation status, configuration, synchronous run, and synchronous step;
- distributed-job discovery and polling flags;
- model import support;
- terminal job-result retrieval support.

At the same time, the capability payload also marks `supportsBackgroundExecution = false` and `supportsStreamingEvents = false`. That is the practical boundary: the worker currently executes jobs synchronously and does not provide streaming progress updates.

?

Figure not found

`figs/generated/worker-token-request-flow`

Figure 10.2: Token-protected worker request flow.

Figure 10.2 should make the request chain easy to audit: session creation, bearer token extraction,

session lookup, model and simulation operations, and worker-job lifecycle endpoints.

10.3 Sessions and model scope

The session manager creates one `Simulator` per session and stores each session under a workspace directory rooted at the system temporary directory. The workspace root is:

```
std::filesystem::temp_directory_path() / "genesys_worker_sessions"
```

Each session gets a generated identifier with the `sess_` prefix and a generated opaque access token. The token service creates random hexadecimal strings. The current implementation uses:

- a session identifier built from 12 random bytes;
- an access token built from 24 random bytes.

The token is opaque and random. It is enough to gate the current session API, but it is not production-grade authentication.

The session API allows clients to:

- inspect simulator metadata;
- create a new model;
- inspect the current model;
- save and load a model inside the session workspace;
- read simulation status;
- configure and execute a simulation.

Model persistence is deliberately constrained to the session workspace. The worker accepts only a filename basename through the save/load routes. The service rejects paths that contain separators, parent-directory traversal, or characters outside `[A-Za-z0-9._-]`.

The model-import route is plain-text only. It accepts the current GenESyS model language specification as request body text and rejects blank or invalid specifications.

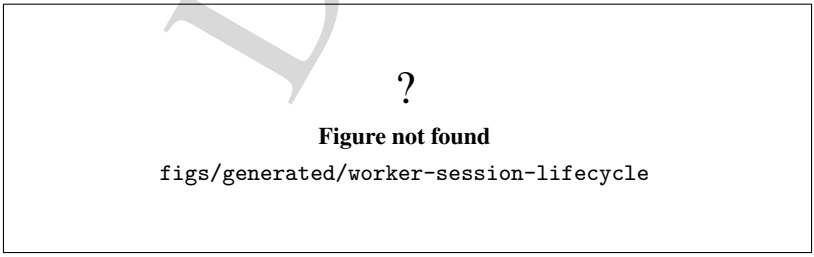


Figure 10.3: Session lifecycle and workspace boundary.

Figure 10.3 should make the session boundary visible, especially the fact that each session owns both a simulator instance and a private workspace directory.

10.4 Worker jobs

Worker jobs are a separate in-memory abstraction layered on top of the session API. They are currently created from the active model in the owning session and stored with a monotonic identifier of the form `job-N`.

The job lifecycle is:

- *Queued* when the job is created;
- *Running* while the synchronous execution is in progress;
- *Finished* after a successful run;
- *Failed* if snapshot loading or simulation execution fails.

When a job is created, the worker persists a snapshot file in the session workspace with the pattern `job_{jobId}.gen`. That snapshot decouples future execution from later model edits in the live session.

The worker also stores a creation marker of the form `seq-N`. That marker is a stable ordering aid, not a public schedule guarantee.

Job access is session-bound:

- `GET /api/v1/worker/jobs/{jobId}` returns job metadata only when the calling token owns that job;
- `POST /api/v1/worker/jobs/{jobId}/run` loads the stored snapshot and runs the job synchronously;
- `GET /api/v1/worker/jobs/{jobId}/result` is available only after the job has a terminal result;
- jobs from another session return access-denied errors.

If a client asks for the terminal result too early, the worker returns a `ResultNotReady` condition. That is intentional: the result endpoint is terminal-state only.

The synchronous run path updates the in-memory job state, loads the saved snapshot, starts the simulation inline, and then stores a minimal terminal summary. The stored result includes simulated time, replication counters, replication length, warm-up period, and the paused flag when available.

?

Figure not found

`figs/generated/worker-security-boundary`

Figure 10.4: Current worker security boundary.

Figure 10.4 should show the worker inside a trusted boundary, not as a public Internet service.

10.5 Transport and security limits

The transport layer is intentionally simple:

- one request per connection;

- no keep-alive;
- no chunked-transfer decoding;
- no TLS;
- bounded request reads;
- fixed socket timeouts.

The server binds with `INADDR_ANY`, so it listens on all interfaces available to the host. That means the code itself does not enforce a loopback-only policy. The intended operational boundary is therefore a local or trusted private network protected by external controls, not a public Internet deployment.

The current transport guardrails are:

- request size limit: 1 MiB;
- socket receive timeout: 5 seconds;
- socket send timeout: 5 seconds;
- `413 Payload Too Large` for oversized requests;
- `408 Request Timeout` when a request is not completed in time.

The router also returns ordinary HTTP error codes for the usual failure conditions:

- `401` for missing or invalid bearer tokens;
- `404` for unknown routes;
- `405` for unsupported methods;
- `409` for operations that require a current model;
- `400` for malformed payloads;
- `500` for unexpected runtime failures.

The worker README is explicit that the current session token is provisional and not production-grade authentication. The same README also states that public Internet exposure is not approved. The manual should preserve that boundary.

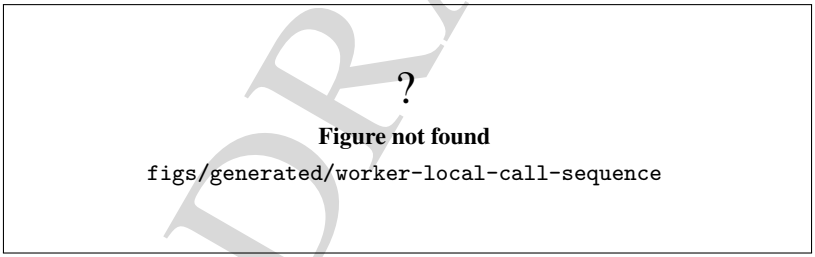


Figure 10.5: Minimal local worker call sequence.

10.6 Recommended local workflow

A conservative local workflow is:

1. start `genesys-worker` on `localhost` or inside a trusted private network;
2. call `GET /health` to confirm the process answers;
3. call `POST /api/v1/auth/session` and keep the returned token;
4. call `GET /api/v1/worker/info` or `GET /api/v1/worker/capabilities` to confirm the worker flavor;
5. create or import a model;

6. save or load the model only inside the session workspace;
7. configure and run the simulation;
8. create a worker job only after the current model is ready;
9. run the job synchronously and retrieve the terminal result after it finishes.

An example local sequence is:

```
curl http://localhost:8080/health
curl -X POST http://localhost:8080/api/v1/auth/session
curl -H "Authorization: Bearer <token>" \
  http://localhost:8080/api/v1/worker/info
curl -H "Authorization: Bearer <token>" -X POST \
  http://localhost:8080/api/v1/models
```

For the MCP connector, the worker address is supplied through `GENESYS_URL`. The connector README assumes that a session token is created once per conversation and then reused by subsequent tool calls.

10.7 Troubleshooting and current limitations

The current implementation is still staged and conservative. The most useful operator-facing constraints are:

- there is no TLS;
- there is no production-grade authentication;
- there is no background job executor;
- there is no streaming job progress feed;
- worker jobs remain session-scoped;
- model save/load is restricted to safe filenames in the session workspace;
- the server does not guarantee loopback-only binding.

If a request fails, the first things to check are:

- the worker is actually listening on the expected port;
- the bearer token came from `/api/v1/auth/session`;
- the token still belongs to the same session as the job or model;
- the request body is plain text for model import and JSON-with-filename for save/load;
- the model exists before configuring or running simulation;
- the job belongs to the same session before polling or running it.

Common failure signatures are:

- `401 UNAUTHORIZED`: missing, malformed, or expired token;
- `400 BAD_REQUEST`: invalid request body or malformed payload;
- `409 NO_CURRENT_MODEL`: no active model exists yet;
- `413 PAYLOAD_TOO_LARGE`: transport limit exceeded;
- `408 REQUEST_TIMEOUT`: the request did not arrive in time;
- `404 NOT_FOUND`: route mismatch;
- `405 METHOD_NOT_ALLOWED`: wrong HTTP method.

The chapter deliberately does not promise public deployment readiness. The current worker is a controlled application service for local or private use until the security backlog items are completed and validated.

11. Standalone Tools

This chapter collects the current standalone tools that sit beside the main GUI and the shell. Here, *standalone tools* means auxiliary front ends and support entry points, not the core kernel classes themselves.

Objective. Present the standalone tools that accompany GenESyS and explain how they fit the user workflow.

Audience. Users who need auxiliary tools for analysis, optimization, or specialized workflows.

Scope. Keep the chapter limited to user-facing tools, their intended purpose, and the level of support currently documented. The chapter describes the current build surface conservatively and does not claim full end-to-end certification for every helper window.

Status. Partial documentation, grounded in the current CMake targets and source tree.

11.1 Current standalone front ends

The current repository exposes four standalone Qt front ends that share the same backend libraries as the main application family. Figure 2.2 in Chapter 2 already maps them at a high level; this chapter explains what each one is for.

- **HTTP Worker GUI.**
  Target. `genesys_httpworker_gui_application`.
  Executable. `genesys-httpworker-gui`.

Purpose. A control dialog for the worker runtime. It is intended for worker-related configuration and interaction, not for general model editing.

- **Data Analyser.**
  Target. `genesys_dataanalyser_gui_application`.
  Executable. `genesys-dataanalyser-gui`.
  Purpose. A dedicated analysis window for inspecting simulation result data without opening the full model editor.
- **Optimizer.**
  Target. `genesys_optimizer_gui_application`.
  Executable. `genesys-optimizer-gui`.
  Purpose. A specialized window for optimization-oriented workflows.
- **AI Assistant.**
  Target. `genesys_ai_assistant_gui_application`.
  Executable. `genesys-ai-assistant-gui`.
  Purpose. A graphical assistant front end for AI-assisted workflows.

These helpers are built as separate targets in the GUI umbrella, and each one pulls only the back-end libraries it needs. In practice, that separation keeps the main editor lighter and makes the auxiliary workflows easier to reason about.

11.2 Support-package entry point

The repository also contains the `source/tools` package, which is a support layer for analysis and numerical workflows. Its small command-line entry point prints the available tool interfaces with `-list` or `-help`. The output currently covers:

- analysis interfaces such as `DataSet_if` and `DataAnalyser_if`;
- fitting and hypothesis-testing interfaces;
- distribution abstractions;
- numerical support interfaces, including solver, quadrature, root-finding, and ODE contracts;
- optimization and factorial-design helpers.

That entry point is mainly a diagnostic inventory for developers and power users. It confirms what the package exports, but it is not the primary route for everyday model authoring or simulation work.

11.3 Usage boundary

When the task is interactive model editing or execution, the main GUI remains the first choice. When the task is scripted execution, use the shell. When the task is worker control, analysis, optimization, or AI-assisted helper flows, choose the corresponding standalone tool instead of forcing the main GUI to do everything.

The practical rule is simple: select the smallest tool that matches the task. That keeps the workflow clearer for users and keeps the documentation aligned with the current build layout.

12. Model-Specific and Advanced Examples

This chapter documents the current model-specific application route as it exists in the repository today. It does not invent a generic "advanced examples" story. Instead, it explains how the build selects one concrete example, how the legacy compatibility path behaves, which example families exist in the tree, and which examples were actually reproduced during this revision.

Objective. Explain the model-specific build route, show the current example catalog, and document a small set of reproducible advanced examples.

Audience. Advanced users, instructors, and maintainers who need concrete example flows beyond the introductory user chapters.

Scope. Cover the current CMake selection mechanism, the legacy compatibility alias, the example catalog under `source/applications/modelSpecific`, the reproduced example outputs, and the maturity boundaries of the remaining historical examples.

Status. Partially validated against the current branch. Two examples were reproduced successfully in this revision, while one historical smarts example still shows a runtime crash and is therefore documented as a limitation rather than a supported flow.

The chapter follows Figure 12.1, Figure 12.2, Figure 12.3, Figure 12.4, and Figure 12.5.

12.1 How the build selects one example

The model-specific route is build-time selected. The current CMake logic in `source/applications/modelSpecific/CMakeLists.txt` accepts a relative source path through `GENESYS_MODELSPECIFIC_APP`. The optional `GENESYS_MODELSPECIFIC_APP_CLASS` and `GENESYS_MODELSPECIFIC_APP_HEADER` variables allow the build to override the class name or header name when the source stem does not match the actual symbols. The deprecated aliases `GENESYS_TERMINAL_EXAMPLE` and `GENESYS_TERMINAL_EXAMPLE_CLASS` remain accepted for compatibility, but they map back to the newer names and are documented as legacy inputs.

The build target is `genesys_modelspecific_app`. After the executable is built, the post-build step creates a compatibility copy named `genesys_terminal_application`. That copy preserves the older launch name for workflows that still expect it, but the canonical target is the newer model-specific one.

The selected source must stay inside `source/applications/modelSpecific`, must end in `.cpp`, and must have a matching header file. The selected binary links the shared kernel, parser, plugin, simulator-support, and simulator-runtime libraries, so the runtime behavior depends on the exact source file chosen at configure time.

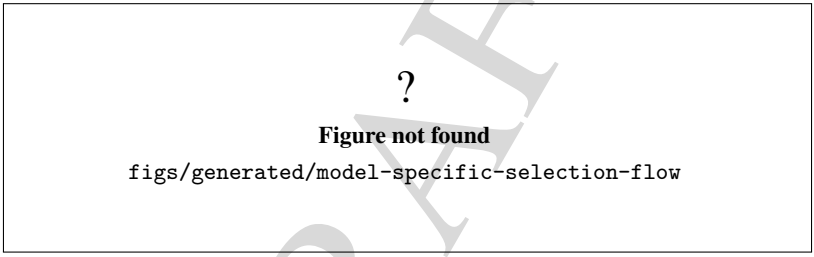


Figure 12.1: Build-time selection flow for the model-specific executable.

Figure 12.1 is the key to the rest of the chapter. If a reader understands the selection flow, the rest of the example catalog becomes easier to interpret: the examples are not interchangeable runtime plugins, but separate source files chosen by the build.

12.2 Current example catalog

The current tree contains multiple families of model-specific applications. The practical split is by teaching style and maturity:

- **arenaExamples** are the clearest user-facing examples. They tend to save a `.gen` file and then run the model immediately.
- **smarts** are compact capability exercises. Some save a model, some exist mainly to exercise a feature path, and some are still historical snapshots that need revalidation.
- **teaching** and **wcm** examples are present in the tree and serve more specialized educational or scientific purposes, but they are not the focus of this chapter.

The override file `source/applications/modelSpecific/modelSpecificApps.tsv` captures special cases that the build script should treat conservatively. In this revision it records the modal-model examples that need class or header adjustments and distinguishes generators from validation-only exercises.

?

Figure not found

`figs/generated/model-specific-catalog-matrix`

Figure 12.2: Current model-specific catalog grouped by family and maturity.

Figure 12.2 should be read as a status map, not as a promise that every row has the same support level. The family buckets exist because the source tree itself is already organized that way.

12.2.1 Reproducible generator examples

Two examples were reproduced successfully in this revision with the helper script `scripts/regenerate-models-from-modelspecific.sh` and the `genesys_modelspecific_app` preset:

- **`arenaExamples/AirportSecurityExample.cpp`.** This example generated `./models/AirportSecurityExample.gen`. The runtime built a small flow around `Create`, `Process`, `Decide`, and `Dispose`, then completed two replications of a 12 hour run.
- **`arenaExamples/Banking_Transactions.cpp`.** This example generated `./models/Banking_Transactions.gen`. The runtime completed three replications of an 8 hour run and exercised a richer mix of `Station`, `Enter`, `Route`, `Seize`, `Delay`, and `Release`.

Both examples are useful because they show a real advanced-user boundary: the model-specific executable is not a generic launcher, but a concrete generator that saves a model file and then runs it immediately.

?

Figure not found

`figs/generated/model-specific-airport-security-example`

Figure 12.3: Airport security example reproduced from the model-specific tree.

Figure 12.3 is the simpler of the two reproduced cases. It is a good first advanced example because it confirms that the model-specific route can generate a concrete model, save it, and run the saved model without any additional user wiring.

Figure 12.4 shows a richer example than the airport flow. It is useful for teaching because it combines multiple customer classes and several station transitions, which makes the build-time selection rule feel concrete rather than abstract.

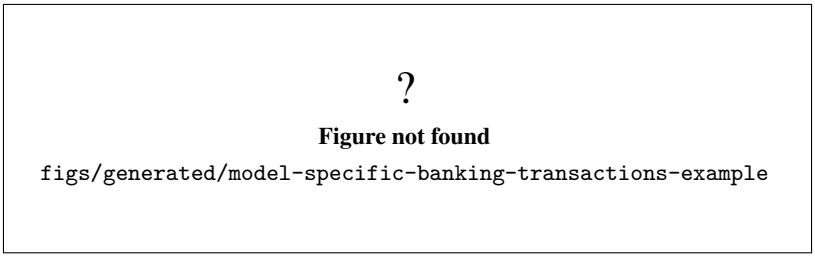


Figure 12.4: Banking transactions example reproduced from the model-specific tree.

12.2.2 Historical smarts and maturity boundaries

The modal-model smarts are still present in the tree and are worth keeping in the chapter because they exercise the selection overrides described earlier. However, they do not all have the same maturity.

- `smarts/ModalModel/Smart_Modalmodel.cpp` saves `./models/Smart_ModalModel.gen` and uses `ModalModelDefault`, but the current validation run ended with a segmentation fault during simulation. That makes it a snapshot example, not a supported advanced flow yet.
- `smarts/ModalModel/Smaty_DefaultModalModel.cpp` is the historical stem-typo case that resolves to the `Smart_DefaultModalModel` class through header/class overrides. It remains in the catalog because the build logic still needs to support that legacy naming pattern.

The practical lesson is simple: a source file being present in the model- specific tree does not automatically mean it is mature enough to recommend without qualification.

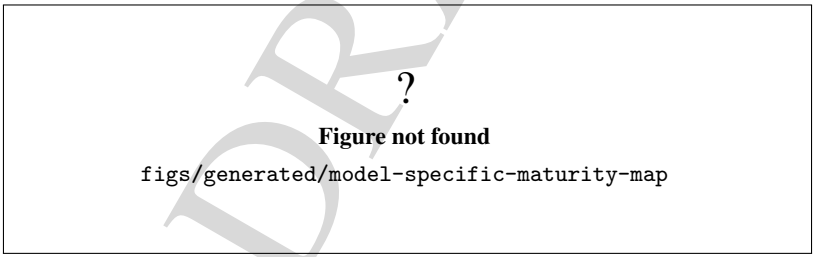


Figure 12.5: Current maturity map for model-specific examples.

Figure 12.5 is the guardrail for readers. It separates "present in the tree" from "reproduced and recommended", which is the distinction that matters for a practical manual.

12.3 Usage boundary

Use the model-specific route when you want a concrete compiled example rather than the general shell or GUI front ends. The route is a good fit for:

- teaching a specific workflow around a known model;
- validating a model-specific source file against the current branch;

- regenerating a `.gen` file from a concrete example source;
- checking whether a historical example still runs on the current code.

Do not treat this route as a general plugin loader or as a guarantee that all example files in the tree are equally maintained. The current validation sweep shows that some examples are stable enough to reproduce, while others still need repair or revalidation before they can be recommended.

In short: choose a model-specific example when the source itself is the thing you want to compile and run. Choose the GUI or shell when the task is general model editing or interactive simulation work.

13. Troubleshooting and Known Limitations

This chapter documents the current troubleshooting surface of the manual as it exists in the live repository. The text is grounded in the current GUI shell, model lifecycle, parser diagnostics, plugin manager, worker boundaries, and issue-reporting flow. It intentionally stays conservative: when the code gives only a generic failure message, this chapter says so instead of inventing a more specific diagnosis.

Objective. Help readers identify the subsystem behind a failure, find the current diagnostic entry points, and understand which limitations are real current-state boundaries rather than temporary editorial gaps.

Audience. Users, instructors, operators, and developers who need a single place to check common failure modes and current support limits.

Scope. Cover build and startup problems, display issues, model opening and validation failures, parser and expression errors, plugin loading issues, worker/tool boundaries, log locations, reporting paths, and limitations that should be stated explicitly before release.

Status. Confirmed by code inspection and aligned with the current repository documentation. The chapter uses structural figure placeholders until the final screenshots and diagrams are produced.

The chapter follows this sequence: Figure 13.1, Figure 13.2, Figure 13.3, and Figure 13.4.

13.1 How to triage a failure

The first question is always which subsystem failed:

- build or configure failure: the toolchain, preset, or dependency set is usually wrong;
- startup failure: the runtime environment or display backend is usually the problem;
- model open or check failure: the model file, its companion layout file, or a component validator is usually the cause;
- expression failure: the parser grammar or an invalid symbol reference is usually the cause;
- plugin failure: the plugin file or its current system dependencies are usually the cause;
- worker or helper-tool failure: the current scope is often a partially validated boundary rather than a complete workflow contract.

When in doubt, start with the current console, simulation, and report panes, then inspect the process log described later in this chapter. The manual favours specific evidence over general reassurance.

?

Figure not found

`figs/generated/troubleshooting-decision-tree`

Figure 13.1: Current troubleshooting decision tree.

Figure 13.1 should be the quickest way to route a new problem to the right subsystem.

13.2 Build and startup problems

13.2.1 Toolchain or preset mismatch

Symptom. Configuration or build stops before the application starts.
Likely cause. The current CMake preset does not match the available toolchain, Qt6 setup, or build directory state.
Diagnosis. Re-run the exact preset from the repository root and compare the reported compiler, CMake, and Qt versions against the current baseline in the repository README and status documents.
Recovery. Use the canonical preset for the target you are trying to build, then rebuild from a clean configuration directory if the previous cache was produced with a different toolchain.
Current limitation. A successful configure does not prove that the runtime workflow is healthy; it only proves the selected preset configured.

13.2.2 No display or invalid display backend

Symptom. The GUI does not appear, exits immediately, or reports a Qt display connection failure on a headless host.
Likely cause. There is no usable display server for the current session.
Diagnosis. Check whether the session has an interactive display. For headless validation, use a private Xvfb or equivalent virtual desktop and keep TCP listening disabled.
Recovery. Relaunch the GUI in a valid graphical session, or reproduce the launch under the same virtual-desktop setup used by the local runbook.
Current limitation. Startup under Xvfb is only startup evidence; it is not proof of end-to-end interaction or scientific correctness.

The visual summary of these two failure classes appears in Figure 13.2.

?

Figure not found

`figs/generated/troubleshooting-error-examples`

Figure 13.2: Representative current error messages.

13.3 Model and parser problems

13.3.1 A model does not open

Symptom. Opening a model ends with the generic warning that the model could not be opened.
Likely cause. The selected file is unreadable, not a valid model file, or does not match the companion graphical layout expected by the open flow.
Diagnosis. The current open flow prefers a companion `.gui` file when a `.gen` file is selected. If the open action fails, inspect the exact file that was chosen and check whether the matching layout file is present.
Recovery. Reopen the companion `.gui` when it exists, or reopen a known-good model from the `models/` tree and confirm that the current file path is valid.
Current limitation. The dialog message is intentionally generic; the actual cause has to be recovered from the file choice and the console trace.

13.3.2 Model check fails

Symptom. The model check dialog reports failure instead of the success message.
Likely cause. A component validator rejected a parameter, a connection, or a model rule that is checked during the current model validation pass.
Diagnosis. Inspect the console. The current dialog only tells the user that the model has errors and to see the console for more information.
Recovery. Fix the component or property that produced the console message, then run model check again.
Current limitation. Model check success only proves the current validation pass succeeded; it does not prove that the model is scientifically validated or that every future scenario is safe.

13.3.3 Parser or expression errors

Symptom. Expression evaluation in the parser checker or a property editor fails and the console shows an error line instead of a numeric result.
Likely cause. The expression does not match the current grammar, a symbol is missing from the current model context, or the parser returned a syntax error.
Diagnosis. Use the parser grammar checker dialog and the console output. The checker currently reports either the real parser message or a fallback `Unknown parser error.` message when no more

specific detail is available.

Recovery. Rewrite the expression to match the grammar currently implemented in the parser, then evaluate it again in the same model context.

Current limitation. The parser diagnostics are helpful but not a full formal proof of semantic correctness for a model.

13.4 Plugins, worker, and helper tools

13.4.1 Plugin load or dependency problems

Symptom. The plugin manager shows blocked plugins, or the shell reports that auto-loading plugins from `autoloadplugins.txt` failed.

Likely cause. The plugin file is missing, incomplete, structurally invalid, or still depends on system packages that are not installed.

Diagnosis. Use the plugin manager's loaded and problem tabs, then check the current dependency preflight and the system package state. The shell starts even when the auto-load list is not usable, so a plugin warning alone does not imply a fatal application failure.

Recovery. Load the plugin list that matches the current checkout, or resolve the blocked plugin dependencies through the plugin manager if the system package prerequisites are available.

Current limitation. Plugin loading still follows the current static build graph and the existing deployment/search contract; do not assume a future dynamic-plugin boundary or a fully automatic recovery path.

13.4.2 Worker reachability and scope

Symptom. The worker does not answer, or a worker-based workflow cannot complete from outside the local machine.

Likely cause. The worker is not running, is bound outside the expected local test setup, or is being used outside the trusted boundary currently documented by the repository.

Diagnosis. Confirm the worker target and launch path first, then inspect its local reachability from the same machine or private network segment used for development.

Recovery. Use the current local/private deployment pattern only. Do not try to turn this into a public service as a troubleshooting shortcut.

Current limitation. The worker is not approved for direct public Internet exposure, and its full security profile is not yet the supported production profile.

13.4.3 Standalone tools that start but do not complete a workflow

Symptom. A helper window opens, but the intended workflow does not look complete or reproducible yet.

Likely cause. The tool is only startup-validated, or the specific analysis/optimization/AI workflow is still outside the fully validated scope.

Diagnosis. Distinguish startup from workflow validation. Check whether the current tool is one of the standalone helper front ends or a partially validated workflow behind them.

Recovery. Reduce the problem to the smallest current supported path and capture the exact inputs, outputs, and logs that were produced.

Current limitation. Startup evidence does not prove complete functional or scientific correctness for the helper tools.

Figure 13.3 summarizes the current diagnostic path across these subsystems.

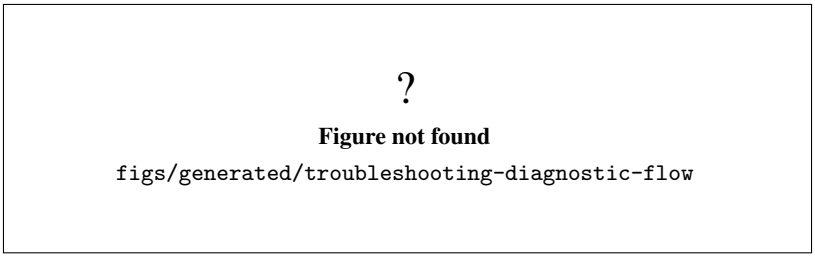


Figure 13.3: Subsystem-oriented diagnostic flow.

13.5 Logs and reporting

The current GUI application installs a log file in the working directory named `crashesAndLogs.log`. The application appends Qt message output and startup diagnostics to that file as soon as the GUI process starts.

The issue-report workflow in the About menu can attach the tail of that log file together with the current console, simulation, reports, and optionally the current model text. That makes `crashesAndLogs.log` the first file to check when a GUI problem needs to be reported or reproduced.

For terminal workflows, capture the exact command and the full terminal output instead of relying on memory. For model problems, also capture the model file that was opened and whether the companion `.gui` file was present.

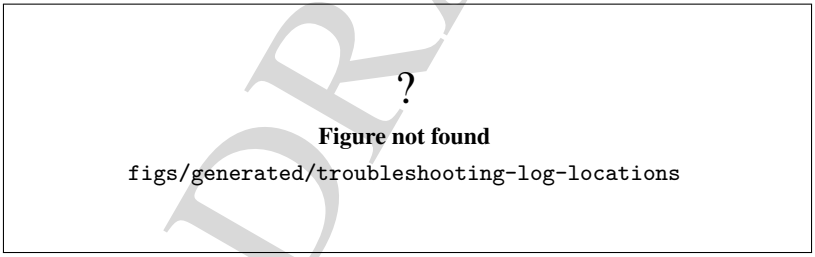


Figure 13.4: Current log and reporting locations.

13.6 Known limitations

The following boundaries are current state, not promises of future support:

- startup does not prove complete workflow correctness;
- helper-tool startup does not prove the associated analysis or optimization workflow is scientifically validated;
- model validation failures are reported through generic dialogs with console details, not through a fully structured problem taxonomy;

- the shell auto-load contract around `autoloadplugins.txt` remains a current deployment and fallback gap;
- the worker must remain in the current local or private-network boundary;
- numerical, statistical, and biological results must still be treated as bounded evidence unless they are explicitly validated elsewhere;
- a reported problem should be read against the current code and status, not as a guarantee of future support policy.

This chapter exists so the manual can say, clearly and conservatively, where the current repository stops today.

14. Development Environment and Build

This chapter describes the current build surface for GenESyS as it exists in the repository now. It is based on the active root README, the current `CMakePresets.json`, the local-agent runbook, and the focused sanitizer path that already exists in `source/tests/`. The chapter does not invent a release workflow that the repository does not currently expose.

Objective. Describe the environment, presets, target families, and validation paths used by developers building GenESyS from source.

Audience. Contributors, maintainers, and anyone who needs to configure, build, and check the project locally.

Scope. Cover the toolchain baseline, the current preset matrix, separate build trees, debug and sanitizer posture, and the current cleanup/incremental-build expectations.

Status. Confirmed by repository inspection and local command execution on 2026-07-23. The chapter records the current build surface rather than a hypothetical future release configuration.

The chapter follows this sequence: Figure 14.1, Figure 14.2, Figure 14.3, and Figure 14.4.

14.1 Baseline toolchain

The documented platform baseline is Ubuntu 24.04 with `CMake` 3.24 or newer, `Ninja`, `C++23` with compiler extensions disabled, and Qt6 for the graphical applications. The current `CMakePresets.json` en-

codes that policy directly by requiring `CMAKE_CXX_STANDARD 23`, `CMAKE_CXX_STANDARD_REQUIRED ON`, and `CMAKE_CXX_EXTENSIONS OFF`.

The local validation snapshot captured the following versions:

- `cmake` 3.28.3;
- `ninja` 1.11.1;
- `g++` 13.3.0;
- Qt 6.4.2 via `qtpaths6 -qt-version`.

The repository root README still describes the intended baseline as `CMake` 3.24 or newer, `Ninja`, `C++23`, Qt6, and the configured Google Test fallback. That is the policy baseline; the versions above are the observed local environment used for this chapter revision.

14.1.1 Why the build surface is split

GenESyS is not built as one monolith with a single universal preset. The current preset matrix separates tests, shell, worker, GUI, and model-specific targets into distinct build trees so that each workflow can be configured and rebuilt independently.

This separation matters because different development tasks need different dependencies and different validation scopes. A shell or model-specific build does not need the same GUI surface as `gui-app`; a focused unit or sanitizer check does not need the entire application set.

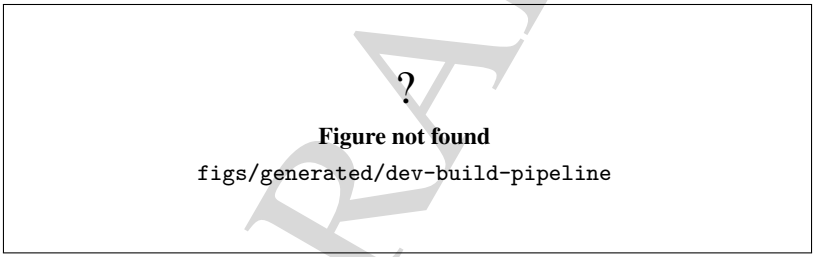


Figure 14.1: Current configure, build, and test pipeline.

Figure 14.1 is the canonical view of the local developer workflow. It should make it obvious that configure, build, and test are separate steps and that the test step only exists where the preset family defines it.

14.2 Current preset matrix

The current `cmake -list-presets=all` output is grouped into configure, build, and test presets. The configure presets are:

- `tests-unit`;
- `tests-kernel-unit`;
- `genesys_shell`;
- `terminal-smart`;
- `terminal-model-specific`;
- `genesys_modelspecific_app`;
- `terminal-smart-hold-search-remove`;

- `terminal-smart-ai-assistant`;
- `worker-app`;
- `genesys_worker_app`;
- `gui-app`;
- `gui-httpworker`;
- `gui-dataanalyser`;
- `gui-optimizer`;
- `gui-ai-assistant`;
- `tests-smoke`.

The build presets mirror the same names, with a special case for `tests-kernel-unit-run` as the command-side alias for the kernel unit test runner. The test presets are currently limited to `tests-unit`, `tests-kernel-unit`, and `tests-smoke`.

All named configure presets currently inherit `debug-base`. That means the current named surface is debug-oriented. There is no dedicated release preset in the current inventory, so release builds must be introduced and revalidated explicitly if they are needed later.

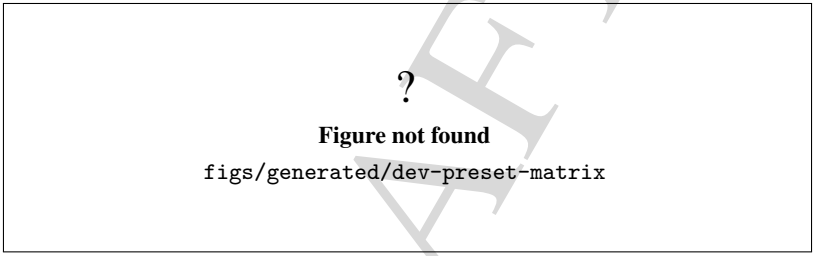


Figure 14.2: Current preset matrix and build-tree separation.

Figure 14.2 should make the current split explicit: tests use `build/tests-unit`, `build/tests-kernel-unit`, and `build/tests-smoke`; the shell uses `build/genesys_shell`; the worker uses `build/genesys_worker_app`; the GUI uses `build/gui-app`; and the model-specific variants use their own build trees under `build/`.

14.2.1 Main build commands

The repository root README and the local-agent runbook both use the same command shape for configure and build. The current baseline examples are:

- `cmake -preset tests-unit`;
- `cmake -build -preset tests-unit -parallel "$(nproc)"`;
- `ctest -preset tests-unit -output-on-failure`;
- `cmake -preset tests-kernel-unit`;
- `cmake -build -preset tests-kernel-unit -parallel "$(nproc)"`;
- `ctest -preset tests-kernel-unit -output-on-failure`;
- `cmake -preset tests-smoke`;
- `cmake -build -preset tests-smoke -parallel "$(nproc)"`;
- `ctest -preset tests-smoke -output-on-failure`.

Application builds follow the same pattern but target the application preset:

- `cmake -preset genesys_shell`;

- `cmake -build -preset genesys_shell -parallel "$(nproc)"`;
- `cmake -preset genesys_worker_app`;
- `cmake -build -preset genesys_worker_app -parallel "$(nproc)"`;
- `cmake -preset gui-app`;
- `cmake -build -preset gui-app -parallel "$(nproc)"`.

The same pattern applies to the independent GUI and model-specific presets. The build tree remains separate for each preset, so changing one family does not overwrite another family unless the developer deliberately reuses the same binary directory.

14.3 Target families

The current preset set maps onto a small number of real target families:

- regression tests: `genesys_kernel_unit_tests`, `genesys_kernel_unit_tests_run`, and `genesys_smoke_tests`;
- shell: `genesys_shell`;
- model-specific terminal application: `genesys_terminal_application`;
- dedicated model-specific app: `genesys_modelspecific_app`;
- worker: `genesys_worker_app`;
- main GUI: `genesys_gui`;
- independent GUI frontends:
  - `genesys_httpworker_gui_application`;
  - `genesys_dataanalyser_gui_application`;
  - `genesys_optimizer_gui_application`;
  - `genesys_ai_assistant_gui_application`.

The shell and model-specific presets also encode source selection through cache variables such as `GENESYS_TERMINAL_EXAMPLE` and `GENESYS_MODELSPECIFIC_APP`. That means the source tree selection is part of the build contract, not just an ad hoc command-line argument.

Figure 14.3: Current target families and their executable boundaries.

Figure 14.3 should help developers avoid the common mistake of building the wrong family when the task only needs a single target or one independent GUI.

14.4 Dependency and decision tree

The current repository does not expose every possible build choice as a named preset. The conservative decision tree is simple:

1. If the task is about ordinary regression, use one of the test presets.
2. If the task is about the interactive shell, use `genesys_shell`.
3. If the task is about the worker runtime, use `genesys_worker_app`.
4. If the task is about the main GUI, use `gui-app`.
5. If the task is about an independent GUI, use the matching GUI preset.
6. If the task is about model-specific generation, use the matching terminal/model-specific preset.
7. If the task needs a sanitizer, use the focused sanitizer-enabled test executable that already exists, rather than assuming a whole-repository sanitizer preset.

The current sanitizer path is intentionally focused. In `source/tests/CMakeLists.txt`, the `genesys_test_simulator_plugin_completion_lifetime` target adds `-fsanitize=address` and `-fno-omit-frame-pointer` under GNU or Clang, links with the matching sanitizer option, and runs with `ASAN_OPTIONS=detect_leaks=1:halt_on_error=1` and `LSAN_OPTIONS=exitcode=23`. That proves a focused ownership/lifetime sanity path exists. It does not prove repository-wide leak freedom.

The chapter's dependency diagram makes that choice explicit.

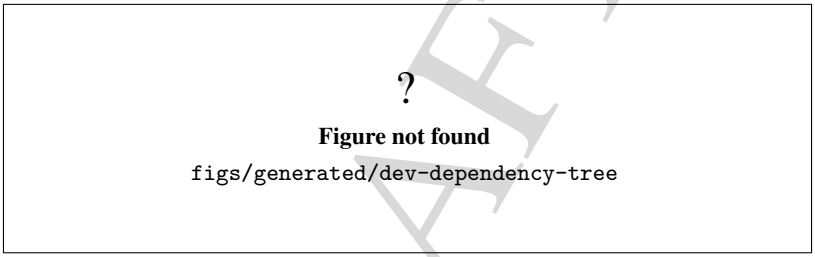


Figure 14.4: Current build dependency decision tree.

Figure 14.4 is the practical guide for selecting the right preset. It also makes the sanitizer boundary explicit: there is a real focused sanitizer path, but not a universal sanitizer preset.

14.5 Warnings, cleanup, and incremental build

The current workflow assumes incremental rebuilds inside each preset-specific build tree. That is the default `cmake -build` behavior: only changed targets are rebuilt unless the developer asks for a fresh configuration.

Use a separate build tree when changing incompatible options. If the current cache becomes stale, rebuild from the affected `build/<preset>` tree instead of mixing unrelated preset state.

Warnings should be treated conservatively:

- configuration warnings should be resolved or explicitly understood;
- build warnings should be checked against the target being changed;
- a successful build does not by itself validate runtime behavior;
- a successful startup does not by itself validate the workflow or the science behind it.

The current preset inventory is debug-oriented, so a release-oriented build requires a deliberate new configuration and a fresh validation pass. Likewise, sanitizers are currently a focused diagnostic path rather than a repository mode that can be assumed everywhere.

14.6 Validation snapshot

The local documentation pass for this chapter used the following commands:

- `cmake -list-presets=all`;
- `cmake -version`;
- `ninja -version`;
- `g++ -version`;
- `qtpaths6 -qt-version`;
- `source/tests/CMakeLists.txt` inspection for the focused sanitizer target.

The observed versions were `cmake` 3.28.3, `ninja` 1.11.1, `g++` 13.3.0, and Qt 6.4.2. Those values are the current evidence snapshot for this chapter revision and should be refreshed when the toolchain changes.

15. Repository Architecture

This chapter documents the current repository layout as it exists in the active branch and build surface. It is based on the root `README.md`, the root `CMakeLists.txt`, `CMakePresets.json`, the canonical architecture/status documents, and the filesystem tree observed on 2026-07-23. It does not treat historical paths or planned workspaces as if they were current build boundaries.

Objective. Explain how the repository is organized, which directories are source architecture, and how the current CMake hierarchy maps to real modules, targets, executables, tests, and support areas.

Audience. Developers who need a faithful map of the codebase before changing modules, build wiring, documentation, or packaging.

Scope. Cover the top-level repository tree, the source tree, the current CMake target families, the non-build support areas, and the current reading path for technical work.

Status. Confirmed by repository inspection on 2026-07-23. The chapter describes the current architecture snapshot, not a future idealized layout.

The chapter follows this sequence: Figure 15.1, Figure 15.2, Figure 15.3, Figure 15.4, and Figure 15.5.

15.1 Repository snapshot

The repository is intentionally organized around a small number of top-level roots:

- `CMakeLists.txt` and `CMakePresets.json` define the build surface;
- `source/` contains the compiled code and test code;
- `models/` contains example and didactic `.gen` inputs;
- `docs/` contains the manuals and governance material;
- `debian/`, `packaging/`, and `scripts/` contain packaging and automation helpers.

Generated build directories such as `build/` or `cmake-build-debug/` exist in the checkout, but they are not source architecture and must not be read as canonical repository structure.

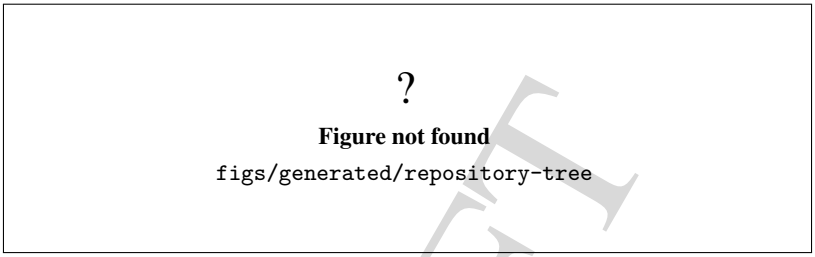


Figure 15.1: Current repository tree and the major top-level roots.

Figure 15.1 should make the boundaries obvious at a glance: the code lives under `source/`, the reference models live under `models/`, and the documentation and packaging helpers are separate subtrees rather than hidden inside the application code.

15.2 Source tree and build graph

The current build graph is rooted in `CMakeLists.txt`. That file does not build the repository by directory name alone; it turns on and off module groups through explicit options and then adds only the relevant subdirectories.

The current root options are:

- `GENESYS_BUILD_KERNEL`;
- `GENESYS_BUILD_PARSER`;
- `GENESYS_BUILD_PLUGINS`;
- `GENESYS_BUILD_TERMINAL_APPLICATION`;
- `GENESYS_BUILD_TERMINAL_EXAMPLES`;
- `GENESYS_BUILD_WORKER_APPLICATION`;
- `GENESYS_BUILD_GUI_APPLICATION`;
- `GENESYS_BUILD_TOOLS`;
- `GENESYS_BUILD_TESTS`;
- `GENESYS_ENABLE_PYTHON_INTEGRATION`.

The compatibility aliases currently retained in the build layer are:

- `GENESYS_TERMINAL_APPLICATION`;
- `GENESYS_BUILD_WEB_APPLICATION`;
- `GENESYS_TERMINAL_EXAMPLE`;
- `GENESYS_TERMINAL_EXAMPLE_CLASS`.

They are transitional names only. The planned GUI experiment slot is `GENESYS_BUILD_GUI_DOEXPERIMENTS`, and it is intentionally guarded as not-yet-implemented, so it is not treated here as a current architecture

surface.

?

Figure not found

`figs/generated/repository-target-graph`

Figure 15.2: Current target graph derived from the live CMake files.

Figure 15.2 is the canonical view of the current build surface. The core static chain is the following dependency ladder:

- `genesys_kernel_util`;
- `genesys_kernel_statistics`;
- `genesys_kernel_simulator_support`;
- `genesys_kernel_simulator_runtime`.

The plugin and parser side feed the same runtime chain. The relevant static libraries are:

- `genesys_plugins_data`;
- `genesys_plugins_components_minimal`;
- `genesys_plugins_components` for the broader component library with an optional GLPK path;
- `genesys_parser`, which remains a separate parser library with optional Bison/Flex regeneration.

The executable families hang off that shared core:

- `genesys_shell`;
- `genesys_modelspecific_app` and the legacy `genesys_terminal_application` copy target;
- `genesys_worker_app` / `genesys_worker`;
- `genesys_gui_application` / `genesys_gui`;
- `genesys_httpworker_gui_application`;
- `genesys_dataanalyser_gui_application`;
- `genesys_optimizer_gui_application`;
- `genesys_ai_assistant_gui_application`;
- `genesys_tools_application`;
- the test executables under `source/tests/`.

15.3 Layered boundaries

The repository is easier to understand when read in layers rather than as a flat directory listing.

Figure 15.3 captures the most useful reading order for new work:

1. kernel;
2. parser and plugins;
3. applications and tools;
4. tests;
5. support material such as docs, models, packaging, and scripts.

?

Figure not found

`figs/generated/repository-layer-map`

Figure 15.3: Current repository layers and their responsibilities.

The kernel contains the runtime foundation:

- `source/kernel/util/`;
- `source/kernel/statistics/`;
- `source/kernel/simulator/`.

The parser owns expression interpretation in `source/parser/`. The plugins layer owns data and component definitions in `source/plugins/data/` and `source/plugins/components/`.

The applications layer contains the user-facing and service-facing binaries:

- `source/applications/shell/`;
- `source/applications/worker/`;
- `source/applications/gui/`;
- `source/applications/modelSpecific/`; and
- the separate `source/applications/mcp/` workspace.

The MCP directory is useful integration material, but it is not added by the root CMake graph and should therefore be treated as a companion workspace rather than a compiled target family.

The tools layer is a shared backend library tree in `source/tools/`. Its source families currently include AI assistant support, biochemical, continuous, factorial design, optimization, and statistics. Only `source/tools/AIAssistant/` has its own subdirectory CMake file; the other tool subdirectories are source families consumed through the top-level `genesys_tools` library.

15.4 Non-build support areas

Several top-level directories are part of the repository but are not source architecture in the narrow CMake sense.

The support areas matter because they shape how the code is delivered and documented, but they do not define the runtime module boundaries of the simulator itself.

15.5 Dependency and build flow

The current build flow is a configure-build-test pipeline with separate preset families for unit tests, shell, worker, GUI, and model-specific applications. The safest reading is:

1. configure with the correct preset;
2. build the matching target family;

Table 15.1: Support areas that are part of the repository but not compiled as core architecture.

Area	Current role	Build graph status
`docs/`	Manuals, governance, architectural references, and generated PDFs	documentation source, not runtime architecture
`models/`	Example and didactic `.gen` models used by the manual and validation flows	data/content root, not a compiled target family
`debian/`	Debian packaging metadata and install manifests	packaging-only
`packaging/`	Linux desktop, Docker, and OVA helpers	packaging-only
`scripts/`	Maintenance and automation scripts	tooling-only
MCP workspace	Python MCP workspace for assistant integration	integration workspace; not in the root CMake graph

3. run the matching tests when the preset family defines them;
4. inspect packaging or installation artifacts only after the compiled surface is confirmed.

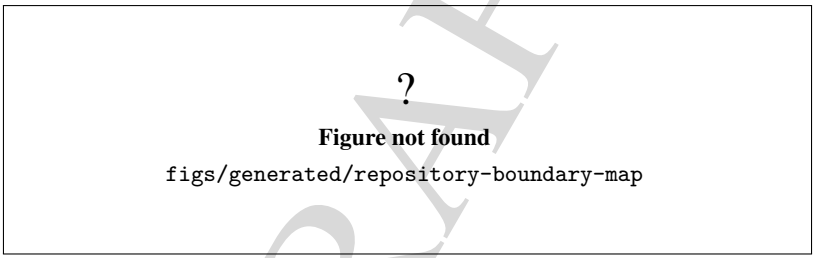


Figure 15.4: Current compiled boundary versus support-only repository areas.

Figure 15.4 is the easiest way to avoid false assumptions. A directory existing on disk does not mean it is in the root build graph. The compiled boundary is created by `add_subdirectory()` calls in the root CMake file and by the options that enable or disable them.

The current preset inventory makes this flow concrete. For example:

- `tests-unit`, `tests-kernel-unit`, and `tests-smoke` are the test-oriented families;
- `genesys_shell` and the model-specific presets build the terminal family;
- `genesys_worker_app` and `gui-app` build the service and main GUI families;
- the independent GUI presets build their own executables in separate build trees.

The build tree separation is part of the architecture. It prevents one family from overwriting another family accidentally and makes it possible to reason about target boundaries from the preset name alone.

15.6 Reading order and final notes

The most reliable reading order for this repository is:

1. root `README.md`;
2. `docs/ai_assistants/ARCHITECTURE.md`;

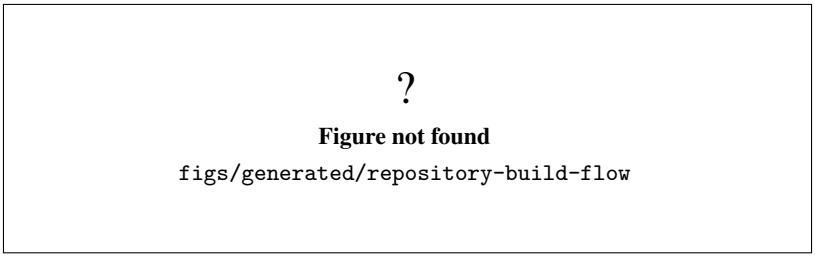


Figure 15.5: Current configure, build, test, and support workflow.

3. root `CMakeLists.txt` and `CMakePresets.json`;
4. the `source/kernel/` subtree;
5. the parser and plugins subtrees;
6. the application and tools subtrees;
7. the tests subtree;
8. the support roots (`docs/`, `models/`, `debian/`, `packaging/`, and `scripts/`).

This chapter deliberately avoids overclaiming the current maturity of any individual application or scientific feature. It is a map of current structure, not a promise that every directory is equally production-ready.

15.7 Test your understanding

1. Why is the root `CMakeLists.txt` a better source of truth for build architecture than directory names alone?
2. Which directories are support areas rather than compiled runtime architecture?
3. Why should `source/applications/mcp/` be treated differently from the current compiled application families?
4. Which target family forms the runtime core that the applications and tests share?

16. Kernel Lifecycle and Managers

This chapter documents the current kernel entry point and the manager objects that coordinate runtime behavior in the active branch. It is based on the live kernel sources inspected on 2026-07-23. The inspection focused on:

- `source/kernel/simulator/Simulator.*`;
- `source/kernel/simulator/model/Model.*`;
- `source/kernel/simulator/model/ModelManager.*`;
- `source/kernel/simulator/PluginManager.*`;
- `source/kernel/simulator/TraceManager.*`;
- `source/kernel/simulator/ParserManager.h`;
- `source/kernel/simulator/ExperimentManager.*`.

Objective. Describe the kernel lifecycle and the manager objects that own, expose, and shut down the runtime services used by the simulator.

Audience. Developers who need to reason about kernel startup, ownership, teardown order, and the current set of manager classes.

Scope. Cover the `Simulator` entry point, the owned manager set, the `Model` lifecycle, ownership boundaries, and the current TODOs that still mark incomplete manager behavior.

Status. Confirmed by repository inspection on 2026-07-23. This chapter records the current runtime snapshot, not a future refactoring target.

The chapter follows this sequence: Figure 16.1, Figure 16.2, Figure 16.3, Figure 16.4, and Fig-

ure 16.5.

16.1 Kernel entry point

The kernel is centered on the `Simulator` class. The class is exposed as the main simulation kernel object and gives the rest of the repository access to the current managers: `LicenceManager`, `PluginManager`, `ModelManager`, `TraceManager`, `ParserManager`, and `ExperimentManager`.

The constructor is intentionally explicit. It creates those managers in one place, prints the startup banner, and exposes the resulting kernel through a single object that applications can obtain through the connector layer. That means the kernel is not a monolithic simulation loop; it is a coordinator for specialized subsystems.

?

Figure not found

`figs/generated/kernel-uml`

Figure 16.1: Kernel entry point and the main manager objects.

Figure 16.1 should make the kernel composition obvious before the reader drills into teardown order and manager responsibilities.

16.2 Startup and ownership

The constructor of `Simulator` creates the owned manager objects in a fixed order. The current implementation constructs:

1. `LicenceManager`;
2. `PluginManager`;
3. `ModelManager`;
4. `TraceManager`;
5. `ParserManager`;
6. `ExperimentManager`.

That order matters because the managers are not just passive data holders. `PluginManager` needs the kernel and the trace infrastructure. `ModelManager` creates and tracks models that are themselves kernel objects. `TraceManager` mediates most runtime diagnostics. `ExperimentManager` coordinates the higher-level experiment abstraction that is still under development.

The model-side lifecycle follows the same pattern. A new `Model` is constructed with a parent `Simulator`. It immediately binds to the parent trace manager, creates its own event manager, model-data manager, component manager, simulation controller, parser, checker, persistence layer, future-event list, controls, and responses, and then exposes those objects through getters.

The key ownership rule is simple:

- raw pointers are not automatically owning pointers;
- ownership is confirmed only when constructor and destructor paths delete the pointed-to object;
- non-owning back-references remain non-owning even though they are stored as raw pointers.

In the current code, the top-level owning relationships are clear enough to document conservatively:

- `Simulator` owns the manager objects it creates;
- `ModelManager` owns the open models it stores;
- `PluginManager` owns the connected plugin wrappers plus the connector and system command executor it creates or receives;
- `TraceManager` owns its handler lists and error-message storage;
- `ExperimentManager` owns the experiment objects in its list;
- `Model` owns its runtime services, containers, and model-owned runtime objects;
- `Model` does not own its parent `Simulator` or the trace manager reference it borrows from the simulator.

?

Figure not found

`figs/generated/kernel-ownership`

Figure 16.2: Confirmed ownership and back-reference boundaries in the kernel.

Figure 16.2 should be read together with the destructor paths. The chapter intentionally treats ownership conservatively and does not infer ownership from pointer syntax alone.

16.3 Model lifecycle

The `Model` class is the core runtime object beneath the simulator. Its constructor shows the current model stack clearly:

- the parent simulator is stored as a back-reference;
- the trace manager is borrowed from the simulator;
- `OnEventManager` is created for model-level events;
- `ModelDataManager` and `ModelComponentManager` are created for model content;
- `ModelSimulation` is created as the execution controller;
- parser, checker, and persistence services are created through the kernel traits layer;
- the future-event list is created and sorted chronologically;
- controls and responses are allocated as model-owned lists.

The current header documentation also states that the model is represented by components, model-data definitions, the future-event list, controls, responses, persistence, parser access, and simulation state. That matches the current implementation.

Two lifecycle details matter for developers:

1. `Model::clear()` tears down the current runtime state and then calls `Util::ResetAllIds()`.

2. `Model:: Model()` destroys runtime state in a deliberate order: future events, transient entities, components, remaining model data definitions, then the owned services and containers.

That order is important because several destructors and manager methods are expected to keep manager state internally consistent while objects disappear. The model destructor therefore destroys owned runtime objects before it destroys the containers and managers that track them.

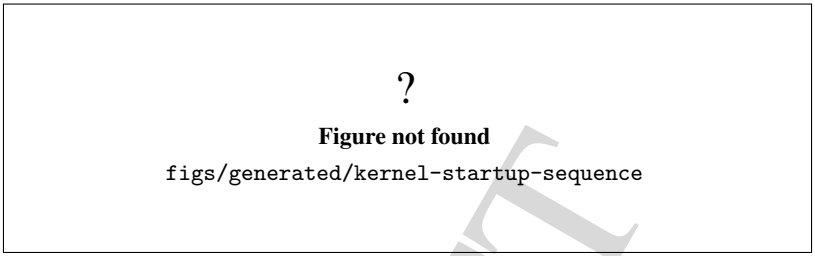


Figure 16.3: Kernel startup sequence from simulator creation to model readiness.

Figure 16.3 should make the current creation sequence easy to audit. It is especially useful when diagnosing startup failures or when adding new manager responsibilities.

16.4 Teardown and shutdown safety

The shutdown path is as deliberate as the startup path. The current `Simulator` destructor deletes the managers in a fixed order:

1. `ExperimentManager`;
2. `ParserManager`;
3. `ModelManager`;
4. `TraceManager`;
5. `PluginManager`;
6. `LicenceManager`.

This order is conservative and avoids leaving late users behind the objects they still reference. The trace infrastructure deserves special mention: `TraceManager::beginShutdown()` marks the tracer as shutting down and clears all callback lists before the storage is released. That prevents late GUI or application callbacks during teardown.

The model shutdown path also keeps teardown deterministic. The destructor first destroys model-owned runtime data, then deletes the simulation services, then deletes the containers and managers that own the runtime graph.

Figure 16.4 captures the reason the shutdown path is safe enough to document: callbacks are neutralized, runtime objects are destroyed first, and container storage is released only after the live objects have been removed.

16.5 Manager responsibilities

The manager set is small but has distinct roles.

`ModelManager` is responsible for the open-model list. Its current behavior is straightforward:

?

Figure not found

`figs/generated/kernel-shutdown-sequence`

Figure 16.4: Kernel shutdown sequence and teardown safety guard.

Table 16.1: Current manager responsibilities in the kernel.

Object	Current role
`Simulator`	Kernel facade and owner of the top-level managers.
`ModelManager`	Creates, stores, loads, saves, and deletes open models.
`PluginManager`	Tracks connected plugins, default kernel elements, and load diagnostics.
`TraceManager`	Routes traces, reports, and errors to registered handlers.
`ParserManager`	Placeholder for parser generation and activation workflows.
`ExperimentManager`	Tracks higher-level simulation experiments; save/load remains incomplete.
`Model`	Owns the runtime graph of a model and the services needed to execute it.
`ModelSimulation`	Controls execution, replications, and simulated-time progression.

- `newModel()` creates a new model and inserts it as the current model;
- `insert()` adds an existing model to the open list;
- `remove()` erases the model from the list and deletes it;
- `loadModel()` creates a model, loads it, and deletes it on failure;
- the destructor deletes every remaining model in the list.

`PluginManager` currently owns the connected plugin wrappers and starts with the default kernel plugin elements already inserted. The default set includes `EntityType`, `Attribute`, `Counter`, and `StatisticsCollector`. The manager also keeps plugin load issues so that interactive front ends can display diagnostics after startup.

`TraceManager` is the runtime diagnostics hub. It stores trace and error handlers, report handlers, simulation handlers, and error-message buffers. Its shutdown path clears the callbacks before the storage goes away, which is the main reason the chapter treats it as a core teardown dependency.

`ParserManager` is intentionally minimal at this stage. The header exposes generation and connection entry points, but both methods are marked with TODOs that describe the missing workflow and lifecycle contract.

`ExperimentManager` is likewise incomplete. It already owns a list of `SimulationExperiment` objects and tracks a current experiment, but the save/load methods are explicitly marked as not implemented yet. The class is a work-in-progress layer on top of the currently functional `ModelSimulation` path.

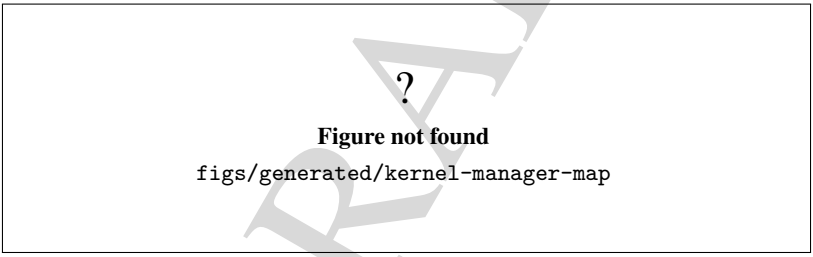


Figure 16.5: Current kernel manager map and dependencies.

Figure 16.5 is the compact summary of the chapter. It separates the top-level simulator services from the per-model runtime services and makes the confirmed dependencies easier to review.

16.6 Confirmed limitations and TODOs

This chapter deliberately records the current limits instead of smoothing them over.

- `ParserManager` still advertises generation and connection methods that are not implemented as a full workflow.
- `ExperimentManager` still leaves save/load unimplemented.
- `Simulator` still relies on raw-pointer ownership for its manager set, so the delete order must remain deliberate until a later refactor documents a different ownership model.
- `Model` still uses several helper managers and containers that are destroyed manually in a controlled order.
- `Model::createEntity()` and `Model::removeEntity()` route through the event system, so entity lifecycle and event lifecycle should be read together rather than in isolation.

The chapter does not present those limits as defects by themselves. It only records them so later phases can decide whether to tighten behavior or replace the current scaffolding.

16.7 Final considerations

The current kernel design is small enough to describe clearly and rich enough to matter for the rest of the manual. `Simulator` is the kernel facade; `ModelManager`, `PluginManager`, `TraceManager`, `ParserManager`, and `ExperimentManager` are the top-level services; and `Model` is the runtime object that binds execution, persistence, model contents, and event scheduling together.

With that picture in place, the next chapter can move from kernel ownership to event scheduling and replication behavior without re-explaining the same foundations.

17. Event Scheduling and Replications

This chapter documents the current event-driven execution core of GenESyS as implemented in the repository. It does not invent a different scheduling model: it explains the classes, methods, and callbacks that already exist in `ModelSimulation`, `Event`, and `OnEventManager`.

Objective. Explain event scheduling, replication handling, and their role in simulation control.

Audience. Developers who need to understand how the simulator advances time and repeats experiments.

Scope. Keep the chapter centered on events, replication boundaries, event dispatch, observers, and the resulting runtime semantics.

Status. Partially validated by code inspection and the current unit-test inventory. The chapter matches the current kernel and GUI callback surface, but the visual figures are structural placeholders until the final artwork is produced.

The chapter follows this sequence: Figure 17.1, Figure 17.2, Figure 17.3, Figure 17.4, and Figure 17.5.

17.1 The execution core

The current simulation lifecycle is centered on `ModelSimulation`. That class starts the simulation, starts each replication, advances the simulation clock by processing events from the future-event list, and coordinates pause, resume, step, and stop requests.

The object model is intentionally split. `Model` owns the structure of the simulated system, including the future-event list. `ModelSimulation` owns the execution state and the replication loop. `Event` represents a scheduled instant, while `OnEventManager` exposes observation and callback hooks around the runtime.

This separation matters because the simulator does not advance time in a simple incrementing loop. It advances from scheduled event to scheduled event, and the runtime state changes are concentrated in those event-processing points.

17.2 The future-event list

The future-event list is the kernel structure that drives event order. The current implementation reads the first event from the list, treats it as the next event to process, and removes it only when the event is actually dispatched. The list is also cleared at the start of each replication to avoid carrying stale queued events into the next run.

The implementation keeps the list sorted chronologically. When a new event is created by the model or a component, it enters that list with the time stamp that determines when it will become current.

The chapter should be read together with the kernel overview chapter, which describes the future-event list as the central mechanism for chronological progress.

?

Figure not found

`figs/generated/event-scheduling-lifecycle`

Figure 17.1: Lifecycle of a single scheduled event.

17.3 Replication lifecycle

Each simulation is composed of one or more replications. The current code starts the simulation once, then starts each replication in order until the replication count is exhausted or a stop/pause request interrupts the loop.

The first simulation startup performs common initialization:

- it checks the model before running;
- it prepares the simulation header and tracing;
- it computes the time-scale factor between replication and report units;
- it creates the simulation-scoped statistics collectors and counters;
- it sets the current replication number to one.

Each replication then performs its own initialization:

- pending events are destroyed and removed from the queue;
- transient entities are removed from the model data manager;

- simulated time is reset to zero;
- per-replication model data is reinitialized;
- source components are initialized after the remaining components;
- statistics and counters are cleared when that option is enabled;
- the replication-start observer notification is emitted.

The replication ends when one of the current end conditions is satisfied:

- the future-event list becomes empty;
- the next event time goes beyond the configured replication length;
- the terminating condition evaluates to true.

When a replication ends, the runtime emits the replication-end notification, shows the replication report when enabled, and aggregates statistics into the simulation-wide collectors.

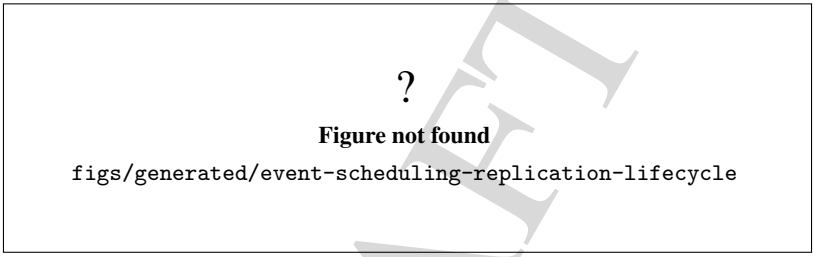


Figure 17.2: Lifecycle of a simulation replication.

17.4 Event dispatch

The main event-processing path lives in `ModelSimulation::_stepSimulation()`. The current code emits a replication-step notification before it even removes the next event from the queue. After that, it takes the next event, checks the warm-up boundary when needed, evaluates breakpoints, and then either pauses or processes the event.

For a normal event, the runtime does the following:

1. remove the event from the future-event list;
2. set the current event pointer;
3. update simulated time;
4. emit the process-event notification;
5. dispatch the event;
6. emit the after-process-event notification;
7. delete the processed event;
8. clear the current event pointer.

The dispatch step distinguishes two paths:

- ordinary events are sent to `ModelComponent::DispatchEvent()`;
- internal events are handled by `InternalEvent::dispatchEvent()` and are not tied to a specific entity/component move.

This distinction matters because the internal event path is used for kernel-side callbacks that are unrelated to an entity traveling through a component.

?

Figure not found

`figs/generated/event-scheduling-sequence`

Figure 17.3: Event-processing sequence inside a replication step.

17.5 Observability and observers

`OnEventManager` is the callback hub for model and simulation events. It stores handler lists for model check, load, save, replication start, replication step, replication end, process event, after-process event, entity create, entity move, entity remove, simulation start, simulation pause, simulation resume, simulation end, and breakpoint notifications.

The class uses both function pointers and bound method handlers, and it deduplicates handlers before insertion. The concrete tests in `source/tests/unit/test_support_oneventmanager.cpp` exercise that deduplication and method-dispatch behavior.

`SimulationEvent` carries the state that observers need to react to a runtime notification: current event, current replication number, simulated time, pause and stop flags, whether the simulation is running or paused, the created entity, the destination component, and an optional custom object. The model-level counterpart, `ModelEvent`, carries the current model.

The parser tests also prove that these callbacks are used during actual simulation execution. In `source/tests/unit/test_parser_expressions.cpp`, a replication-start handler injects parser state before the process-event observer reads and writes attributes during the run.

17.6 Pause, resume, step, and breakpoints

The current runtime supports pause and resume as explicit simulation states. `ModelSimulation::pause()` sets a pause request. `step()` is implemented by forcing a temporary pause request, calling `start()`, and then restoring the previous pause flag. `stop()` requests termination and also performs a direct shutdown path when the simulation is already paused and not running.

Breakpoints are supported on three dimensions:

- component;
- entity;
- time.

When a breakpoint matches the next event, the simulation emits the breakpoint notification and pauses before processing the event. The code also guards against immediately retriggering the same breakpoint twice in a row.

The GUI-facing execution states are still derived from the booleans in `ModelSimulation` rather than from a dedicated enum. The supporting behavior is exercised by the simulation startup smoke path in `source/tests/smoke/smoke_simulator_start.cpp`, which confirms that the simulator object initializes the core managers needed by the execution flow.

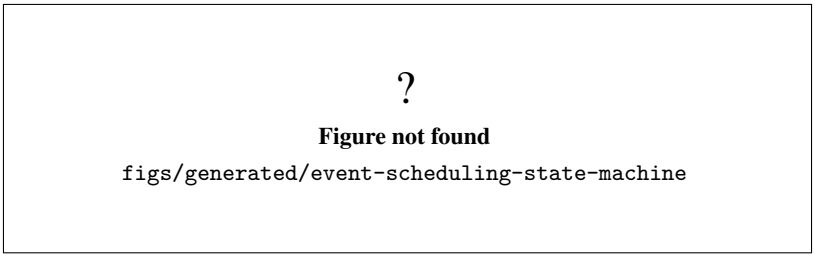


Figure 17.4: Effective simulation state machine for event-driven execution.

17.7 Warm-up and termination

Warm-up is applied at the replication level. When a warm-up period is configured, the next event is checked before processing. If the current simulated time is still inside the warm-up window and the next event crosses the boundary, the kernel clears the replication statistics so the initial transient does not contaminate the measured results.

Termination is also replication-scoped. The kernel considers the event queue, the replication-length horizon, and the terminating condition together. That is why the chapter should not describe the run as ending only because a fixed number of events have been executed. The configured replication length and the optional terminating expression both matter.

17.8 Cleanup and limits

At the start of each replication, the kernel clears queued events and removes transient entities before reinitializing the model data. After a processed event, the code deletes the event only after the after-process notification so observers can still inspect it.

The current implementation has a few explicit limits that the chapter should state plainly:

- the event queue is owned by the model and is not a general-purpose event broker;
- internal events exist, but they are a separate path from ordinary entity dispatch;
- the experiment-oriented classes exist, but `ModelSimulation` remains the operational execution core;
- the GUI and the kernel share runtime state through boolean flags and callbacks, not through a dedicated event-state enum.

The current test inventory provides support for the main flow categories but does not prove every possible model, breakpoint, or terminating expression. That boundary should remain explicit in the text.

17.9 Validation anchors

The following tests support the main claims in this chapter:

- `source/tests/unit/test_support_oneventmanager.cpp` verifies handler deduplication and method-handler dispatch in `OnEventManager`;
- `source/tests/unit/test_parser_expressions.cpp` exercises replication-start and process-event callbacks during a real simulation run;

?

Figure not found

`figs/generated/event-scheduling-list-evolution`

Figure 17.5: Evolution of the future-event list across a replication.

- `source/tests/unit/test_support_simulationscenario.cpp` confirms the current replication and control metadata model;
- `source/tests/smoke/smoke_simulator_start.cpp` confirms the simulator can construct the core managers needed by the execution path.

These tests validate the callback surface and startup path, but they do not replace code inspection for exact event order, breakpoint behavior, or the precise replication termination conditions.

18. Model Representation and Persistence

This chapter explains how a GenESyS model exists in memory, how the kernel serializes that state, and how the current load/save path rebuilds the object graph through the plugin-based persistence layer. It is the bridge between the model structure already described in the previous chapters and the on-disk artifacts that users and developers actually inspect.

Objective. Explain how models are represented internally and persisted to disk.

Audience. Developers who need to inspect model files, understand loading behavior, or reason about save and reload semantics.

Scope. Cover the runtime object graph, the current `.gen` persistence path, the load and save pipelines, ownership during round-trip, and the practical limits of the implementation.

Status. Confirmed by code inspection and supporting unit tests. The chapter follows the current `Model`, `Persistence_if`, `PersistenceRecord`, `GenSerializer`, and `PersistenceDefaultImpl2` implementations, plus the round-trip tests already present in the repository.

The chapter follows Figure 18.1, Figure 18.2, Figure 18.3, Figure 18.4, and Figure 18.5.

18.1 What is represented in memory

The persistence story begins with the model object itself. A `Model` owns the simulation-facing managers and the runtime services that must be reconstructed when a model is opened again. In the current im-

plementation that includes the data manager, component manager, simulation controller, parser, checker, persistence service, event manager, trace manager reference, and the future-event list.

The important point is that persistence does not save a loose bag of objects. It saves a model with ownership boundaries:

- `ModelDataManager` groups data definitions by runtime typename.
- `ModelComponentManager` keeps the component set ordered by ID.
- `ModelDataDefinition` carries the metadata and the internal or attached data relations used by specialized objects.
- `ModelComponent` extends that same base infrastructure for flow behavior.
- `ModelSimulation` and `ModelInfo` hold simulation settings and model metadata.

This is why the persistence code always serializes the same core categories: simulator information, model metadata, simulation parameters, model data definitions, and components. It is also why a model reload must re-establish the object graph instead of just filling a text buffer.

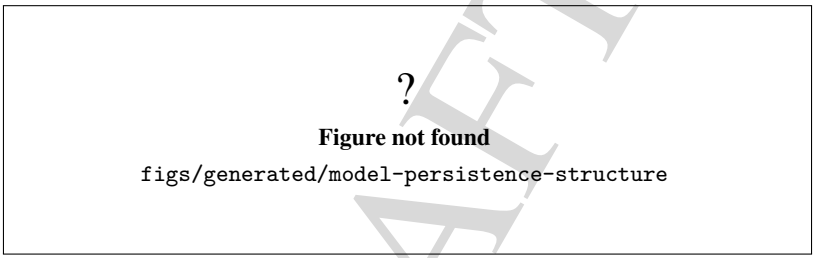


Figure 18.1: Current in-memory structure that persistence must preserve.

Figure 18.1 is the right mental model before reading the file format itself. If the ownership picture is wrong, the save and load pipeline will also be interpreted incorrectly.

18.2 How saving works

The public entry point for saving is `Model::save(std::string)`. That method delegates to the configured persistence service and only adds a trace message around the result. The real formatting and file writing happen inside `PersistenceDefaultImpl2::save`.

The save pipeline is straightforward but important:

- the persistence service selects a serializer from the filename extension;
- `.xml` uses `XmlSerializer`;
- `.json` uses `JsonSerializer`;
- `.cpp` uses `CppSerializer`;
- every other extension falls back to `GenSerializer`.

For the `.gen` path, the serializer then gathers the model in a fixed order. It writes simulator information, model metadata, and simulation parameters first. After that it writes model data definitions and model components. The current implementation also skips internal statistical helper objects such as `Counter` and `StatisticsCollector`, because they are runtime support artifacts rather than user-authored model content.

The save code serializes only top-level components at level zero. That keeps the saved graph focused

on the actual model structure instead of duplicating nested runtime helpers that are recreated by the owning objects themselves.

?

Figure not found

`figs/generated/model-persistence-save-pipeline`

Figure 18.2: Current save pipeline from the model to the output file.

Figure 18.2 shows why the save path is not just a string dump. It is a controlled traversal of the model and its registries.

18.3 How loading works

Loading starts in `Model::load(std::string)`. Before the file is parsed, the model is cleared so that stale runtime state does not survive the reload. That clear step matters: it destroys pending events, transient entities, components, remaining data definitions, and then resets all IDs.

The actual parsing logic lives in `PersistenceDefaultImpl2::load`. The same extension-based dispatch is used here, except that `.cpp` is not loadable. The loader accepts XML, JSON, and the default GenESyS text format. If the file cannot be opened or parsed, the method returns `false` and traces an error.

The load sequence has three distinct phases:

1. Parse the file into a serializer-specific in-memory store.
2. Instantiate the loaded objects through the plugin registry.
3. Reconnect the top-level component graph once all components exist.

During instantiation the loader handles the core headers specially. It checks the simulator version in the saved metadata, loads `ModelInfo` and `ModelSimulation`, and then dispatches each remaining entry by `typename`. If the typename cannot be resolved to a plugin, loading fails. If a component is loaded successfully, its raw field record is kept so the final connection pass can resolve the saved `nextId` and port data.

Figure 18.3 is the operational view of the round-trip. First the model is reset, then the file is read, and only after that do the component links return.

18.4 The `.gen` layout

The default textual format is handled by `GenSerializer`. Its output is deliberately simple and line oriented. The file begins with comment headers, then writes simulator and model metadata, then writes data definitions, and finally writes model components.

Each serialized line is produced from a `PersistenceRecord`. That record behaves like a small associative table of fields and is the common container passed through the `_saveInstance()` and

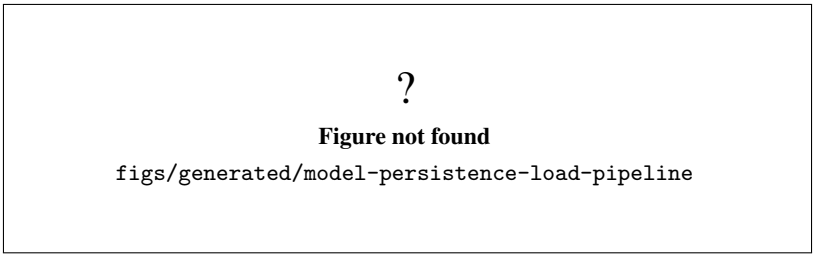


Figure 18.3: Current load pipeline from the file back into the model.

`_loadInstance()` hooks. The serializer linearizes the record by printing the numeric ID, the type name, the quoted object name, and the remaining attributes as key-value pairs. Textual values are quoted; numeric values are emitted without quotes.

The loader reverses that process with a conservative tokenization strategy. It first protects quoted strings with placeholders, splits the line, reconstructs key-value pairs, and then hands the resulting field table back to the plugin loader. The code is intentionally pragmatic rather than fancy: it is a compatible text format, not a general-purpose grammar tutorial.

The most important ordering rule is that entries are iterated by ID, not by the order they happened to be inserted. That keeps the file stable enough to read and diff while still letting the registries keep their own internal order.

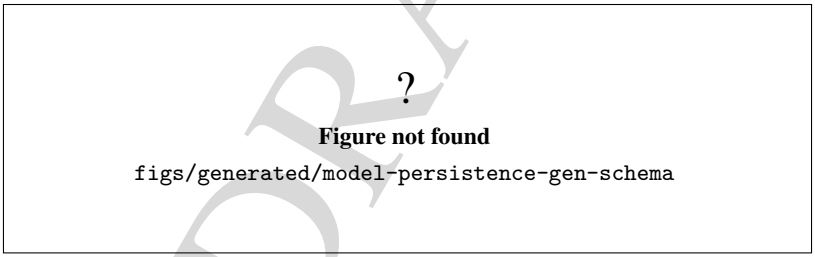


Figure 18.4: Logical schema of the current `.gen` persistence format.

Figure 18.4 is intentionally conservative. It documents the current emitted shape without pretending the format is richer or more abstract than the code makes it.

18.5 Ownership during round-trip

The persistence layer does more than store fields. It must preserve ownership and reconstruction boundaries.

For data definitions, the base class `ModelDataDefinition` is responsible for serializing its own core metadata. Its `_saveInstance()` method writes `typename`, `id`, `name`, `label`, and `reportStatistics`. Its `_loadInstance()` method requires a positive ID before it accepts the record back. Specialized subclasses extend those fields with their own state.

For components, the load path depends on the plugin registry. A component is created, inserted into the model, and then later reconnected using the saved component fields. That two-pass reconstruction is necessary because component connections can only be resolved once every referenced target already exists in the component manager.

The manager layer also matters here. `ModelDataManager::insert()` makes internal data materialize immediately after insertion, which keeps the runtime state consistent for GUI flows and validation. `ModelComponentManager` keeps components searchable by name and ID, and the loader uses that registry when it reconnects the graph.

This is the point where `Model::clear()` becomes part of persistence, even though it is not a serializer. Without the reset, a reload would mix new objects with old runtime objects and the ownership graph would no longer be trustworthy.

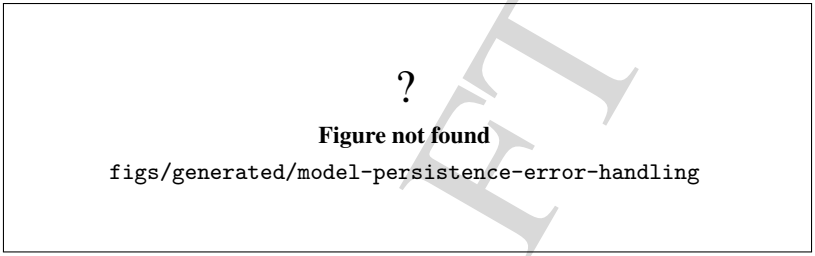


Figure 18.5: Current persistence error-handling boundaries.

Figure 18.5 should be read as a list of current boundaries, not as a promise that every malformed file will be repaired.

18.6 Validation evidence

The repository already contains tests that support this chapter.

- `test_support_persistence.cpp` exercises `PersistenceRecord` insertion, clearing, cloning, range insertion, and the numeric and string load/save helpers.
- `test_simulator_runtime.cpp` contains multiple save/load round-trip tests for concrete model elements, including examples such as `CreateSaveLoadRoundTripPreservesBasicConfiguration`, `ProcessPersistenceRoundTripPreservesConfigurationAndReconcilesInternals`, `WritePersistenceRoundTripPreservesAllTextElementsInOrder`, and `LabelSaveAndLoadRoundTri`
- The same runtime test file also exercises serializer output and confirms that the save path uses the current API and property names.

Those tests do not turn this chapter into a formal specification. They do, however, establish that the current persistence code is not speculative text: the round-trip behavior is already exercised in the tree.

18.7 What this chapter does not claim

This chapter stays within the current code base and does not generalize beyond it. It does not promise a stable public file format ABI, because the code does not document one. It does not claim that every old or future `.gen` variant will remain loadable forever. It also does not describe the `.cpp` serializer as a loadable format, because the current loader explicitly refuses it.

The safest reading is practical: the model can be saved, the supported file types can be loaded, the core object graph is rebuilt through the plugin system, and the implementation already exposes the expected failure modes for unknown typenames, unresolved component targets, and invalid input files.

18.8 Final considerations

This chapter closes the gap between the abstract model structure and the concrete file that leaves the application. For day-to-day work, the important facts are simple:

- saving is a controlled traversal of the live model;
- loading begins with a clean model state;
- the `.gen` file is a linearized snapshot ordered by ID;
- plugin dispatch reconstructs concrete data definitions and components;
- component connections are restored only after all targets exist.

That is enough to reason about persistence bugs, round-trip behavior, and file compatibility without guessing how the kernel behaves.

The next chapter moves from persistence to component-level data definitions, which is where specialized objects and their supporting data become more visible.

19. Model Components and Data Definitions

This chapter explains the shared data-definition layer used by the current kernel, the component specialization built on top of it, and the registry and ownership rules that keep the model consistent. In the current code base, the distinction between a "component" and a "model data definition" is not a separate storage model. It is a behavioral split built on a common base class.

Objective. Describe how model components and model data definitions are represented, managed, created, connected, validated, and persisted in the current branch.

Audience. Developers who extend the simulation kernel or add new component and data-definition types.

Scope. Cover the shared base class, the component subclass, the managers, the internal/attached data lifecycle, the persistence round-trip, and a concrete example of the current behavior.

Status. Confirmed by code inspection on 2026-07-23 and by the lifecycle tests that exercise `Station`, `Queue`, `Resource`, and the core model-data destructor paths.

The chapter follows Figure 19.1, Figure 19.2, Figure 19.3, Figure 19.4, and Figure 19.5.

19.1 Shared vocabulary

The current kernel uses `ModelDataDefinition` as the shared base for most runtime model objects that need identity, persistence, trace integration, and model-managed lifetime. Concrete examples in the repository include `Queue`, `Resource`, `Variable`, `Station`, `Attribute`, and many plugin specific definitions.

`ModelComponent` is a specialization of `ModelDataDefinition`. It adds the outgoing connection manager and the event-dispatch hook used by the future event list. That means components are not a separate class family outside the model-data layer; they are the event-driven subset of it.

The managers keep those responsibilities separated:

- `ModelDataManager` stores all registered `ModelDataDefinition` instances grouped by runtime typename;
- `ModelComponentManager` stores only `ModelComponent` instances and preserves a component-oriented iteration order;
- `ModelDataDefinitionAssociations` maintains the internal and attached child registries that belong to one definition instance.

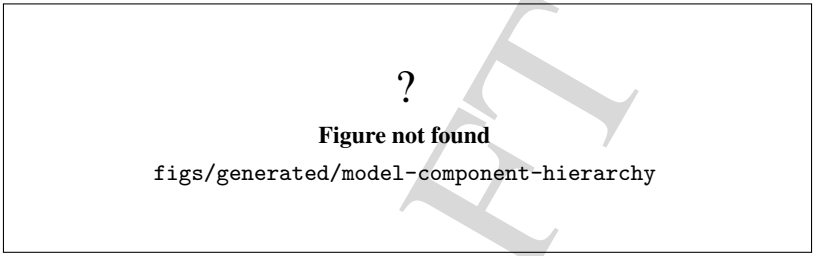


Figure 19.1: Current hierarchy for model data definitions and components.

Figure 19.1 is the right starting point for the rest of the chapter. The important idea is that the same base class provides identity, tracing, persistence, and model-managed lifetime, while the component subclass adds event graph behavior.

19.2 How the model owns the objects

`ModelDataDefinition` constructs an instance with a generated id, a runtime typename, a stable name, and a back-reference to the parent `Model`. When the constructor is told to register immediately, it inserts the instance into the model's data manager. It also creates the standard control set for `Name`, `Label`, `Report Statistics`, and `Trace Level`, then registers those controls with the model.

The destructor reverses that work conservatively. It clears the internal and attached association buckets, removes the instance from the model data manager, and then removes and deletes any owned `SimulationControl` or `SimulationResponse` objects. The code paths exercised by `SimulatorRuntimeTest` confirm that owned internal data is deleted, attached data is not deleted, and owned controls are removed from the model registry.

`ModelComponent` adds one more layer on top. Its constructor calls the base constructor with deferred insertion, then registers the new component in the component manager and creates a dedicated `ConnectionManager`. Its destructor removes the component from the component manager before deleting the connection manager. The connection manager stores outgoing links together with the target input port number.

The ownership boundary is therefore explicit:

- a raw pointer to a model data definition does not imply ownership by itself;
- internal data is owned by the parent definition when inserted through the association helpers;
- attached data is a non-owning relationship used as a reference;

- component connections are owned by the source component's connection manager, not by the target component;
- `ModelDataManager` and `ModelComponentManager` are registries, not the lifetime owners of external objects by pointer syntax alone.

?

Figure not found

`figs/generated/model-component-modeldata-relationship`

Figure 19.2: How the model, registries, definitions, and component connections relate.

Figure 19.2 should be read with the destructor code in mind. The current implementation is intentionally careful about cleaning up owned children while leaving non-owning references intact.

19.3 Creation and validation

The model layer creates definitions and components through a combination of constructors, plugin factories, and template helper methods:

- the constructors establish identity and insert the instance into the correct manager;
- `ModelDataDefinition::CreateInternalData` materializes the internal helper objects needed by a definition;
- `ModelComponent::CreateInternalData` does the same for components;
- the `_templateCreate...` wrappers call the overridable hooks and convert exceptions into trace output instead of leaving the model in an unreported partial state;
- `ModelDataDefinition::Check` and `ModelComponent::Check` drive object-specific validation;
- `ModelDataDefinition::InitBetweenReplications` resets internal children between replications.

The association layer is the part that turns those hooks into concrete data. `ModelDataDefinitionAssociations` keeps the legacy `internalData` and `attachedData` maps alive. It also provides narrower buckets for statistic reporters, mandatory attached attributes, and editable versus non-editable data definitions. That compatibility layer matters because the current tree still uses the older direct map accessors in several places.

Concrete subclasses use those hooks in different ways:

- `Station` lazily creates internal `StatisticsCollector` objects when report statistics are enabled;
- `Queue` and `Resource` lazily create their accounting collectors on the first operation that needs them;
- `Variable` keeps a runtime value store synchronized with its initial values and exposes a `Scope` control;
- some biochemical and whole-cell definitions use the editable-data hook to materialize model-specific helper objects.

?

Figure not found

`figs/generated/model-component-creation-flow`

Figure 19.3: Current creation and validation flow for model data definitions.

Figure 19.3 is the practical view of the class design. It explains why the current code can create a model object, materialize its helper data, validate it, and then reuse the same object across replications without changing the ownership model.

19.4 Persistence round-trip

Persistence uses the same split between base data and component behavior. `ModelDataDefinition::SaveInstance` writes the shared metadata fields. `ModelComponent::SaveInstance` adds the component-specific connection fields. On load, `ModelDataDefinition::_loadInstance` restores the core metadata, and `ModelComponent::_loadInstance` restores the free-form description and the component-specific fields.

The important detail is that connections are not restored as a fully linked graph in one pass. The serializer writes `nextId` and `nextinputPortNumber` fields, but loading still needs the target components to exist before those links can be reconnected. That is why the chapter treats persistence as a two-step round-trip: first the definitions are instantiated, then the topology is reattached.

The same distinction applies to data definitions that own internal children. Their internal objects are serialized through the owning object's own save logic, not through a generic graph dumper. The current format is therefore a managed object graph, not a free-standing abstract tree.

?

Figure not found

`figs/generated/model-component-persistence-flow`

Figure 19.4: Current persistence round-trip for model data definitions and components.

Figure 19.4 highlights the boundary between generic metadata persistence and component topology reconstruction.

19.5 Concrete example

The current branch contains several concrete examples of the shared pattern. `Station` is the clearest data-definition example because it creates and reuses internal statistics collectors only when report statistics are enabled. The lifecycle tests show that the first `enter()` call materializes `NumberInStation` and `TimeInStation`, repeated calls reuse the same objects, and disabling report statistics suppresses the collectors entirely.

`Queue` and `Resource` show the same lazy-materialization pattern for their accounting collectors. Their tests confirm that the internal definitions are created on first use, reused on later operations, and absent when report statistics are disabled.

`Variable` demonstrates a different specialization. It still uses the same shared base, but its runtime behavior is centered on the value store and the `Scope` control rather than on statistics collectors.

Taken together, those examples show the current design intent:

- the base class handles identity, trace integration, controls, and persistence metadata;
- the association layer handles owned versus referenced helper data;
- concrete subclasses choose which helper objects to materialize and when;
- the managers keep the registry state synchronized with the lifetime of the objects themselves.

?

Figure not found

`figs/generated/model-component-concrete-example`

Figure 19.5: Concrete current examples of the shared model-data pattern.

Figure 19.5 is the practical bridge between the shared abstractions and the plugin classes that developers usually touch first.

20. Plugin Registration and Architecture

This chapter documents the plugin boundary as it exists in the current code base. The key point is simple: GenESyS still uses plugins as a structural extension mechanism, but the present implementation is organized as static libraries and build-time aggregation, not as a separate dynamic-plugin runtime.

Objective. Explain the current plugin architecture, how the build registers it, and which parts are only planned for a future dynamic model.

Audience. Developers who need to understand how the current repository assembles plugin code and where the architectural debt still lives.

Scope. Cover the CMake target map, the current registration and metadata flow, the lifecycle of plugin code in the build, and the planned future architecture. The future state is described as planned only; it is not presented as implemented behavior.

Status. Current architecture documented from the live CMake layout and the kernel/plugin manager boundary. The dynamic-plugin future is a design target, not a shipped capability.

20.1 Current plugin architecture

In the current repository, the plugin layer is assembled at build time. The root plugin CMake file:

- adds the `data` and `components` subdirectories;
- defines the aggregate static library `genesys_plugins_components_minimal`.

This means the current plugin model is a static-linking architecture with clear internal grouping, not a runtime loader that discovers shared objects on disk.

The current target map is:

- `genesys_plugins_data`: static library with the plugin-side data definitions;
- `genesys_plugins_components`: static library with the broader component set;
- `genesys_plugins_components_minimal`: static aggregate over the component sources in `source/plugins/components/`.

The data library links to `genesys_kernel_util` and `genesys_ai_provider`. It also carries the `GENESYS_HAS_PYTHON_INTEGRATION` definition and conditionally links to `Python3::Python` when Python integration is enabled. The component library links to `genesys_kernel_util` and `genesys_plugins_data`, and it optionally uses GLPK when the system dependency is available. That optional dependency is still resolved at build time.

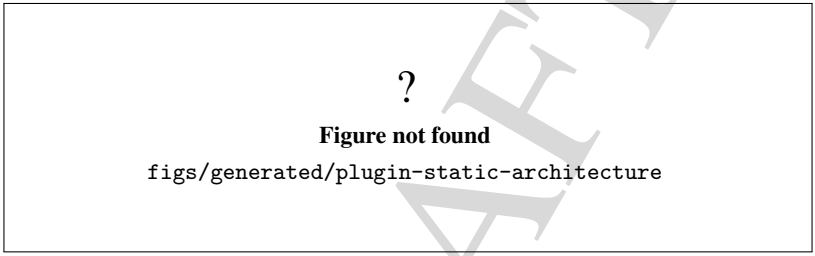


Figure 20.1: Current plugin architecture as static build-time aggregation.

The kernel side also reflects this design. The simulator still exposes a `PluginManager`, but the code base does not describe a separate dynamic-plugin package boundary. The practical meaning is that plugin registration today is mostly a question of how the build assembles and links static libraries, how metadata is surfaced, and how the kernel keeps track of the connected plugin wrappers.

20.1.1 Registration and metadata flow

The current registration flow is build-first:

- source files are grouped under the plugin subtrees;
- CMake turns those sources into static libraries;
- the application targets link those libraries into the final binaries;
- the kernel/plugin manager layer consumes the resulting plugin metadata and connected wrappers.

That flow matters because it keeps registration deterministic. There is no claim here that a plugin can be dropped into a runtime directory and loaded as an independent shared object. The existing model is closer to a curated compile-and-link registry than to a late-bound extension loader.

This is consistent with the surrounding architecture documented elsewhere in the manual. Components and data definitions live in the plugin tree, but their lifecycles are controlled by the normal build and object-graph rules of the current code base. In other words, the plugin layer is an extensibility mechanism, yet it is still embedded in the build graph.

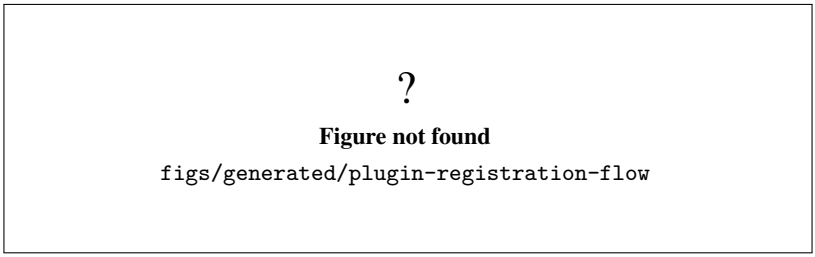


Figure 20.2: Current registration and metadata flow for plugins.

20.2 Current lifecycle

The lifecycle of a plugin in the current implementation has three distinct stages.

1. **Build stage.** CMake discovers the sources, defines the static libraries, and resolves optional dependencies such as Python integration and GLPK.
2. **Link stage.** The selected applications and test targets link the plugin libraries into the final executable or test binary.
3. **Kernel stage.** The simulator starts with the linked plugin code already present, and the kernel/-plugin manager boundary exposes the loaded metadata and connected wrappers to the rest of the system.

The important architectural consequence is that plugin availability is decided before runtime. This keeps the current system predictable, but it also means that a future dynamic model would require a genuine boundary decision around loading, discovery, visibility and compatibility. The code base does not claim that boundary today.

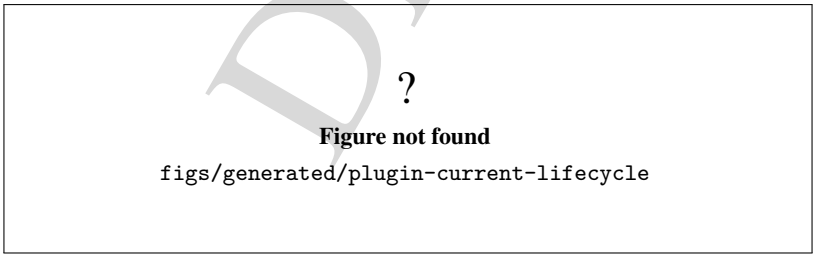


Figure 20.3: Current plugin lifecycle from build to startup.

The current lifecycle also explains the main failure modes. A plugin problem is usually a build, dependency, or linkage issue, not a runtime installation problem in the sense of a separate plugin package. That distinction is useful because it prevents the reader from assuming a deployable plugin ecosystem that the repository does not yet provide.

20.3 Future plugin architecture

The future architecture discussed in this chapter is explicitly planned. It is not implemented, and no stable ABI is claimed.

The planned direction includes:

- dynamic plugins loaded from separate artifacts;
- explicit factories for plugin discovery and instantiation;
- versioned metadata and compatibility checks;
- controlled symbol visibility and export policy;
- packaging rules for plugin distribution;
- isolation boundaries for safer loading and failure handling;
- a documented ABI strategy if and when the project chooses one.

These points are architectural goals, not current behavior. In particular, the repository does not yet provide a stable dynamic-plugin ABI, and the text in this chapter must not be read as implying one.

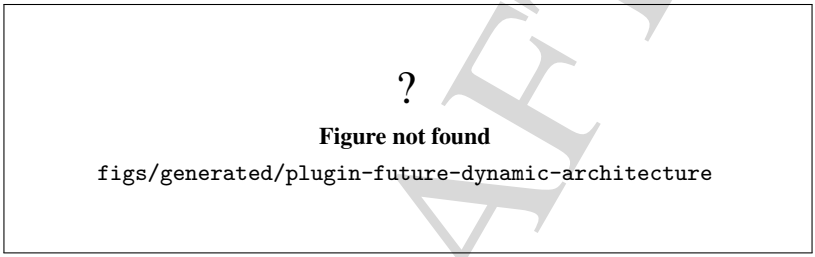


Figure 20.4: Planned future dynamic-plugin architecture, shown as a design target only.

The most important debt item here is the ABI question. A future ABI boundary cannot be improvised from the current static-linking model. It needs an explicit decision about versioning, symbol export, compatibility guarantees, and the level of isolation expected from a plugin package. Until that happens, the conservative reading is the correct one: the current repository uses static plugin aggregation.

20.3.1 Architectural debt

The chapter records the following open debts so that the future work does not silently overclaim what is already available:

- no dynamic loader is present yet;
- no stable plugin ABI is documented or implemented;
- no packaging contract for standalone plugin artifacts exists yet;
- no runtime compatibility matrix is enforced yet;
- no isolation model for untrusted plugin code is defined yet.

20.4 Current versus future

The simplest way to read the boundary is to compare the current and planned models directly:

- **Current.** Static libraries, build-time aggregation, linked metadata, and kernel-facing plugin wrappers already present in the binary.

- **Future.** Dynamic artifacts, explicit discovery, factories, compatibility checks, packaging, and ABI control.

This comparison is intentionally asymmetric. The current side is factual; the future side is a roadmap. The manual should keep that distinction visible so that readers do not confuse a planned extensibility direction with a shipped runtime feature.

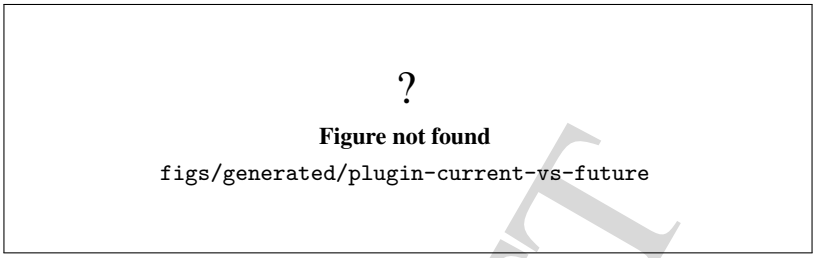


Figure 20.5: Comparison between the current static model and the planned future model.

20.5 What this chapter deliberately does not claim

This chapter deliberately avoids three overclaims:

- that GenESyS already ships a dynamic plugin runtime;
- that the plugin ABI is frozen;
- that loading, packaging, and compatibility are solved for third-party runtime plugin distribution.

Those are future decisions. The current code base documents a static plugin architecture with build-time aggregation and a kernel-facing metadata layer. That is enough to support the repository as it exists today, and it is the correct foundation for the next phase of the work.

21. Parser Architecture

This chapter documents the current parser subsystem as it exists in the repository today. The parser is not treated as a generic text utility; it is a model-aware service that translates expressions, function calls, assignments, and selected control forms into numeric values and model mutations.

Objective. Explain how the parser is wired, how the scanner and grammar cooperate, how model context influences evaluation, and which parts of the regeneration workflow are still incomplete.

Audience. Developers who need to change parsing, expression evaluation, model-language semantics, or parser-related build machinery.

Scope. Cover the current `Parser_if` contract, the `genesyspp_driver` implementation, the Bison/Flex source files, the generated parser artifacts, model-backed symbol resolution, error handling, ownership, regeneration, and the available tests.

Status. Confirmed by code inspection and unit tests. The current parser grammar, scanner, and driver are active; optional regeneration is present in CMake; the higher-level `ParserManager` generation flow is still a stub and must not be read as a finished feature.

The chapter follows Figure 21.1, Figure 21.2, Figure 21.3, and Figure 21.4.

21.1 What the parser service is

The public contract is intentionally small. `Parser_if` exposes two expression-evaluation forms, an error-message accessor, sampler injection, and access to the internal driver for advanced use cases. That is the architectural signal that matters most: parser access is a simulator service, not an isolated library API.

The interface currently supports:

- expression evaluation that may throw;
- expression evaluation that reports success and an error string;
- access to the last parser error message;
- association with a `Sampler_if` instance for stochastic terms;
- exposure of the internal `genesyspp_driver`.

This contract is used by the kernel and by model-facing code. The current implementation is centered on `genesyspp_driver`, which owns the parse session state, the selected model, the sampler pointer, error text, result value, and the registry used to track referenced data elements.

21.2 Source text to value

The concrete parser lives under `source/parser/`. The important files are:

- `Genesys++-driver.h` and `Genesys++-driver.cpp` for the parse-session wrapper;
- `parserBisonFlex/bisonparser.yy` for the grammar;
- `parserBisonFlex/lexerparser.ll` for the scanner;
- `GenesysParser.h`, `GenesysParser.cpp`, and `Genesys++-scanner.cpp` for the committed generated artifacts;
- `CMakeLists.txt` for the parser target and optional regeneration.

The path from source text to numeric value has three stages:

1. the scanner converts characters into typed tokens;
2. the Bison grammar reduces those tokens into expressions, commands, and model-aware function calls;
3. the driver stores the resulting value, error state, and side effects.

The scanner recognizes numbers, operators, booleans, math functions, probabilistic functions, simulation symbols, kernel helpers, and plugin extensions. Identifiers are not treated as plain text; the scanner resolves them against the current model first. It checks attributes, statistics collectors, counters, simulation responses, simulation controls, and then the known plugin-backed data definitions.

That resolution step is why the parser is model-aware. The scanner already depends on the live model state before the grammar even starts reducing.

Figure 21.1 should be read as the operational view of the subsystem. The scanner is not a passive preprocessor, and the grammar is not a generic calculator grammar. Both are coupled to model-aware semantics.

21.3 Driver and model dependency

The driver is the bridge between the grammar and the rest of the kernel. It stores the current model pointer, sampler pointer, parse string or file name, error state, the final numeric result, and an optional registry of referenced data elements. The copy and move operations are implemented explicitly, with a deep copy of the referenced-data registry so copied drivers do not share list ownership.

That ownership detail matters. The registry is heap-allocated, and the driver cleans it up in the

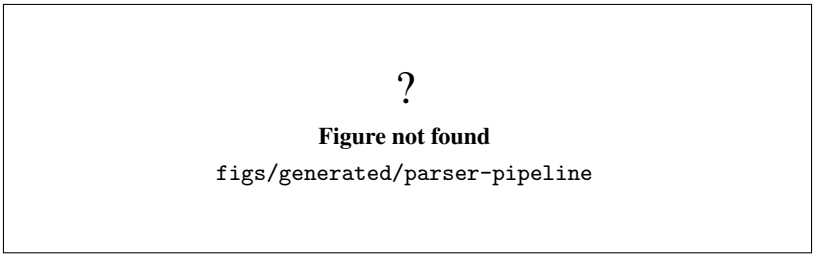


Figure 21.1: Current parser pipeline from source text to value.

destructor. The copy and move paths therefore preserve the driver as a self-contained parse-session object rather than a shallow handle.

The model dependency is also explicit in the helper functions exposed by the driver:

- `findSimulationResponse();`
- `findSimulationControl();`
- `getSimulationResponseValueAsDouble();`
- `getSimulationControlValueAsDouble();`
- `traceWarning()`.

These helpers are used when the grammar evaluates model symbols that are not plain arithmetic values. A simulation response or control is looked up by name, converted to `double` when possible, and traced as a warning if its stored value is non-numeric.

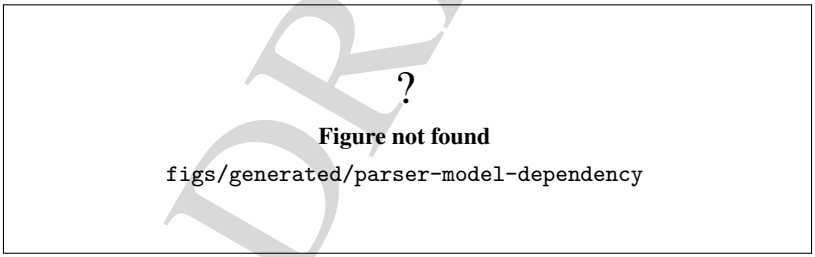


Figure 21.2: Current model and kernel dependencies used by the parser.

Figure 21.2 makes the central point visible: the parser evaluates expressions in model context, not in isolation. This is why parse errors and value lookups are tied to current simulation state.

21.4 Grammar and semantics

The grammar in `bisonparser.yy` is a typed Bison C++ grammar using the `lalr1.cc` skeleton. It defines the current expression language through precedence levels for logical, relational, additive, multiplicative, power, unary, and primary forms. It also defines assignment, command forms, kernel helpers, math functions, probability functions, plugin-backed functions, and model-backed symbols.

The current grammar covers:

- arithmetic, relational, and logical operators;
- conditional forms with `if`;
- basic loop syntax with `for`;
- kernel functions such as `TNOW`, `TFIN`, `MAXREP`, `NUMREP`, `IDENT`, and `entitiesWIP`;
- math and trigonometric functions;
- stochastic functions such as `expo`, `norm`, `unif`, `weib`, `logn`, `gamm`, `erla`, `tria`, `beta`, and `rnd`;
- model symbols for attributes, simulation responses, simulation controls, variables, formulas, queues, resources, sets, and counters.

The most important semantic decision is that grammar actions can mutate the model. Assignments to attributes and variables do not just return a numeric value; they also write back to the current entity or to the data-definition object. The parser therefore acts as both evaluator and model mutator.

Some details remain intentionally conservative:

- `FORM` still returns a placeholder numeric value instead of re-entering the parser recursively;
- `fDISC`, `fLASTINQ`, and `fRESSEIZES` are present in the grammar but are not fully implemented;
- a few current actions still rely on the current event and current entity being present, so those paths must be handled with care.

Those gaps are architectural debt, not hidden success conditions. The chapter records them so the reader does not infer support that the current code does not actually provide.

?

Figure not found

`figs/generated/parser-generation-flow`

Figure 21.3: Current parser generation and rebuild flow.

Figure 21.3 is important because it separates two things that are easy to confuse. The repository already ships generated parser artifacts, but the regeneration workflow is only optionally enabled by CMake and the higher-level `ParserManager` API is still unfinished.

21.5 Error handling and limits

Error handling is split between the lexer, the grammar, and the driver. `ILLEGAL` tokens are turned into explicit parser failures. The driver also catches thrown `std::exception`, thrown `std::string`, and unknown exceptions in `parse_file()` and `parse_str()`. When that happens, it stores an error message, sets the result to `-1`, and either rethrows or returns depending on the configured exception mode.

The grammar also distinguishes between deterministic evaluation and warning paths. For example, non-numeric simulation-response or simulation-control values are converted conservatively to `0.0` after a trace warning. That behavior is intentionally pragmatic, but it should not be confused with a full validation guarantee.

?

Figure not found

`figs/generated/parser-error-flow`

Figure 21.4: Current error flow for lexing, parsing, and evaluation.

Figure 21.4 should make the current failure behavior obvious. A parse failure is not just a syntax error in the abstract; it is a specific path through the lexer, grammar, and driver that ends with a stored message and a negative result.

21.6 Regeneration and build integration

The parser build is controlled by `source/parser/CMakeLists.txt`. By default, the static `genesys_parser` library is built from the checked in `*.cpp` files in `source/parser/`. An optional `GENESYS_PARSER_REGENERATE` switch enables Bison and Flex, which then regenerate the parser and scanner sources from `parserBisonFlex/bisonparser.yy` and `parserBisonFlex/lexerparser.ll`.

This is a useful but limited workflow:

- the regeneration path is present and explicit;
- the project does not require regeneration for every build;
- the current generation API in `ParserManager` is only a declared extension point;
- the chapter should not imply a fully automated parser-creation pipeline.

The practical consequence is that parser changes usually follow two distinct steps:

1. update the grammar/scanner and, if needed, regenerate the committed artifacts;
2. rebuild the parser library and run the expression tests.

That sequence keeps the checked-in generated files and the source grammar in sync without pretending that the project already owns a dynamic regeneration service.

21.7 Tests and safe modification

The current test surface confirms the parser in a focused way. The most useful checks are:

- `test_parser_expressions.cpp`, which exercises precedence, right-associative power, logical operators, math functions, indexed variable access, indexed attribute access, assignment, and event-bound evaluation;
- `test_support_parsermanager.cpp`, which confirms the default construction state of `ParserManager` result and path holders.

Those tests are enough to anchor the current chapter text, but they do not prove broad language completeness. In particular, they do not resolve the remaining stubs around formula evaluation, discrete sampling, or parser regeneration.

When modifying this subsystem, the safe sequence is:

1. inspect `bisonparser.yy`, `lexerparser.ll`, and the driver together;
2. verify whether the affected symbol is a scanner token, a grammar production, or a driver helper;
3. check whether the change requires regeneration or only a source edit;
4. run the parser-focused tests before widening validation.

That workflow keeps expression semantics, model coupling, and error behavior aligned. It also prevents the chapter from overstating the maturity of the unfinished regeneration layer.

22. Expression Language Reference

This chapter documents the expression language as it exists in the current repository. The scope is intentionally strict: only constructs that are implemented in the scanner, grammar, driver, and unit tests are treated as part of the reference. Anything else is listed as a limit.

Objective. Provide the reference description of the current simulator expression language.

Audience. Developers and advanced users who need the supported syntax surface, the current semantic model, and the known gaps.

Scope. Cover lexical forms, numbers, identifiers, operators, precedence, functions, distributions, model references, indexed access, errors, examples, and unsupported syntax.

Status. Confirmed by code inspection and unit tests. The language is model-aware, sparse-indexed, and deliberately conservative about unsupported constructs.

The chapter follows Figure 22.1, Figure 22.2, Figure 22.3, and Figure 22.4.

22.1 How the language is resolved

The expression language is not a free-form string evaluator. The scanner first tries to resolve a token against the live model. Only if that lookup succeeds does the parser treat the text as a model reference. If it does not succeed, the scanner falls back to numbers, operators, keywords, or an illegal-token result.

That means the language is contextual:

- the same symbol can mean different things in different models;
- the current event and current entity can influence the value of an expression;
- model-backed names are resolved before the parser sees them as plain text.

?

Figure not found

`figs/generated/expression-precedence-map`

Figure 22.1: Expression precedence and grouping in the current grammar.

22.2 Lexical surface

The scanner recognizes a small, explicit surface:

- decimal numbers such as `12` or `12.5`;
- hexadecimal numbers such as `0x10`;
- booleans `true` and `false`;
- arithmetic and relational symbols;
- bracketed index syntax;
- keywords for commands, functions, and model-backed symbols;
- model names that are resolved at scan time.

Identifiers are not accepted as arbitrary text. The scanner checks the live model for a matching data definition or simulation symbol. When it finds one, it emits the corresponding token for the parser. When it does not, the token becomes illegal and the parser reports an error.

There are no quoted string literals in the current scanner. The language is expression-first, not string-literal-first. The only string-oriented keywords currently declared by the scanner are `val`, `eval`, and `leng`, and those tokens are not wired into active grammar productions yet.

22.3 Numbers and booleans

Numbers are accepted as decimal or hexadecimal literals. Decimal parsing includes floating-point notation. Boolean literals are mapped to numeric values:

- `true` becomes `1`;
- `false` becomes `0`.

This numeric treatment is consistent with the rest of the grammar, where logical forms ultimately evaluate to numeric values too.

22.4 Operators and precedence

The current precedence structure is:

- parentheses;
- unary + and −;
- power ^ with right associativity;
- multiplication and division;
- addition and subtraction;
- relational operators <, >, <=, >=, ==, !=, and <>;
- logical not, and, nand, xor, and or.

The tests confirm that precedence is real and not merely documented: 1+2*3 evaluates to 7, (1+2)*3 evaluates to 9, and 2^3^2 evaluates to 512, which proves that power is right-associative in the current grammar.

22.5 Functions

The active grammar currently recognizes several function families.

22.5.1 Math and trigonometric functions

The following functions are implemented in the grammar:

- sin, cos;
- round, trunc, frac;
- exp, sqrt, log, ln;
- mod, min, max.

The current unit tests directly reproduce several of these: round(3.6) returns 4, trunc(3.9) returns 3, frac(3.9) returns 0.9, and sqrt(9) returns 3.

22.5.2 Probability distributions

The current stochastic functions are:

- rnd;
- expo;
- norm;
- unif;
- weib;
- logn;
- gamm;
- erla;
- tria;
- beta;
- disc.

The implemented distributions call through Sampler_if. The unit tests reproduce valid and invalid cases for unif, tria, expo, weib, norm, logn, gamm, erla, and beta. The invalid cases are important because they prove that the parser reports recoverable errors rather than silently accepting bad parameters.

22.5.3 Kernel and model-aware functions

The current grammar also exposes simulation and model-dependent helpers:

- tnow, tfin, maxrep, numrep;
- ident, entitieswip;

- `tavg`, `count`.

These values come from the live simulation or from model data definitions. They are not pure arithmetic functions.

22.5.4 Queue, resource, set, variable, and formula references

The grammar includes model-backed references for:

- attributes;
- variables;
- formulas;
- queues;
- resources;
- sets;
- counters;
- statistics collectors;
- simulation responses;
- simulation controls;
- entity groups.

This is the core of the language's domain specificity. It is a language for simulation state, not a generic calculator.

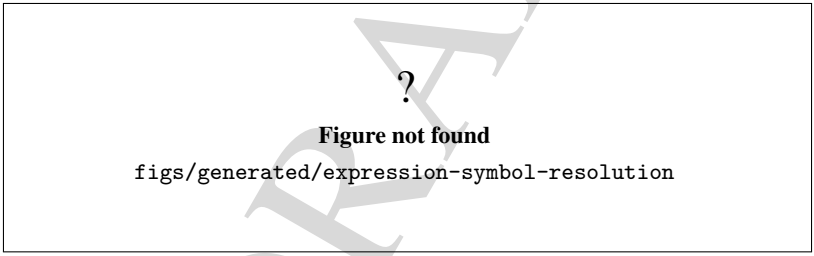


Figure 22.2: Symbol resolution from scanner token to model-aware value.

22.6 Indexed access, arrays, and matrices

The language supports sparse indexed access with bracket syntax. The grammar builds a comma-separated index key and uses it to look up values in the underlying sparse store.

The practical surface is:

- `name`;
- `name[1]`;
- `name[1,2]`;
- `name[1,2,3]`;
- longer sparse keys when the backing data definition supports them.

This is how the current code represents array-like and matrix-like data. There is no separate array literal syntax or matrix constructor in the expression language. Instead, the language uses indexed model data.

The unit tests cover both reading and writing through this form:

- variable reads for scalar and indexed values;
- attribute reads for scalar and indexed values;
- indexed assignments for variables and attributes;
- missing sparse entries returning `0`.

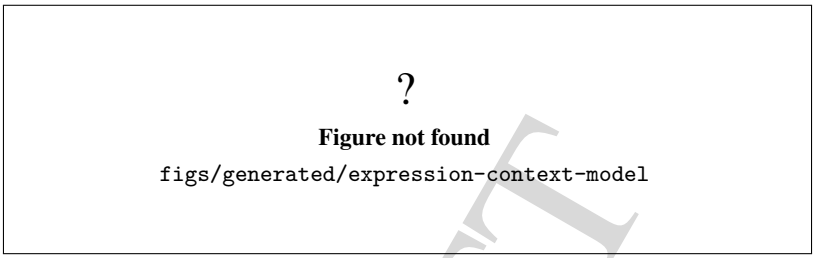


Figure 22.3: Context-dependent evaluation for attributes and variables.

22.7 Errors and recovery

The parser distinguishes between illegal characters and unknown literals. It also surfaces function-specific validation errors for stochastic functions.

The current behavior is:

- illegal tokens become parse failures;
- unknown symbols become literal-not-found errors;
- invalid distribution parameters return recoverable errors when the caller uses the non-throwing parse path;
- the driver stores a last-error message for inspection.

The exact message format is intentionally conservative. The important point is that the parser does not silently accept invalid syntax or invalid stochastic parameters.

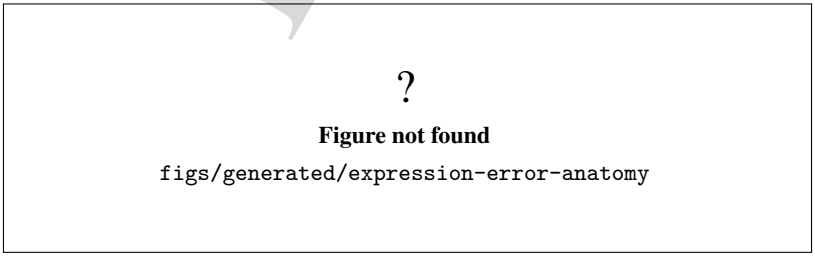


Figure 22.4: Error anatomy for illegal tokens and invalid parameters.

22.8 Examples reproduced by tests

The examples below are all covered by the current unit tests or by direct evidence in the parser implementation.

Table 22.1: Current expression examples reproduced by tests.

Expression	Expected result	Evidence
`1+2*3`	`7`	precedence test
`(1+2)*3`	`9`	precedence test
`2^3^2`	`512`	right-associative power test
`1<2 and 3>2`	`1`	logical precedence test
`not(1<2)`	`0`	logical-not test
`round(3.6)`	`4`	math-function test
`ParserVarND[1,2,3,4,5]`	`14`	sparse variable read test
`ParserAttrND[1,2,3,4,6]=36`	`36`	sparse attribute write test
`unif(0.8,1.0)`	finite value in range	stochastic validity test
`beta(2,3,1,0)`	failure	invalid-parameter recovery test

These examples matter because they show the language as an operational tool. The reference is not complete unless an expression can be reproduced under the current parser, driver, and model semantics.

22.9 Unsupported syntax and explicit limits

The current reference does not claim support for the following:

- quoted string literals;
- standalone string-expression evaluation via `val`, `eval`, or `leng`;
- a dedicated array literal syntax;
- a dedicated matrix literal syntax;
- recursive formula evaluation through `FORM[...]`;
- full discrete-distribution support through `disc(...)`;
- a general-purpose programming language with full `for` and `else` semantics.

Some of those forms are partially tokenized or partially parsed, but the current implementation does not provide a finished semantic contract for them. In particular, the grammar still contains clear TODO paths for `disc`, formula recursion, and some plugin-backed functions, so those forms must not be documented as complete features.

22.10 Summary

The current expression language is a model-aware, sparse-indexed, numeric language. It supports arithmetic, logic, selected math functions, stochastic functions, and direct reads and writes against live model data. It also has clear limits: no quoted strings, no dedicated array or matrix literals, and no finished recursion for formulas or discrete distributions.

That conservative boundary is the correct reference for the manual today.

23. Tools and Algorithms

This chapter documents the current `source/tools` package as it exists in the repository now: a support layer for statistical analysis, fitting, hypothesis testing, numerical solvers, optimization scaffolds, factorial design, and AI-assisted workflows. It is intentionally conservative. When a family is only partially implemented, the text says so instead of treating a scaffold as a finished algorithm.

Objective. Describe the reusable tools and algorithms exposed by the project, together with their current maturity and the evidence that supports each family.

Audience. Developers, maintainers, and advanced users who need to reuse support libraries or understand which computational helpers are real, which are legacy, and which remain placeholders.

Scope. Focus on the tool layer, algorithm families, validation evidence, and the current boundary between implemented behavior and deferred work. The chapter does not claim that every interface in `source/tools` is already a complete production algorithm.

Status. Substantive documentation grounded in `source/tools`, the package inventory entry point, and the current unit-test surface.

23.1 Package boundary and inventory

The `source/tools` package is the shared support layer for the manual's developer-facing numerical and analysis topics. Its command-line entry point, implemented in `source/tools/main.cpp`, prints a small inventory when invoked with `-list` or `-help`; the output groups the current interfaces into anal-

ysis, fitting, hypothesis testing, distributions, numerics, optimization, and factorial design. Figure 23.1 summarizes that package boundary.

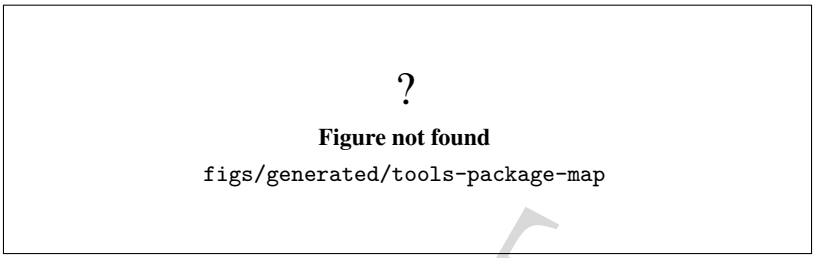


Figure 23.1: Current `source/tools` package boundary and exported tool families.

As shown in Figure 23.1, the package is not a monolith. It exposes a shared static library, a small inventory executable, and several subfamilies with different maturity levels. That distinction matters because a header existing in the tree does not automatically mean the algorithm behind it is complete.

23.2 Statistical analysis and data support

The statistical side of `source/tools` covers dataset-oriented analysis, distribution fitting, hypothesis testing, and probability/distribution utilities. The core abstractions are `DataSet_if`, `DataAnalyser_if`, `SimulationResultsDataset`, `SimulationResultsDatasetParser`, `Fitter_if`, `HypothesisTester_if`, `ProbabilityDistributionBase`, `ProbabilityDistribution`, `Distribution_if`, `ContinuousDistributio` and `DiscreteDistribution_if`.

The current documentation and code describe these roles consistently:

- `DataAnalyser_if` is a high-level facade that orchestrates dataset-oriented analysis services.
- `SimulationResultsDatasetParser` loads numeric result files and Genesys `Record` outputs, preserving replication and optional time metadata.
- `Fitter_if` defines the distribution-parameter inference contract.
- `HypothesisTester_if` exposes parametric confidence-interval and test routines.
- `ProbabilityDistributionBase` and `ProbabilityDistribution` provide static probability and quantile helpers instead of a full OO distribution hierarchy.

The flow from dataset to fitting and testing is summarized in Figure 23.2. The figure is important because the package has enough functionality to support real analysis work, but the steps are still layered across multiple interfaces rather than hidden behind a single all-in-one class.

As illustrated in Figure 23.2, the package supports two broad input shapes for analysis: free numeric datasets and Genesys `Record`-style outputs. The parser keeps replication ids and optional time columns because those fields are needed for downstream plots and per-replication analysis.

23.2.1 Simulation result datasets

`SimulationResultsDatasetParser` is the package's concrete data-ingestion helper. The current unit tests confirm that it can:

- load raw numeric datasets;
- reject malformed decimal and empty datasets;

?

Figure not found

`figs/generated/tools-fit-test-flow`

Figure 23.2: Dataset ingestion, fitting, and hypothesis-testing flow in the tools package.

- load legacy `Record` datasets with or without time columns;
- load enriched `GenesysRecordDataset` headers;
- normalize variable-type strings;
- load GUI-oriented tabular datasets with semantic headers.

The parser behavior is exercised by `source/tests/unit/test_tools_simulation_results_dataset.cpp`, which is the strongest direct evidence for this part of the package. The code path is therefore functional, not hypothetical.

23.2.2 Distributions and fitting

The probability-distribution side is intentionally conservative. `ProbabilityDistributionBase` and `ProbabilityDistribution` are static mathematical facades, while `Distribution_if`, `ContinuousDistributio` and `DiscreteDistribution_if` establish interface boundaries for future growth. The current documentation explicitly says that the package does not yet present a fully unified OO distribution hierarchy.

On fitting, the promoted default implementation is `FitterDefaultImpl`. The package documentation and status notes describe it as functional for uniform, triangular, normal, exponential, Erlang, Beta, and Weibull families, while `FitterDummyImpl` remains as a legacy/documental placeholder. That distinction is important for users reading the headers: the dummy class exists, but it is no longer the default trait binding.

The fitting and testing story is therefore one of real capability with explicit boundaries:

- fitting is real for a defined family set;
- dummy behavior is preserved only for compatibility and documentation;
- hypothesis testing currently has a functional baseline in `HypothesisTesterDefaultImpl1`;
- the remaining two-population paths are still documented as partially consolidated rather than finished.

The current regression suite supports that reading. `source/tests/unit/test_tools_hypothesistester.cpp` confirms proportion confidence intervals and one-population average/variance tests with finite, bounded p-values. Those tests are enough to establish that the baseline works, but not enough to claim that every inferential branch is fully mature.

23.2.3 Hypothesis testing

`HypothesisTesterDefaultImpl1` currently covers one-population proportion intervals, one-population average tests, and one-population variance tests with a coherent confidence-level path. The tests check both rejection and non-rejection cases, and they also confirm that the interval center stays where the estimator says it should be.

The important limitation is that the chapter does not extrapolate this baseline into a blanket claim for all future multi-sample or two-population workflows. The code and tests support a solid starter implementation, not a finished statistical library.

23.3 Optimization and design of experiments

The optimization and DOE part of the package is split between a scaffolded simulation optimizer and a concrete factorial-design helper.

`Optimizer_if` defines the intended contract for simulation-based optimization: model association, control and response selection, objective and constraint registration, settings, lifecycle control, and solution summaries. `OptimizerDefaultImpl1` is the current concrete class, but the code comments and tests make its status explicit: it is still a scaffold, not an executed search engine. The unit test `source/tests/unit/test_optimizer_ownership_contract.cpp` exists primarily to protect ownership semantics, default constructibility, and non-copyable/non-movable behavior for the seven owned `List` wrappers.

Figure 23.3 shows that divide clearly. The optimizer has a real contract and real state transitions, but the search algorithm itself is still deferred.

?

Figure not found

`figs/generated/tools-optimizer-workflow`

Figure 23.3: Current optimizer contract and scaffolded execution lifecycle.

As shown in Figure 23.3, the optimizer is intentionally split into configuration and execution. The configuration API is real, but the step function currently advances counters and state without running a full optimization search. That is a legitimate placeholder, not a hidden claim of completeness.

`FactorialDesign` is the concrete DOE helper in the package. It is a console-oriented class that generates combinations, creates sign tables, computes means and statistics, and prints residual and impact summaries. Unlike the optimizer scaffold, it does real work now, but it is still a low-level helper rather than a polished user workflow.

Figure 23.4 captures that DOE helper. The point of the figure is to show that the class is a computational utility with explicit inputs, intermediate tables, and reported statistics, not a general optimization engine.

As illustrated in Figure 23.4, `FactorialDesign` is best understood as a procedural DOE helper. The class exposes methods such as `checkFeasibility()`, `generateIndexCombinations()`, `createColumnLabels()`, `generateInputs()`, `calculateResultsMean()`, `createSignTable()`, and `calculateStatistics()`. That is enough to make it useful, but not enough to treat it as a fully validated DOE framework.

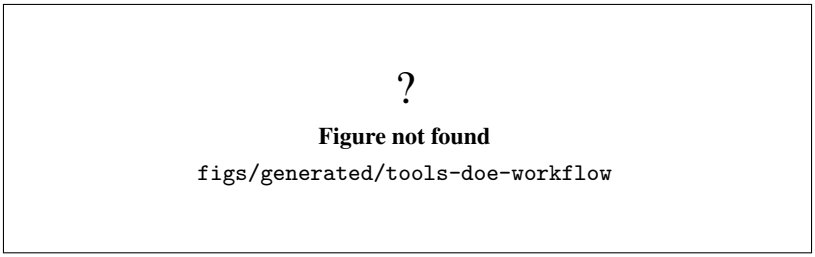


Figure 23.4: Fractional factorial design helper workflow.

23.4 Continuous numerical tools

The continuous-numerics family combines the legacy solver contract with more explicit ODE abstractions. The package currently includes `Solver_if`, `SolverDefaultImpl1`, `Quadrature_if`, `RootFinder_if`, `OdeSystem_if`, `OdeSolver_if`, `OdeSolverFactory`, `RungeKutta4OdeSolver`, `DormandPrince54OdeSolver`, and `DiffusionMethodOfLinesSystem`.

This is one of the clearest places where the chapter needs to separate levels of maturity:

- `SolverDefaultImpl1` is a legacy baseline, and the regression tests keep its Simpson-style integration and derivative-contract behavior stable.
- `OdeSolverFactory` and the concrete ODE solvers are real and test-backed.
- `DiffusionMethodOfLinesSystem` has dedicated numerical checks, but it is still a model-reduction helper rather than a general PDE framework.

The solver family is not shown with a dedicated figure in this chapter because the surrounding chapters already document the simulation and application surfaces that consume it. The current evidence is in `source/tests/unit/test_tools_ode_solver_factory.cpp`, `source/tests/unit/test_tools_diffusion_mol.cpp`, `source/tests/unit/test_tools_legacy_solver_regression.cpp`, and the broader simulation runtime regression coverage that reuses `RungeKutta4OdeSolver`.

In practical terms, this means the solver stack is available and numerically checked, but the manual should still describe it as a support layer. The factory knows how to construct the registered solvers by key, the concrete solvers are validated against analytic decay and harmonic-oscillator cases, and the diffusion method-of-lines helper preserves the expected decay or mass-conservation behavior in the targeted tests.

23.5 AI-related tools

The AI-related part of `source/tools` is not a generic chatbot wrapper. It is a GenESyS-specific assistant layer with explicit model-building and simulation-control duties. The relevant classes are `AIAssistant_if`, `AIAssistantDefaultImpl`, `AIProviderClient_if`, `HttpProviderClientBase`, `OpenAIProviderClientImpl`, `AnthropicProviderClientImpl`, `LocalProviderClientImpl`, `AISecretStore`, `AIAuditLog`, and `AIConversationService`.

The current implementation path is already more complete than a usual stub. The code and package documentation describe:

- configuration and secure API-key handling;
- provider-client abstraction and HTTP transport;
- knowledge-base refresh from the simulator and plugin metadata;

- prompt analysis and model-plan generation;
- model construction and simulation configuration;
- simulation execution and result collection;
- audit logging and bounded conversation history.

`source/tests/unit/test_ai_plugins.cpp` confirms the prompt-template and assistant wiring on the plugin side, and the package documentation in `source/tools/README_tools.md` records the staged AI-assistant roadmap. The important reading is that this is a real tool-layer service, but the chapter still needs to preserve the distinction between implemented stages and planned stages. Stage 7 integration and Stage 8 security/governance hardening remain planned.

23.6 Maturity and validation

Figure 23.5 condenses the maturity boundary of the package. It helps prevent two common mistakes: treating every header as production-ready, and dismissing scaffolded code that already has real ownership or contract value.

?

Figure not found

`figs/generated/tools-maturity-map`

Figure 23.5: Current maturity map for the `source/tools` families.

As summarized in Figure 23.5, the package currently falls into four practical maturity bands:

- **Functional and test-backed**: simulation-result dataset parsing, hypothesis testing baseline, ODE factory and concrete solvers, diffusion method-of-lines checks, AI assistant integration, and the package inventory entry point.
- **Functional but deliberately conservative**: distribution utilities and fitting interfaces, where the code is real but the manual must still describe scope and family coverage carefully.
- **Legacy compatibility**: `FitterDummyImpl` and `SolverDefaultImpl1`, which remain available because the tree still needs compatibility and regression coverage.
- **Scaffolded**: `OptimizerDefaultImpl1`, which preserves contract and ownership behavior while its search algorithm remains deferred.

That classification is the main editorial point of the chapter. The package contains enough usable code to support simulation analysis and numerical experimentation, but it is not honest to flatten all of it into a single "complete algorithms" label. The manual should preserve the real boundaries so later chapters can reference them consistently.

The current validation evidence behind this chapter is therefore mixed but concrete:

- the parser and hypothesis tests are unit-tested;
- the solver factory, ODE solvers, and diffusion helper are numerically checked;
- the optimizer ownership contract is enforced at compile time;
- the AI assistant and plugin wiring have targeted tests;

- the DOE helper is documented directly from its source rather than from a dedicated regression suite.

That is sufficient for a conservative chapter-level description. It is not sufficient to claim that every family is at the same maturity level, and this chapter does not do that.

24. Continuous, Modal, and Hybrid Simulation

This chapter documents the current continuous, modal, and hybrid simulation surface in the repository as it exists now. It is intentionally conservative: the text distinguishes implemented support from historical scaffolding and does not claim a unified scientific validation package that the codebase does not yet provide.

Objective. Explain the continuous solver layer, the Method-of-Lines diffusion bridge, the modal-model and Petri-net facilities, the cellular-automata helpers, and the way these pieces interact with the discrete-event clock.

Audience. Developers and advanced modelers who need to reason about multi-mode simulation behavior, model-to-solver coupling, and the current validation boundary.

Scope. Cover the ODE solver factory, the diffusion PDE implementation, modal networks, Petri nets, cellular automata, the current hybrid bridges, and the limits of what can be claimed from the available code and tests.

Status. Substantive documentation grounded in `source/tools/Continuous`, `source/plugins/components/ModalModel`, `source/plugins/data/Continuous`, and the current regression tests. The chapter treats numerical accuracy claims as demonstrative unless a test or reference in the repository establishes more.

The chapter follows Figure 24.1, Figure 24.2, Figure 24.3, Figure 24.4, Figure 24.5, Figure 24.6, and Figure 24.7.

24.1 Continuous solver layer

The current continuous-simulation layer is split into a legacy solver contract and a more explicit ODE abstraction. In the tree today, `Solver_if` and `SolverDefaultImpl1` remain as compatibility material, while `OdeSolver_if`, `OdeSystem_if`, and `OdeSolverFactory` define the clearer ODE-oriented path. The factory currently registers `RungeKutta4` and `DormandPrince54`; those are the concrete solver names that the code and tests can resolve today.

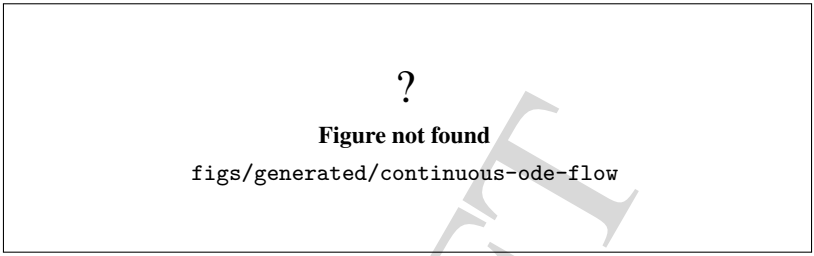


Figure 24.1: Current continuous-solver stack and concrete ODE solver options.

Figure 24.1 should be read as a boundary diagram, not as a promise that every mathematical family has the same maturity. The current factory exposes two solver names, and the surrounding runtime code chooses between them explicitly.

24.1.1 Solver selection and runtime coupling

The ODE-oriented path is centered on `OdeSolverFactory`. That factory creates a solver object from a string key or enum value, and the `DiffusionField` plugin uses that selection mechanism to choose between the supported solver names. The current evidence is direct: the headers in `source/tools/Continuous` define the factory and concrete solvers, and the regression tests exercise both supported solver keys.

The runtime coupling is easier to understand when viewed as a sequence rather than a single class:

- a model or helper selects a solver name;
- the factory materializes a concrete solver object;
- the solver advances a system of equations over a time step;
- the calling component records or propagates the new state;
- the discrete-event kernel remains responsible for the event clock.

Figure 24.2 captures that sequence.

The demonstration in `source/tests/demo_continuous_trace.cpp` is useful because it shows the solver advancing in sync with a discrete-event schedule. That file is not a numerical benchmark paper; it is a clear runtime example that prints the state trajectory, writes a CSV, and records the internal trace.

24.2 Diffusion and Method of Lines

The diffusion path is the clearest current example of a continuous PDE bridge inside GenESyS. The kernel-facing plugin is `DiffusionField`; it solves $du/dt = D\nabla^2 u$ in N dimensions using a dedicated `DiffusionMethodOfLinesSystem`. Spatial discretization is handled by the Method of Lines, and time stepping is delegated to the selected ODE solver from the factory.

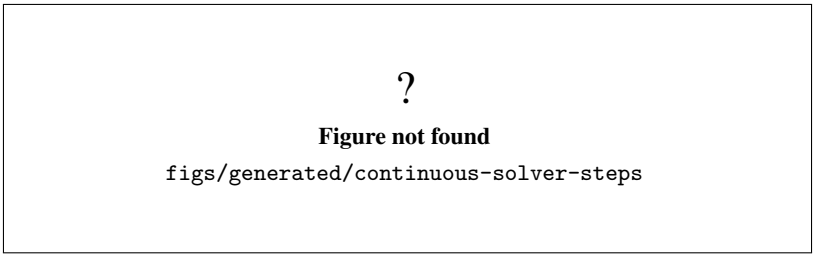


Figure 24.2: Continuous solver steps observed in the current runtime examples.

The component counterpart is `DiffusionSimulate`. It acts as a discrete-to-continuous bridge: when an entity arrives, it can trigger a diffusion solve using either the field's own configured window or the component's override window. The field itself also supports event-driven advancement via internal events when `AutoSchedule` is enabled, so the continuous solve can stay synchronized with the kernel clock.

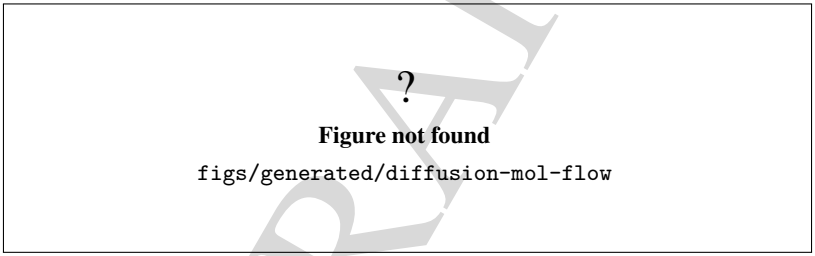


Figure 24.3: Diffusion solve pipeline through Method of Lines and solver selection.

Figure 24.3 is the main evidence that the continuous subsystem is not just a theory chapter. The current tests exercise the default solver, compare Runge-Kutta and Dormand-Prince on a sine mode, check Neumann mass conservation, validate persistence round-tripping, and verify that invalid solver or grid settings are rejected.

24.2.1 What the diffusion tests prove

The strongest direct evidence for the diffusion path is `source/tests/unit/test_plugins_continuous_diffusion.cpp`. That test file confirms all of the following in the current tree:

- the default solver is `RungeKutta4`;
- the analytical decay of a sine mode matches the discretized evolution to the current test tolerance;
- `RungeKutta4` and `DormandPrince54` produce comparable results on the same setup;
- Neumann boundary conditions conserve total mass within the current test tolerance;
- the implementation runs in three dimensions;
- configuration round-trips through persistence;
- invalid solver and invalid grid or boundary settings are rejected;
- the `DiffusionSimulate` component can drive the field.

Those checks are strong runtime evidence, but they are still evidence for the specific scenarios under test. They are not a claim that every diffusion boundary condition, every solver, or every grid size has been scientifically validated.

24.3 Modal models and hybrid execution

The modal-model layer groups several related formalisms under the same component family. `ModalModelDefault` is the base network container. `ModalModelFSM` extends it with finite-state semantics, while `ModalModelPetriNet` extends it with Petri-net semantics. The transition helpers also remain explicit: `EFSMTransition` carries trigger and probability semantics, and `PetriTransition` carries token-based input and output arc weights.

This is the right place to keep the boundary clear:

- an FSM transition is not the same thing as a Petri transition;
- the model holds the state and transition topology;
- the current code keeps the semantics in dedicated classes instead of collapsing everything into one generic transition type;
- the chapter does not claim a finished unified modal runtime beyond what the classes in the tree actually implement.

Figure 24.4 summarizes the current modal state path.

?

Figure not found

`figs/generated/modal-state-transitions`

Figure 24.4: Current modal-model family and the specialization points for state transitions.

Figure 24.4 is deliberately broad because the tree contains several modal variants that are related but not identical. The default modal network keeps the shared container, the FSM extension adds state-machine behavior, and the Petri extension adds token and arc semantics.

24.3.1 Petri nets

The Petri-net side is concrete enough to document directly. `PetriPlace` stores token counts by color, and `PetriTransition` checks and moves those tokens between places according to its input and output arc weights. That is a real enable/fire model, not a generic placeholder.

The model-specific examples in `source/applications/modelSpecific/smarts/ModalModel` show that path in a runnable form. The current tree includes examples for a default modal network, an FSM, and Petri-based examples such as `Smart_PetriPlace.cpp` and `Smart_ModalModelPetriNet.cpp`.

Figure 24.5 makes the semantics boundary visible. In the current code, Petri places own token counts and Petri transitions explicitly read and mutate those counts. That is a clear runtime contract, even

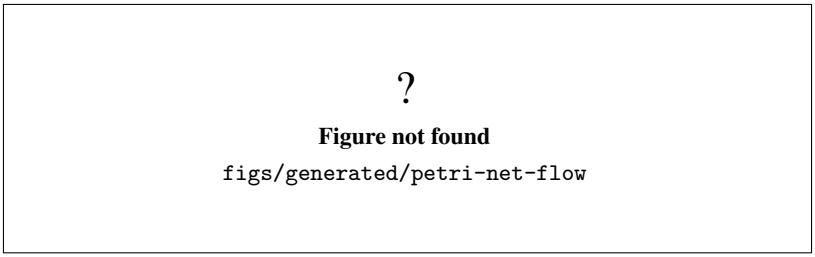


Figure 24.5: Petri net place/transition semantics in the current modal-model tree.

though the chapter does not elevate it to a validated scientific claim.

24.4 Cellular automata

Cellular automata are also present in the modal-model area, but they have their own dedicated component and runtime helpers. The component facade is `CellularAutomataComp`, which can be configured as classic, timed 1D, asynchronous, or user-defined. The underlying helper types include `CellularAutomataBase`, `CellularAutomata`, `Lattice`, `Neighborhood`, `StateSet`, and a set of local rules such as Game of Life, HPP lattice gas, sand/rock, forest fire, and related examples.

The important distinction is that the tree contains multiple CA helpers, but they do not all represent the same execution model:

- `CellularAutomata_Classic` applies the same local rule to all cells and then updates their states;
- `CellularAutomata_1DTimed` treats the second dimension as time;
- the component facade carries configuration for lattice, neighborhood, boundary, state set, and local rule selection;
- the current classes are supportive infrastructure, not a claim that every automaton family is fully polished or equally validated.

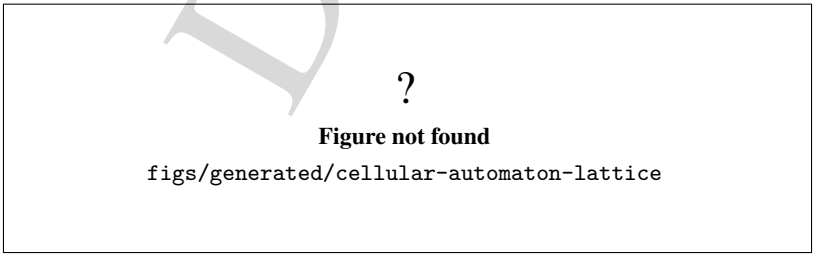


Figure 24.6: Cellular-automaton facade and lattice/rule execution model.

Figure 24.6 is the clearest way to show that the CA support is not a single opaque feature. The component facade selects a runtime shape, and the runtime helpers then execute the chosen lattice/rule combination.

24.5 Hybrid coupling and validation boundary

The hybrid-simulation story is the interaction between the discrete-event calendar and the continuous or modal submodels. The current code supports that interaction in several ways:

- `DiffusionField` can auto-schedule internal events so the field advances one step per kernel event;
- `DiffusionSimulate` can launch a diffusion solve when an entity arrives at the component;
- `ContinuousSystemComponent` and the ODE examples show how a continuous solver can be driven from a discrete-event workflow;
- the model-specific continuous trace example demonstrates that solver time and simulated time are related but not identical concepts.

The correct mental model is that the discrete-event kernel owns the event clock, while the continuous subsystem owns its internal state and step policy. Fixed-step advancement is therefore not automatically equivalent to elapsed discrete-event time. That distinction is documented in the scientific-domain reference and is visible in the trace example.

?

Figure not found

`figs/generated/hybrid-coupling-flow`

Figure 24.7: Discrete-event and continuous-solver coupling in the current tree.

Figure 24.7 is the most important boundary diagram in the chapter. It explains why the continuous subsystem is documented as a coupled mode rather than as a standalone analytical solver library.

24.5.1 Current validation boundary

The current tree gives enough evidence to document the implemented behavior, but not enough to claim global numerical certification. The supported and demonstrated points are:

- explicit ODE solver selection works through the factory;
- diffusion examples can be reproduced and checked against the current analytical reference in the tests;
- modal-network, Petri, and CA helpers exist and are wired into the source tree;
- event-clock coupling is implemented for the documented diffusion and continuous-system examples;
- the chapter does not extend those results into a general scientific validity claim for all future models.

Historical LSODE-based examples still exist in the tree, including `source/tests/test_lsode.cpp` and the model-specific continuous examples under `source/applications/modelSpecific/smarts/Continuous`. They are useful as legacy context, but the explicit ODE solver factory and the diffusion bridge are the clearest current support surface for this chapter.

That is the right level of claim for the current manual revision: the feature families are real, the

coupling is real, and the validation is real for the scenarios already tested, but the codebase still contains experimental and historical areas that should remain labeled as such.

25. Application Architecture

This chapter documents the current application architecture as it exists in the active branch and current CMake files on 2026-07-23. It is based on the root `README.md`, the root `CMakeLists.txt`, the application and tools CMake files, and the observed entry points in `source/applications` and `source/tools`. It describes the current process boundaries and shared backends, not a future idealized layout.

Objective. Explain how the current executable applications are composed, which shared libraries they use, and where the process boundaries sit.

Audience. Developers who need a faithful map of the current application layer before changing startup code, target wiring, shared libraries, or deployment assumptions.

Scope. Cover executable families, composition roots, shared libraries, process boundaries, configuration flags, startup/shutdown paths, and the most visible error exits. Keep the discussion limited to the current code tree and the current CMake build surface.

Status. Confirmed by repository inspection on 2026-07-23. The chapter describes the current architecture snapshot, not a planned future state.

The chapter follows Figure 25.1, Figure 25.2, Figure 25.3, Figure 25.4, Figure 25.5, and Figure 25.6.

25.1 Current executable families

The current application layer is a set of separate executables, not one monolithic process. The build surface currently exposes these runtime families:

- the command-line shell, `genesys_shell`;
- the worker runtime, `genesys_worker_app`, with the executable name `genesys-worker` and the alias target `genesys_worker`;
- the main Qt GUI, `genesys_gui_application`, with the executable name `genesys-gui` and the alias target `genesys_gui`;
- the HTTP Worker GUI, `genesys_httpworker_gui_application`, with the executable name `genesys-httpworker-gui`;
- the Data Analyser, `genesys_dataanalyser_gui_application`, with the executable name `genesys-dataanalyser-gui`;
- the Optimizer, `genesys_optimizer_gui_application`, with the executable name `genesys-optimizer-gu`
- the AI Assistant, `genesys_ai_assistant_gui_application`, with the executable name `genesys-ai-assistant-gui`;
- the model-specific route, `genesys_modelspecific_app`, with the compatibility copy `genesys_terminal_application`;
- the optional tools CLI, `genesys_tools_application`, which is a support entry point rather than a user-facing simulation frontend.

The current GUI umbrella also keeps a planned-but-not-implemented `GENESYS_BUILD_GUI_DOEXPERIMENTS` flag. That flag is a guarded absence, not a runnable process.

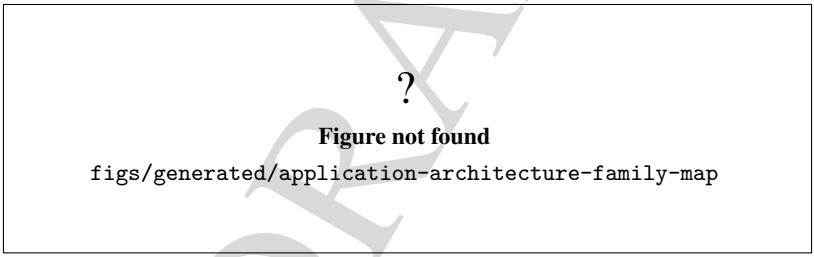


Figure 25.1: Current executable families and their build-time names.

Figure 25.1 should be read as a process map, not as a list of libraries. The important distinction is that each runtime family is a separate executable boundary even when several of them share the same backend libraries.

25.2 Composition roots

Each executable owns its own composition root. The startup code is intentionally local to the process so that one application cannot silently depend on another application's window or event loop.

25.2.1 Shell

The shell uses a minimal console bootstrap: `main.cpp` creates the selected terminal application and forwards `argc`/`argv` to its `main()` method. The current implementation keeps the entry point generic through `GENESYS_TERMINAL_APP_CLASS` and `GENESYS_TERMINAL_APP_HEADER`.

25.2.2 Worker

The worker executable constructs `BaseGenesysWebApplication` and then enters its HTTP serve loop. The worker bootstrap creates `TokenService`, `SessionManager`, `WorkerJobManager`, `SimulatorSessionService`, `ApiRouter`, and `SimpleHttpServer` before it starts listening.

25.2.3 Main GUI

The main GUI installs crash and logging handlers, creates a `QApplication`, loads system preferences, applies the theme, builds the main window, and then enters the Qt event loop. That bootstrap is the composition root for the primary desktop process.

25.2.4 Standalone Qt frontends

The HTTP Worker GUI, Data Analyser, Optimizer, and AI Assistant each keep their own `main.cpp` and their own Qt application object. The GUI umbrella only selects which executable target is built; it does not merge those windows into one binary.

25.2.5 Model-specific route and tools CLI

The model-specific route is build-time selected through `GENESYS_MODELSPECIFIC_APP` and related overrides. Its bootstrap creates the selected terminal application class and then runs it as a separate process. The optional tools CLI is simpler: it prints the supported tool interfaces for `-list` or `-help`, and exits with an error for unknown commands.

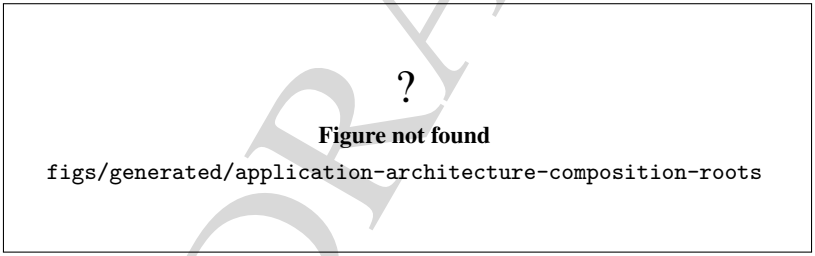


Figure 25.2: Composition roots for the current application processes.

Figure 25.2 is the shortest way to see why the application layer stays modular. Each process creates its own runtime graph and does not rely on another executable to host it.

25.3 Shared libraries and services

The current architecture reuses a small set of static libraries across multiple processes:

- `genesys_kernel_util`;
- `genesys_kernel_statistics`;
- `genesys_parser`;
- `genesys_plugins_data`;
- `genesys_plugins_components_minimal`;
- `genesys_kernel_simulator_support`;

- `genesys_kernel_simulator_runtime`;
- `genesys_worker_core`;
- `genesys_tools`.

The first seven libraries form the common simulator backbone. They are static libraries and aggregate build targets, not runtime plugin processes. The worker stack adds `genesys_worker_core`, which contains `ApiRouter`, `TokenService`, `SimpleHttpServer`, `WebWorkerRuntime`, `SimulatorSessionService`, `SessionManager`, and `WorkerJobManager`. The tools stack adds `genesys_tools`, which groups the assistant, statistics, optimization, continuous, and factorial-design backends and links `genesys_ai_provider`.

The main GUI links both `genesys_worker_core` and `genesys_tools`, which is why its UI can reuse worker-facing and analysis-facing support code without collapsing those flows into a single process. The HTTP Worker GUI links only `genesys_worker_core`; the Data Analyser, Optimizer, and AI Assistant link `genesys_tools` and the shared simulator backbone.

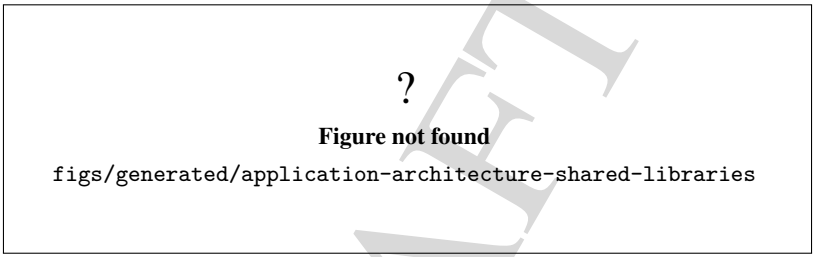


Figure 25.3: Shared static libraries and service layers used by the current applications.

Figure 25.3 should make one rule obvious: the reusable backends are libraries, while the applications are the process boundaries that consume them.

25.4 Process boundaries and configuration

Every executable listed above is a separate operating-system process. Linking a library into a process does not erase the boundary, and no plugin library is a substitute for a process boundary.

The current root configuration options that control this surface are:

- `GENESYS_BUILD_TERMINAL_APPLICATION`;
- `GENESYS_BUILD_WORKER_APPLICATION`;
- `GENESYS_BUILD_GUI_APPLICATION`;
- `GENESYS_BUILD_TERMINAL_EXAMPLES`;
- `GENESYS_BUILD_TOOLS`.

The GUI umbrella adds its own switches for the main GUI, the HTTP Worker GUI, the Data Analyser, the Optimizer, the AI Assistant, and the guarded `GENESYS_BUILD_GUI_DOEXPERIMENTS` placeholder. That layering keeps the process map explicit while still allowing partial builds.

The current static plugins remain part of the simulator backbone, not separate processes. Likewise, `genesys_tools` is a library and `genesys_tools_application` is the optional support process built on top of it. The same distinction applies to `genesys_worker_core` versus `genesys_worker_app`.

Figure 25.4 is the key defense against an easy documentation bug: shared code is not a shared process. The executable names may be related, but the runtime boundaries remain separate.

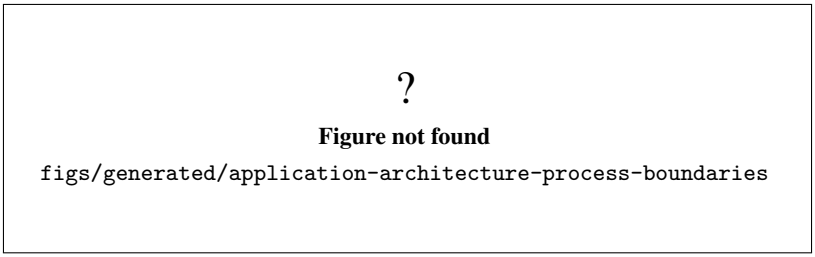


Figure 25.4: Process boundaries and build switches for the current application surface.

25.5 Startup, shutdown, and errors

The startup and shutdown mechanics differ slightly by executable family, but the architectural pattern is the same: the composition root builds the runtime, the runtime enters its main loop, and the process returns an exit code when the loop ends or a startup error occurs.

The shell and model-specific routes use RAII around a selected application object and then return the selected application's exit code. The tools CLI either prints its inventory or reports an unknown command to standard error.

The worker logs that it is listening, then calls `serve()` on its embedded HTTP server. If the server cannot start or bind, the worker prints a startup failure and returns a non-zero exit code. The server loop and request limit control the normal stop path.

The main GUI installs crash and logging handlers before `QApplication` is created. It then loads preferences, applies the theme, builds the main window, and enters the Qt event loop. Shutdown occurs when the event loop returns or a fatal crash handler aborts the process.

The standalone Qt frontends follow the same broad pattern: each process owns its own Qt application object, window, and exit code. Their error handling is local to the process and should not be interpreted as a cross-process recovery mechanism.

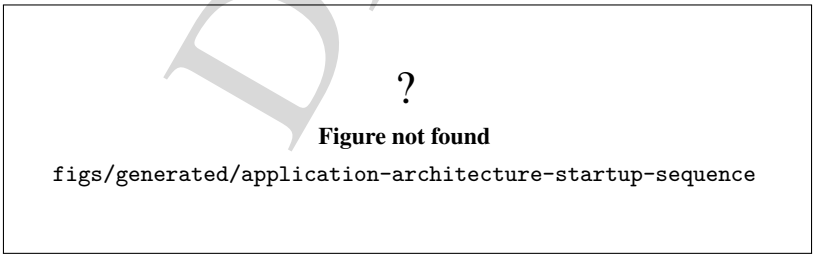


Figure 25.5: Startup sequence for the current executable families.

Figure 25.5 and Figure 25.6 close the loop: they show that the application layer is process-oriented, that each runtime owns its own lifecycle, and that local failures stay local instead of being treated as shared framework state.

?

Figure not found

`figs/generated/application-architecture-shutdown-sequence`

Figure 25.6: Shutdown and error-exit sequence for the current executable families.

26. GUI Architecture

This chapter documents the current Qt6 GUI architecture as it exists in the active branch on 2026-07-23. It is based on the current GUI sources, the generated UI file, and the extracted controller/service classes used by the `MainWindow` compatibility facade. The chapter describes the current snapshot, not a future idealized split.

Objective. Explain how the main graphical interface is composed, which classes own each responsibility, and how user actions travel from widgets to controllers, scene objects, and simulator callbacks.

Audience. Developers who need a faithful map of the GUI before changing actions, view wiring, property editing, plugin presentation, persistence, or simulation interactions.

Scope. Cover the MainWindow composition root, the widget and scene boundaries, controller and service responsibilities, signal/slot and callback wiring, model open/save flows, and the simulation action path.

Status. Confirmed by repository inspection on 2026-07-23. The chapter describes the current architecture snapshot and explicitly includes the remaining compatibility wrappers.

The chapter follows Figure 26.1, Figure 26.2, Figure 26.3, Figure 26.4, and Figure 26.5.

26.1 MainWindow as the composition root

`MainWindow` is still the GUI composition root. The current implementation is split across `mainwindow.cpp`, `mainwindow_controller.cpp`, `mainwindow_scene.cpp`, and `mainwindow_simulator.cpp`, but

the class itself remains the top-level place where UI objects are created, controller dependencies are wired, and compatibility slots are exposed.

The constructor in `mainwindow.cpp` creates the generated UI, the simulator, the trace and property support objects, the extracted controllers, and the helper services used by the editor. The generated `mainwindow.ui` file still provides the widgets, menus, tabs, toolbars, dock panes, and action objects, while the C++ constructor wires them into the controller stack.

The current design is not a pure passive view. `MainWindow` still owns some coordination duties:

- it creates the controller graph;
- it registers the remaining compatibility slots used by the generated UI;
- it connects scene signals to legacy handlers;
- it keeps some cross-cutting state such as the active model tabs, selection state, and recent-file menu;
- it mediates between the generated UI and the extracted controllers.

This means the GUI is already partially modular, but not fully flattened into independent presenter or service layers. The chapter therefore treats `MainWindow` as a compatibility facade that delegates selected responsibilities instead of pretending that the split is already complete.

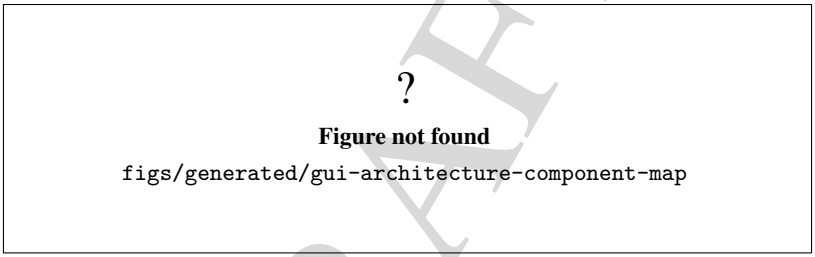


Figure 26.1: Current GUI component map and the main controller blocks attached to it.

Figure 26.1 is the entry point for the rest of the chapter. It separates visible widgets from the controller and service objects that keep them synchronized.

26.2 Controller and service map

The GUI is already organized around focused controller and service classes. Their current roles are visible in the class headers and in the constructor wiring inside `mainwindow.cpp`:

- `ModelLifecycleController` owns new/open/save/close/check/configure/exit orchestration;
- `SimulationController` validates simulation readiness before unsafe commands;
- `SimulationCommandController` turns start/step/pause/resume/stop requests into simulator calls;
- `SimulationEventController` bridges kernel simulation events back to the GUI;
- `TraceConsoleController` renders simulator traces into the console, simulation, and reports panes;
- `ModelInspectorController` keeps the component and data-definition trees synchronized with the scene;
- `PropertyEditorController` binds scene selection to the property editor and coalesces post-commit refreshes;
- `PluginCatalogController` owns the plugin tree presentation and insertion affordances;

- `SceneToolController` owns zoom, drawing, animation, and layout commands;
- `GraphicalContextMenuController` builds the scene context menu using existing actions;
- `DialogUtilityController` owns the auxiliary dialogs and utility actions;
- `GraphicalModelSerializer`, `GraphicalModelBuilder`, `ModelLanguageSynchronizer`, `GraphvizModelExporter`, and `CppModelExporter` keep persistence and representation work out of the widget code;
- `GuiExtensionManager` rebuilds the extension surface from the loaded model plugins.

The important boundary is that these classes are not just helper functions. They are the current seam between the generated Qt widget tree and the rest of the application layer. In other words, the GUI already has a service/controller tier, but the legacy `MainWindow` wrappers still remain as compatibility glue.

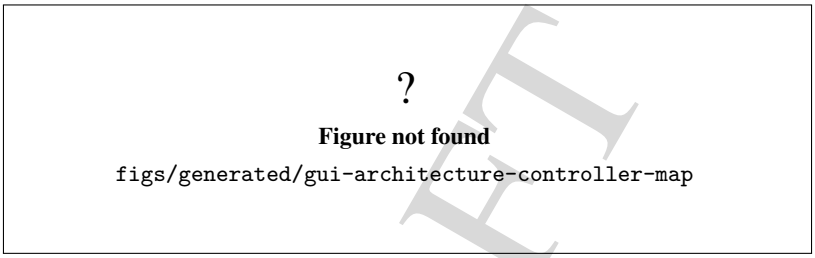


Figure 26.2: Current controller and service responsibilities around the GUI facade.

Figure 26.2 shows the current split: model lifecycle, simulation control, scene tools, property editing, plugin browsing, trace rendering, and dialog utilities are all already separated from the main widget tree.

26.3 Signal, slot, and callback wiring

The GUI uses a mix of Qt signals/slots and explicit callback injection.

The direct Qt links still matter:

- `QAction` triggers map to legacy `MainWindow::on_action..._triggered()` slots;
- `QGraphicsScene::changed`, `focusItemChanged`, and `selectionChanged` are connected back into `MainWindow` scene handlers;
- the plugin tree, property editor tree, and several dialogs still use direct widget callbacks.

The extracted controllers then take over the heavier work through injected dependencies:

- `MainWindow` creates `ModelLifecycleController` with callbacks for console logging, model load/save, UI refresh, scene connection management, and textual synchronization;
- `MainWindow` creates `SimulationCommandController` with callbacks for command logging, action refresh, model checking, text-to-simulation synchronization, and clearing simulation outputs;
- `MainWindow` creates `PropertyEditorController` with callbacks for model-language refresh, component/data-definition refresh, C++ export refresh, model-image generation, and pane/action updates;
- `MainWindow` creates `SimulationEventController` with callbacks for action refresh, event panes, debug tree refresh, and graphical-model refresh;
- `ModelGraphicsView` accepts function callbacks for mouse, wheel, context-menu, and graphical-model event handling instead of hard-coding those paths in the widget class.

This is the main architectural fact to keep in mind when editing the GUI: the code is no longer a

single nested slot file, but it is also not a pure signal-driven MVC stack. The current system is a controlled hybrid. Some behavior still routes through MainWindow wrappers, while the actual orchestration lives in small controllers and services.

?

Figure not found

`figs/generated/gui-architecture-signal-flow`

Figure 26.3: Signal, slot, and callback flow in the current GUI architecture.

Figure 26.3 should be read as a routing diagram, not as an ownership diagram. Ownership and flow are related here, but they are not the same thing.

26.4 Model open and close flow

The model lifecycle is managed by `ModelLifecycleController`, but the compatibility slot still lives in `MainWindow`.

The open flow currently works like this:

1. the user activates `on_actionModelOpen_triggered()` or a recent-file action;
2. `MainWindow` delegates to `ModelLifecycleController::onActionModelOpenTriggered()` or `openModelFile()`;
3. the controller resolves the file path and prefers a companion `.gui` file when the input is a `.gen` file and a matching graphical file exists;
4. the controller logs the command into the console and calls the injected model-load callback;
5. after a successful load, the controller marks the model as loaded, rebuilds the UI for the new model, clears dirty flags, updates recent-file state, and clears the undo stack when available;
6. `MainWindow` then refreshes its recent-model menu and any dependent tabs or actions.

The close flow stays in `MainWindow` for now, but it still interacts with the same lifecycle state:

- it checks whether the current model exists;
- it closes only the current tab/model pair, not every loaded model;
- it refreshes the action state after teardown.

This is a good example of the current compromise. The lifecycle orchestration is extracted, but the compatibility facade still hosts some tab and model-management behavior that has not yet been fully moved into the controller layer.

Figure 26.4 captures the important user-visible promise: opening a model is not just file I/O, because it also restores the GUI state that goes with the loaded model.

26.5 Simulation action flow

Simulation actions use a two-layer guard:

?

Figure not found

`figs/generated/gui-architecture-model-open`

Figure 26.4: Current model open and close lifecycle around the GUI facade.

1. `SimulationController` validates readiness, current-model existence, and the model-check/text-sync preconditions;
2. `SimulationCommandController` translates the action into start, step, pause, resume, or stop behavior and then calls `ModelSimulation`.

The command controller also updates `AnimationTransition` flags and inserts the corresponding command into the console. That keeps simulation bookkeeping aligned with the textual command history and the graphical animation state.

Event reactions are handled separately by `SimulationEventController`:

- model-check success updates the graphical data-definition layer;
- replication and simulation start/end events update the progress bar, event table, and tab state;
- process and entity events drive the simulation and graphical progress paths;
- the `setOnEventHandlers()` bridge registers the kernel callbacks through the compatibility facade.

Trace output is routed through `TraceConsoleController`, not through the simulation command path. That separation matters because trace rendering is presentation logic, while simulation commands are execution logic.

?

Figure not found

`figs/generated/gui-architecture-simulation-flow`

Figure 26.5: Current simulation action flow through the GUI layer.

Figure 26.5 is the cleanest place to see the current split between command issuance, simulation execution, and event-driven refresh.

26.6 Scene, property editor, and plugin catalog boundaries

Three GUI subsystems deserve separate mention because they are still coupled to the scene but no longer live entirely inside `MainWindow`:

- the plugin catalog is rebuilt from the kernel `PluginManager`, grouped by category, and kept as a drag source rather than a drop target;
- the property editor syncs scene selection into `PropertyEditorGenesys` and `ObjectPropertyBrowser`, then defers heavy refreshes when a property commit is already in progress;
- the graphics view owns the current `ModelGraphicsScene` wrapper and forwards context-menu, mouse, wheel, and selection events through injected callbacks.

That boundary is deliberately conservative. The GUI still uses scene-selection callbacks, direct widget refreshes, and a few legacy slots, but the meaningful logic is already in controller classes where it can be tested and reasoned about more locally.

26.7 Current limitations

The current snapshot has a few important limits:

- `MainWindow` is still the compatibility facade, so the GUI is not yet a fully clean presenter/service split;
- several flows still rely on direct wrapper slots instead of a single modern event bus;
- `ModelGraphicsView` and `ModelGraphicsScene` still expose a legacy callback-heavy interface because the scene interactions are not fully signal-based yet;
- startup success does not prove that every GUI workflow is correct, only that the current wiring is coherent enough to initialize and delegate.

For maintainers, the practical rule is simple: change the extracted controller when the behavior is local to that responsibility, and change `MainWindow` only when the glue itself needs to change.

27. Worker and Remote Execution

This chapter documents the current Worker architecture as it exists in the active branch on 2026-07-23. It is based on the standalone worker sources, the reusable embedded worker runtime, the root repository guidance, and the current worker application README. The chapter describes the current snapshot, not a future idealized distributed platform.

Objective. Explain how the Worker process is composed, how requests move through transport, routing, sessions, and job handling, and where the current security and deployment boundary stops.

Audience. Developers and maintainers who need a faithful map of the Worker before changing HTTP routes, session behavior, job handling, deployment assumptions, or embedded-runtime integration.

Scope. Cover the standalone `genesys-worker` executable, the reusable `WebWorkerRuntime` wrapper, the HTTP transport layer, token and session management, the current API surface, worker jobs, logging, and the local/private deployment boundary.

Status. Confirmed by repository inspection on 2026-07-23. The chapter is conservative about maturity: the Worker exposes discovery and job-oriented endpoints, but it does not provide TLS, quotas, public-Internet approval, or background distributed execution.

The chapter follows Figure 27.1, Figure 27.2, Figure 27.3, Figure 27.4, Figure 27.5, and Figure 27.6.

27.1 Worker process boundary

The current Worker is not a GUI widget and not a kernel subsystem. It is an executable application that owns its own entry point, its own HTTP stack, and its own session registry. The standalone target is `genesys_worker_app`, which produces the executable name `genesys-worker`; the build also keeps the alias target `genesys_worker`.

The standalone entry point in `BaseGenesysWebApplication` wires together these objects:

- `TokenService`;
- `SessionManager`;
- `WorkerJobManager`;
- `SimulatorSessionService`;
- `ApiRouter`;
- `SimpleHttpServer`.

The command-line wrapper accepts `-port` and `-max-requests`. The port defaults to 8080 and the request cap defaults to unlimited.

The same stack is also reused by `WebWorkerRuntime`, which is the embedded lifecycle wrapper used by GUI-facing code. That wrapper keeps the worker stack behind a start/stop/restart interface and exposes a bounded snapshot of runtime state, recent events, and recent errors. The important architectural point is that the embedded runtime reuses the same worker core instead of reimplementing the HTTP and session logic in the GUI.

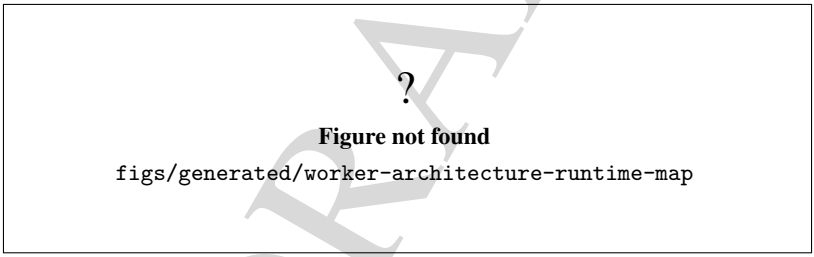


Figure 27.1: Current Worker composition roots and the reusable embedded runtime.

Figure 27.1 is the quickest way to see that the Worker exists in two forms: a standalone HTTP process and an embedded runtime wrapper that reuses the same core services.

27.2 HTTP transport and request routing

The HTTP transport is intentionally small. `SimpleHttpServer` is a blocking, one-request-per-connection server that accepts a connection, reads one bounded HTTP/1.1 request, delegates to the route handler, serializes the response, and closes the socket. It does not implement keep-alive or chunked transfer decoding.

The transport layer enforces a 1 MiB request cap and 5 second socket read/write timeouts. It maps transport failures to standard JSON responses: invalid requests become 400, timeouts become 408, oversized requests become 413, and unexpected server errors become 500. The server binds to `INADDR_ANY` on the configured port.

`ApiRouter` keeps the route logic separate from transport. It resolves public discovery routes, ses-

sion routes, simulation routes, and worker-job routes, then returns a compact JSON response model with stable error codes. The router is explicit on purpose: the current source tree is easier to audit when the public routes are spelled out in one place.

?

Figure not found

`figs/generated/worker-architecture-request-flow`

Figure 27.2: Current HTTP transport and request-routing flow in the Worker.

Figure 27.2 should be read as a boundary diagram. The transport layer knows how to read, bound, and serialize requests, while the router knows how to interpret the API surface.

27.3 Session, token, and workspace model

Authenticated worker sessions are owned by `SessionManager`. Each session binds one bearer token to one simulator instance, one session identifier, and one filesystem workspace rooted under the system temporary directory. The current root is `temp_directory_path()/genesys_worker_sessions`.

`TokenService` generates two opaque identifiers: a session id with the `sess_` prefix and a bearer token encoded as hex. The session manager stores sessions in a token-to-session map, and each created session receives a fresh `Simulator` instance from the provided factory. The simulator is then initialized with the default plugin set so that language import and snapshot loading can resolve built-in model elements immediately.

The workspace boundary matters. Persistence operations are limited to the session workspace directory, and filename inputs are treated as safe basenames only. The current rules reject absolute paths, parent-path traversal, slashes, backslashes, and names that do not match the safe filename pattern. That keeps the Worker from acting as a general file server.

?

Figure not found

`figs/generated/worker-architecture-session-flow`

Figure 27.3: Current session, token, and workspace lifecycle.

Figure 27.3 shows the core promise of the current Worker: every authenticated session gets isolated

state, but that state is still local to the running process.

27.4 Current API surface

The public and authenticated routes are split into discovery, session, simulation, and worker-job groups.

The discovery surface currently includes:

- `GET /health`;
- `POST /api/v1/auth/session`;
- `GET /api/v1/worker/info`;
- `GET /api/v1/worker/capabilities`.

The session-scoped surface currently includes:

- `GET /api/v1/simulator/info`;
- `POST /api/v1/models`;
- `GET /api/v1/models/current`;
- `POST /api/v1/models/save`;
- `POST /api/v1/models/load`;
- `GET /api/v1/simulation/status`;
- `POST /api/v1/simulation/config`;
- `POST /api/v1/simulation/run`;
- `POST /api/v1/simulation/step`;
- `POST /api/v1/worker/models/import-language`;
- `POST /api/v1/worker/jobs`;
- `GET /api/v1/worker/jobs/{jobId}`;
- `POST /api/v1/worker/jobs/{jobId}/run`;
- `GET /api/v1/worker/jobs/{jobId}/result`.

The discovery endpoints do not require a bearer token. The remaining routes do. The capability payload intentionally exposes the current implementation boundary: session API support is present, synchronous run and step are present, model upload through plain text import is present, background execution is false, and streaming events are false. The job-oriented flags currently describe the current API surface, not a background distributed executor.

?

Figure not found

`figs/generated/worker-architecture-api-surface`

Figure 27.4: Current Worker API surface and authentication boundary.

Figure 27.4 is the quickest reference when deciding whether a client needs only discovery or a full authenticated session.

27.5 Worker jobs and synchronous execution

Worker jobs are stored in memory by `WorkerJobManager`. A newly created job receives a monotonic identifier of the form `job-<n>` and a creation marker of the form `seq-<n>`. The creation marker preserves ordering, while the job id is the public handle exposed by the API.

Job creation requires a current model in the session. The implementation then writes a snapshot file named `job_<id>.gen` into the session workspace so that later execution can be decoupled from live model mutations. That snapshot is the execution source for the `/run` path.

Execution is synchronous. The current run path loads the stored snapshot back into the session simulator, calls `simulation->start()`, and records a minimal terminal summary with simulated time, replication numbers, replication length, warm-up period, and optional paused state. If execution fails, the job is marked failed and the terminal summary is still preserved when possible.

The query path and result path remain session-scoped. A job that belongs to another session is treated as inaccessible, and a terminal result is not returned until the job has reached a finished or failed state with stored terminal data.

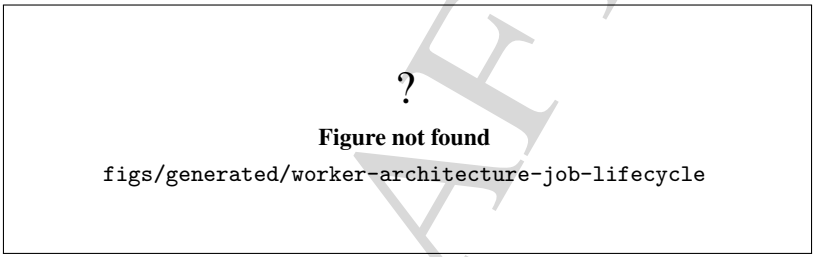


Figure 27.5: Current worker-job lifecycle and synchronous execution path.

Figure 27.5 captures the current job model accurately: jobs are durable only within the running process, and execution is synchronous even though the API is shaped like a worker system.

27.6 Security, logging, and recovery boundary

The current Worker boundary is local or private network only. The standalone server binds to all interfaces on the selected port, but the repository guidance does not approve direct public-Internet exposure. The current code does not implement TLS, credential rotation, quotas, per-request authorization beyond the opaque bearer token, or service-level isolation controls. Those are future hardening items, not current guarantees.

The transport and API layers do provide basic failure handling. Invalid HTTP requests become 400 responses; expired or malformed session tokens become 401 responses; missing current models become 409 responses; missing or invalid worker jobs become 404 or 409 depending on the failure mode; transport overload becomes 413; and unexpected exceptions become 500. That behavior is useful for debugging, but it is not a security story by itself.

`WebWorkerRuntime` keeps recent events and recent errors in bounded in-memory deques, together with a thread-safe snapshot of the worker state. That gives the GUI or other callers a lightweight way to observe whether the background worker is starting, running, stopping, or failed. It does not make job state persistent across process restarts, and it does not replace external monitoring or audit logging.

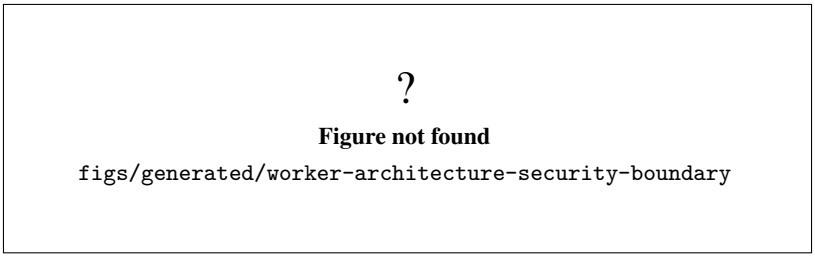


Figure 27.6: Current Worker deployment and recovery boundary.

Figure 27.6 is the security summary for the current codebase. The Worker is usable, but it is still a local/private service with explicit hardening remaining before any stronger deployment claim would be justified.

27.7 Practical reading order

For maintainers, the most useful reading order is:

1. `BaseGenesysWebApplication` for the standalone process boundary;
2. `SimpleHttpServer` for transport limits and response mapping;
3. `ApiRouter` for the route table and public wire contract;
4. `TokenService`, `SessionManager`, and `SessionContext` for authentication scope and workspace isolation;
5. `SimulatorSessionService` for model, simulation, and job behavior;
6. `WorkerJobManager` for job storage and terminal result handling;
7. `WebWorkerRuntime` for the embedded lifecycle wrapper.

As illustrated by Figures 27.1, 27.2, 27.3, 27.4, 27.5, and 27.6, the current Worker is already a coherent application boundary, but it is still intentionally conservative in security, transport, and execution model.

28. AI and External Integrations

This chapter documents the current AI and external-integration boundary as it exists in the active branch on 2026-07-23. It is based on the current tool libraries, GUI sources, model-specific example sources, CMake targets, and the repository guidance that classifies AI, external code, and process execution as explicit boundaries. The chapter describes the current snapshot, not a future idealized integration platform.

Objective. Explain how the current AI assistant stack is configured, how it sends provider requests, how audit and secret handling work, and how the current external code and process integrations are wired.

Audience. Developers and maintainers who need a faithful map of the current AI and external-integration boundary before changing providers, permissions, secret handling, model-specific AI flows, or runtime execution hooks.

Scope. Cover the AI assistant backend in `source/tools/AIAssistant`, the graphical AI Assistant application, the model-specific AI example, the current permission model, the audit and keyring flows, and the currently implemented external code/process integrations. Keep the discussion limited to the code tree that is actually present in the repository.

Status. Confirmed by repository inspection on 2026-07-23. The chapter is conservative about maturity: the AI assistant can talk to providers, keep an audit log, and drive model construction in controlled mode, but that does not imply public deployment readiness or scientific validation of the resulting workflows.

The chapter follows Figure 28.1, Figure 28.2, Figure 28.3, Figure 28.4, Figure 28.5, and Figure 28.6.

28.1 Current AI boundary

The current AI stack is split into three visible layers:

- `genesys_tools`, which links the assistant backend with the rest of the simulator support libraries;
- `genesys_ai_provider`, which contains the provider clients and the conversation service used to talk to OpenAI-compatible or Anthropic-style APIs;
- `genesys_ai_assistant_gui_application`, which exposes the GUI front end for manual configuration, prompt entry, execution, and audit review.

At the model level, the AI-specific component path is implemented by `AISupport` and `AIAssistant`. The example application `StartAIAssistant` wires those classes into a runnable model-specific flow and is a useful concrete reference for how the AI boundary is consumed from a simulation model.

?

Figure not found

`figs/generated/ai-integration-boundary-map`

Figure 28.1: Current AI integration boundary across support libraries, GUI, and model-specific code.

Figure 28.1 is the shortest way to see that the AI work is not one monolithic subsystem. The GUI, the provider transport, the assistant facade, and the model-specific example each own a different piece of the contract.

28.2 Configuration and permissions

The assistant configuration is intentionally deny-by-default. The public configuration structure exposes these controls:

- provider selection: `OpenAI`, `Anthropic`, `Local`, or `CustomHttp`;
- model identifier;
- base URL;
- API key environment-variable name;
- temperature, timeout, and token budget;
- network access permission;
- model-mutation permission;
- simulation-execution permission;
- filesystem-write permission;
- sandbox mode;
- dry-run mode.

The lower-level conversation service currently supports `OpenAI`, `Anthropic`, and `Local`. The higher-level assistant facade also accepts `CustomHttp` and maps it to the OpenAI-compatible local client

with a custom base URL. That distinction matters because it shows which option is a direct provider implementation and which option is a facade-level compatibility choice.

The readiness check in `AIAssistantDefaultImpl` requires a configured provider, a non-empty model name, a key when the selected provider requires one, and a base URL when the provider is `CustomHttp`. It also makes clear that model construction and simulation still require the explicit `allowModelMutation` and `allowSimulationExecution` gates.

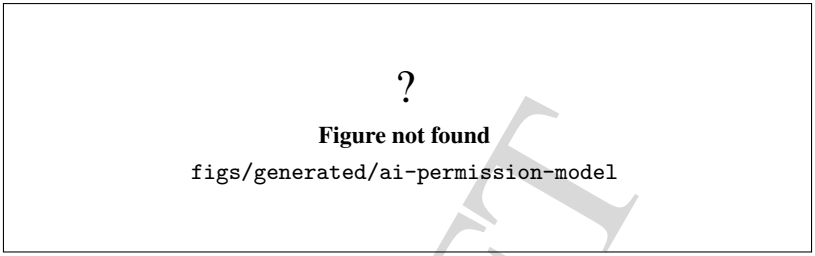


Figure 28.2: Current permission model and readiness gates for the AI assistant.

Figure 28.2 should be read as a control surface, not a promise. The GUI exposes the switches, but each operation still decides independently whether it is allowed to proceed.

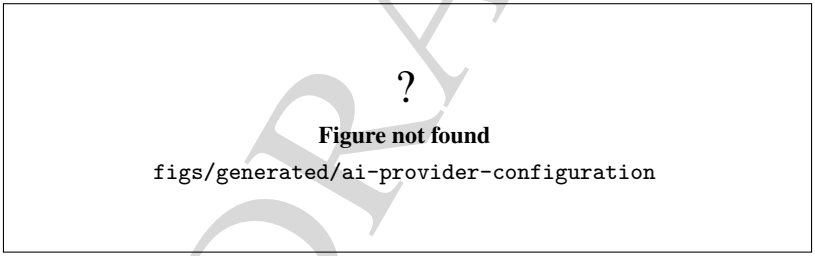


Figure 28.3: Current provider configuration paths and transport assumptions.

28.3 Provider pipeline

The provider pipeline is split into a provider-neutral request layer and provider-specific HTTP clients.

`AIConversationService` builds `ChatMessage` sequences from the system instructions, the conversation history, and the current user prompt. It owns the selected provider client, optionally loads a key from the configured environment variable, and then sends a synchronous chat-completion request with the configured model, temperature, token budget, and reasoning hint.

The transport layer in `HttpProviderClientBase` shells out to `curl`. The request body is written to a temporary file, the API key is written to a separate 0600 curl-config file, and both files are unlinked right after the request begins. That keeps the key out of the command line and out of shell history. The concrete clients then map the raw response body back to the provider-neutral response structure.

The current provider specializations are:

- `OpenAIProviderClientImpl`: OpenAI chat completions, optional organization and project headers, and reasoning-mode support through `highReasoningMode`;
- `AnthropicProviderClientImpl`: Anthropic Messages API with the top-level system field, the required version header, and the extended-thinking budget;
- `LocalProviderClientImpl`: OpenAI-compatible local servers such as Ollama, LM Studio, or llama.cpp, with no key required unless the local server explicitly wants one.

The assistant facade adds one more compatibility path: `CustomHttp` is implemented through the local OpenAI-compatible client with a custom base URL. That keeps the current code aligned with existing OpenAI-style endpoints without inventing a separate transport stack.

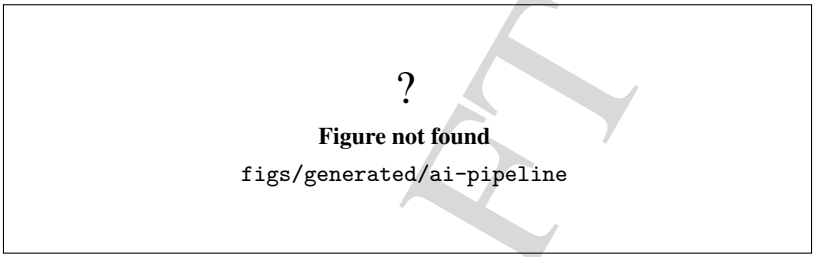


Figure 28.4: Current AI request pipeline from prompt to provider response.

Figure 28.4 makes the operational sequence explicit: the assistant does not talk to the provider directly, and the provider client does not know about the GUI.

28.4 Audit and secret handling

The current audit trail is append-only and local. `AIAuditLog` writes JSON Lines under the XDG data directory, typically `~/.local/share/genesys/ai_audit.log`, and can export the same records to CSV. Each entry stores the operation name, provider, model, prompt preview, success flag, dry-run flag, duration, and a capped error summary. It does not store API keys.

`AISecretStore` is the current OS keyring helper. On Linux it uses `secret-tool` when available, which means the GUI can save, load, and clear a key in the session keyring. If the helper is unavailable, the code falls back to an environment variable or a key typed into memory. The keyring helper shells out carefully, but it is still a host-level integration, not an in-process secret vault.

The GUI reflects that design. The AI Assistant window has dedicated controls to apply configuration, save and load a keyring entry, refresh the audit table, and export the log. It also shows the masked key only. The audit table is a read-only view of recent log entries loaded from the same local audit file.

Figure 28.5 should be read as a local boundary diagram. The audit log is for inspection, and the keyring helper is for convenience on a host that already has the required desktop services.

28.5 Sandbox and workflow control

The backend workflow is intentionally staged.

?

Figure not found

`figs/generated/ai-audit-flow`

Figure 28.5: Current audit and secret-handling flow for the AI assistant.

`analyzePrompt()` performs a provider call that summarizes the scenario and extracts the current GenESyS vocabulary. `createModelPlan()` performs a second provider call that returns a structured Markdown plan. `buildModel()` asks for a `.gen` model, writes it to a temporary file, loads it through `SimulatorFacade::loadModel()`, and deletes the temporary file afterwards. `configureSimulation()` can use the provider to extract simulation parameters from the experiment description, then applies the values to the current model. `runSimulation()` calls `simStart()` synchronously. `collectResults()` renders current simulation responses.

Two flags change the shape of that workflow:

- `sandboxEnabled` clears the current model before `buildModel()` starts, so the generated model begins from an empty canvas;
- `dryRun` converts the mutating steps into previews, so the code reports what it would do instead of changing the model or starting the simulation.

The GUI exposes both flags. It also exposes a filesystem-write checkbox, but the inspected backend gates are centered on network access, model mutation, simulation execution, readiness, and provider availability. I did not confirm a dedicated backend enforcement path for filesystem writes in the current execution flow.

?

Figure not found

`figs/generated/ai-sandbox-boundary`

Figure 28.6: Current sandbox and dry-run boundary for the AI workflow.

Figure 28.6 is the key guardrail for readers who want to understand the difference between a preview and a real model mutation.

28.6 GUI and model-specific example

The AI Assistant GUI is a Qt6 application with four tabs: configuration, prompt and plan, execution and results, and audit log. The configuration tab pushes settings into `AIAssistantDefaultImpl`, optionally loads an API key from the keyring or an environment variable, creates the provider client, and refreshes the knowledge base from the simulator facade. The prompt tab collects the model description, experiment description, and freeform prompt. The execution tab drives the individual workflow steps or the full pipeline. The audit tab shows the local JSONL log and can export it as CSV.

The model-specific example `StartAIAssistant` shows a concrete in-model use case. It creates a `Simulator`, auto-loads plugins, constructs an order-routing flow, configures `AISupport` from environment variables, attaches an `AIAssistant` component, sets action mappings for the JSON response, and then runs a short simulation. The example is valuable because it shows the assistant not as a standalone chatbot, but as a model component that controls routing inside the simulation graph.

The current `AIAssistant` component expects an attached `AISupport` definition, renders a prompt with expression placeholders inside braces, asks the support service for a synchronous response, parses JSON fields such as `action`, `confidence`, `explanation`, and `values`, and then forwards the entity through the selected output port. If the response cannot be parsed, or if the action does not match the action mapping, the component routes the entity to the error output port.

28.7 External code and process integration

The external-integration path is broader than the AI assistant itself.

`CppForG` is a model component that compiles user-supplied C++ hooks, loads them as a shared library, and dispatches replication and entity callbacks through function pointers. Its companion `CppCompiler` can compile to an executable, a dynamic library, or a static library, and it can load and unload the resulting shared object.

`PythonForG` is the Python hook component. It stores hook code in the model, uses `PythonRuntime` to validate and execute the hooks, and can optionally forward an entity on error. The Python runtime is compile-time gated through `GENESYS_HAS_PYTHON_INTEGRATION`, which means the source can exist even when the interpreter embedding support is not available in a particular build.

`RSimulator` and `RSimulatorRunner` are the R-side integration pair. The runner writes a temporary script, invokes `Rscript`, and stores last-run status, exit code, stdout, stderr, response payload, and file paths. The component wraps that runner inside a model component so that the simulation graph can call external R code as part of the flow.

These integrations share the same basic design constraints: keep the model ownership separate from the external runtime, bound execution, keep secrets out of logs and argv, and treat the host toolchain or interpreter as an integration dependency rather than as an implicit simulator feature.

28.8 Current limitations

The current code does not justify stronger claims than the ones above.

- The assistant can call providers, but provider access still depends on explicit permissions and a reachable endpoint.
- The audit log is local and append-only; it is not a distributed observability system.
- The keyring helper is optional and host-dependent.
- The GUI exposes a filesystem-write permission control, but I did not confirm a dedicated backend enforcement branch in the inspected execution path.

- `MCP` is not covered here because I did not confirm a current implemented runtime contract in the inspected AI integration path.
- The Python integration remains experimental and compile-time dependent.
- External execution hooks are integration boundaries, not proof of scientific correctness or general deployment safety.

The practical rule for this chapter is simple: document the implemented boundary, name the explicit permissions, and do not turn a functioning integration path into a maturity claim that the current code does not support.

29. Biological and Whole-Cell Extensions

This chapter maps the biological extension surface that exists in the current tree. It is intentionally conservative: it documents implemented classes, didactic examples, and validation boundaries, but it does not claim organism- level predictive validity.

Objective. Summarize the biological, whole-cell, and related extension areas supported by the project.

Audience. Developers and researchers who need a high-level map of the biological extension surface.

Scope. Cover the scope, terminology, and evidence limits that should govern these extensions.

Status. Temporary skeleton replaced with current implemented surface and evidence boundaries. The scientific claim level remains experimental/research-oriented unless a later chapter states otherwise.

29.1 Scope and contract

The repository currently treats whole-cell, biochemical, SBML, and virtual-cell work as experimental or research-oriented. That boundary is stated in `../README.md`, `../docs/ai_assistants/reference/SCIENTIFIC_DOMAINS.md`, and the human decision backlog. The right reading is:

- the code exposes real classes, examples, and tests;
- the examples are didactic unless a source file explicitly says otherwise;
- successful compilation or startup does not imply scientific validation;
- any future claim beyond the current code must bring a real reference, dataset or model identifier,

benchmark, tolerance, procedure, and explicit scope.

The biological surface in the current tree is split across three layers:

- native biochemical definitions and bridges;
- whole-cell state and components;
- model-specific didactic examples.

The key design rule is that biological meaning stays explicit. A biochemical network, a flux-balance network, a compartment transport rule, and a whole-cell state are not interchangeable abstractions.

29.2 Biochemical surface

The biochemical layer currently includes both native deterministic networks and interoperability helpers.

29.2.1 Native deterministic and network-oriented classes

The current surface centers on the following classes:

- `BioSpecies`, `BioParameter`, and `BioReaction` for native species, scalar parameters, and mass-action reactions;
- `BioNetwork` for a deterministic biochemical network runner with fixed-step RK4 over a mass-action ODE system;
- `MetabolicNetwork` and `MetabolicReaction` for flux-style metabolic networks that keep reaction membership and objective metadata explicit;
- `GeneticCircuit`, `GeneticRegulation`, and related genetic circuit helpers for transcriptional regulation surfaces;
- `BioSimulatorRunner` as the command-driven structural runner for validate/simulate/report/import/export workflows;
- `BioSBMLBridge` as the native import/export boundary for SBML text.

Figure 29.1 sketches that surface as it exists today: native biochemical definitions feed either the deterministic runner or the SBML bridge, while the flux-style network types stay available for the whole-cell examples that need them.

?

Figure not found

`figs/generated/biochemical-network-surface`

Figure 29.1: Current biochemical extension surface and its SBML boundary.

29.2.2 Flux-style metabolic models

The metabolic path is deliberately separate from the deterministic network runner. `MetabolicNetwork` stores the reaction membership, objective reaction, objective sense, compartment, and enabled flag.

`MetabolicReaction` stores reactants, products, bounds, objective coefficient, reversibility, and gene rule text.

This is the layer used by the whole-cell examples that exercise FBA-style reasoning. The manual should treat it as a didactic constraint model rather than as a validated biochemical simulator.

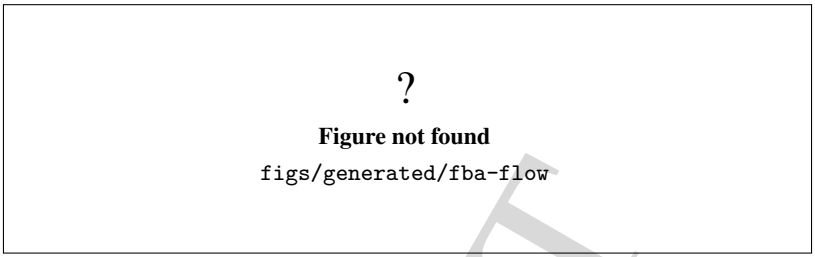


Figure 29.2: FBA-style metabolic flow used by the whole-cell examples.

29.2.3 Compartment transport

Compartment-aware transport is modeled explicitly rather than hidden inside the solver. The main helper classes are:

- `MetabolicStateProjectionComponent` for mapping selected fluxes into global metabolite pools, compartment pools, and pathway activity values;
- `CompartmentExchangeComponent` for moving selected amounts between regions such as extracellular, cytosol, mitochondria, or bud;
- `ResourceAllocationComponent` for distributing RNAP and ribosome resources before expression steps.

Figure 29.3 captures the compartment-aware flow used by the yeast-oriented whole-cell examples.

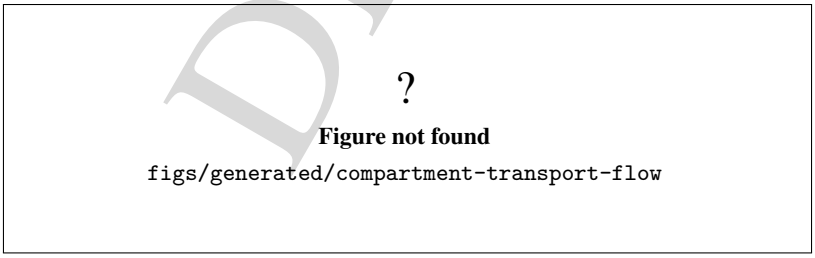


Figure 29.3: Compartment transport and projection in the current whole-cell examples.

29.3 Whole-cell surface

The whole-cell layer centers on `WholeCellState`. That class is the shared container used by the whole-cell components. Its current responsibilities include:

- integer molecule counts for SSA-driven expression and degradation;
- metabolite pools, including compartment-scoped pools;
- pathway activity values used by growth, checkpoint, and stress routing;
- resource budgets for expression allocation;
- cell volume, mass, time, step count, lifecycle phase, generation count, viability, and last-division time;
- JSON path storage for the CovertLab/WholeCell fixed-constants and parameters inputs used by the didactic M. genitalium-oriented path.

The main whole-cell components are:

- `StochasticReactionRule` and `StochasticReactionComponent` for Gillespie-style SSA updates;
- `StochasticTranscription` and `StochasticTranslation` for tau-leaping expression;
- `CellGrowthComponent` for growth and density-aware mass change;
- `CellCycleCheckpointComponent` for lifecycle phase annotation and clock advancement;
- `CellFateDecisionComponent` and `CellDivisionEvent` for viability/division routing;
- `PathwayStressResponseComponent` for sustained low-pathway stress;
- `BioStateProjectionComponent` and `MetabolicStateProjectionComponent` for moving biochemical outputs into whole-cell state;
- `CompartmentExchangeComponent` for transport between regions;
- `EukaryoticCellCycleComponent` and `FtsZPolymerizationComponent` for the more specific eukaryotic and division-oriented examples;
- `MetabolicSubmodelComponent` for compact metabolism examples;
- `ResourceAllocationComponent` for resource partitioning examples.

The components are intentionally explicit: they share one state object, but they do not silently merge semantics. That is why the examples can explain how a biochemical output becomes a resource budget, a growth driver, or a lifecycle checkpoint without pretending that these are the same thing.

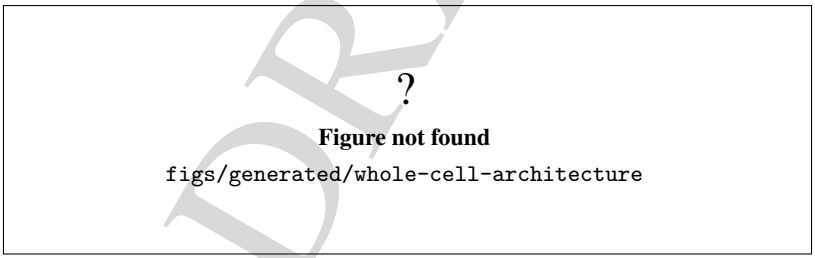


Figure 29.4: Current whole-cell architecture around `WholeCellState`.

29.4 Didactic examples

The model-specific whole-cell examples are the clearest evidence that the surface exists in executable form. They live under `source/applications/modelSpecific/wcm/` and persist matching `models/*.gen` snapshots.

The current families are:

- `WcmSimpleSsa`: one-gene SSA example with transcription, degradation, translation, and protein turnover;

- `WcmGeneExpression`: central-dogma example with four genes and stochastic transcription, translation, and degradation;
- `WcmBioNetworkBridge`: minimal deterministic BioNetwork to stochastic whole-cell bridge;
- `WcmWholeCellMvp`: compact whole-cell analog combining deterministic metabolism, projection, expression, growth, and division;
- `WcmFluxBalanceWholeCellMvp`: FBA-driven whole-cell MVP;
- `WcmCompartmentTransportWholeCellMvp`: FBA plus compartment-aware transport and exchange;
- `WcmPathwayStressWholeCellMvp`: stress routing when the monitored pathway stays below threshold;
- `WcmLifecycleStress`: starvation and death routing example;
- `WcmMgenitaliumKarr2012`: M. genitalium analog inspired by Karr et al. 2012, with explicit note that it is a didactic analog, not a canonical validated public whole-cell model;
- `WcmYeastDidacticWholeCellMvp`: yeast-oriented didactic FBA/whole- cell example;
- `WcmYeastEukaryoticWholeCellMvp`: budding-yeast-inspired eukaryotic whole-cell example with compartments and cell-cycle coordination.

The yeast examples are the most useful way to explain the architecture to a reader. They combine FBA-style metabolism, compartment transport, expression, growth, and lifecycle control in one didactic pipeline without presenting the result as a validated public yeast whole-cell model.

?

Figure not found

`figs/generated/yeast-mvp-structure`

Figure 29.5: Yeast-oriented whole-cell example structure.

?

Figure not found

`figs/generated/pathway-stress-example`

Figure 29.6: Pathway stress example in the current whole-cell surface.

29.5 Evidence and validation

The current whole-cell and biochemical code is not only descriptive; it is also covered by focused unit tests. The most relevant current test files are:

- `../source/tests/unit/test_wcm_plugins.cpp`;
- `../source/tests/unit/test_simulator_runtime.cpp`.

Those tests confirm, among other things, that:

- `WholeCellState` stores and returns molecule counts, metabolite pools, compartment pools, pathway activities, resource budgets, cell geometry, and lifecycle metadata;
- `StochasticReactionRule` handles zero-rate, zero-count, unimolecular, and bimolecular propensity cases;
- `BioNetwork` and `BioSimulatorRunner` expose the current deterministic runner and SBML import/export surface;
- `MetabolicStateProjectionComponent` and `CompartmentExchangeComponent` participate in executable whole-cell pipelines;
- `CellCycleCheckpointComponent` participates in executable whole-cell pipelines;
- `CellFateDecisionComponent` participates in executable whole-cell pipelines;
- `PathwayStressResponseComponent` participates in executable whole-cell pipelines;
- `EukaryoticCellCycleComponent` participates in executable whole- cell pipelines;
- SBML import/export currently round-trips the native biochemical representation through the implemented bridge behavior.

Figure 29.7 summarizes the evidence ladder that should be used when reading this chapter.

?

Figure not found

`figs/generated/evidence-ladder`

Figure 29.7: Evidence ladder for the biological and whole-cell surface.

29.6 Scientific limits and future work

The current surface is useful, but it is not a validated organism model. Specific limits matter:

- `WcmMgenitaliumKarr2012` is an analog inspired by the Karr et al. 2012 whole-cell paper, not a canonical public model with full scientific validation in this repository;
- the yeast examples are didactic architecture examples, not published validated yeast whole-cell simulations;
- `HUM-VC-001` in the human backlog still owns the choice of first bounded virtual-cell use case;
- `HUM-VC-002` still owns the SBML supported subset and compatibility target;
- any future scientific claim must include a real reference, dataset or model identifier, benchmark, tolerance, procedure, and explicit maturity statement.

The practical takeaway is simple: use this chapter to locate the current implementation surface and its validation boundary, not to infer a stronger scientific claim than the code and tests support.

30. Testing and Validation

This chapter documents the current testing and validation surface as it exists in the repository now. It is based on the active root README, the local-agent runbook, `docs/ai_assistants/STATUS.md`, the validation ledger, the current `CMakePresets.json`, and `source/tests/CMakeLists.txt`. It records the observed baseline for the `WorkInProgress` branch at commit `7097bc395d11b269692e62d39021cce220e0b1ad` on 2026-07-23.

This chapter does not treat a successful build as scientific validation. It separates configure/build evidence, CTest inventory, focused sanitizer checks, and scientific or numerical validation.

Objective. Describe the available test layers, the current validation presets, the meaning of the recorded evidence, and the current limits of the repository's validation surface.

Audience. Developers, maintainers, and reviewers who need to judge the current quality bar for code, runtime, and documentation changes.

Scope. Cover Google Test, CTest, unit tests, smoke tests, regression tests, integration checks, numerical tests, scientific validation, ASan, UBSan, LSan, the focused leak path, the test presets, the expected disabled tests, and how to interpret the observed results.

Status. Confirmed by source inspection and local command execution on 2026-07-23. The chapter records the current validation surface rather than a hypothetical future gate.

The chapter follows this sequence: Figure 30.1, Figure 30.2, Figure 30.3, Figure 30.4, and Figure 30.5.

30.1 Validation layers

The repository uses several validation layers, each with a different claim. The layers should be read from bottom to top:

- configure and build confirm that the current preset and target graph are still coherent for the selected scope;
- unit and regression tests confirm narrow code contracts and selected runtime invariants;
- smoke tests confirm that small startup and execution paths still launch and terminate cleanly;
- focused sanitizer runs confirm a bounded ownership or lifetime path;
- scientific validation requires explicit formulations, references, fixtures, expected values, and independent review where appropriate.

The important boundary is that the layers do not imply each other. A green build does not prove a green CTest run. A green CTest run does not prove scientific correctness. A focused sanitizer does not prove repository-wide leak freedom.

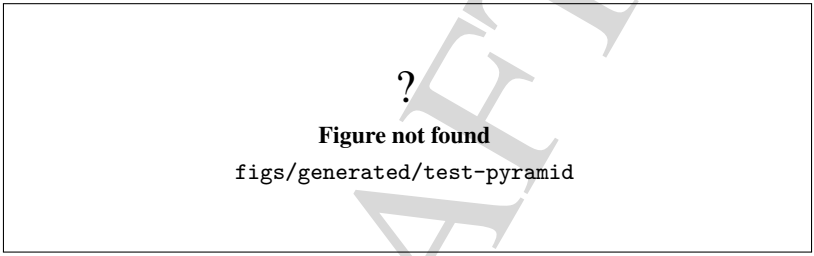


Figure 30.1: Current validation pyramid and claim strength.

Figure 30.1 should make the claim strength obvious. The lowest layers are about coherence and narrow contracts; the upper layers are about repeatable behavior and bounded evidence.

30.2 Current preset matrix

The current test presets are defined in `CMakePresets.json`:

- `tests-unit`;
- `tests-kernel-unit`;
- `tests-smoke`.

The corresponding configure commands are:

- `cmake -preset tests-unit`;
- `cmake -preset tests-kernel-unit`;
- `cmake -preset tests-smoke`.

The matching build commands are:

- `cmake -build -preset tests-unit -parallel "$(nproc)"`;
- `cmake -build -preset tests-kernel-unit -parallel "$(nproc)"`;
- `cmake -build -preset tests-smoke -parallel "$(nproc)"`.

The matching CTest commands are:

- `ctest -preset tests-unit -output-on-failure`;

- `ctest -preset tests-kernel-unit -output-on-failure`;
- `ctest -preset tests-smoke -output-on-failure`.

The current inventory snapshot for a fresh tree was:

- `ctest -preset tests-unit -N`: 0 tests visible in the fresh tree;
- `ctest -preset tests-kernel-unit -N`: 34 registered tests, reported as `_NOT_BUILT` because the executables were not yet present when the inventory was captured;
- `ctest -preset tests-smoke -N`: 3 tests, namely `smoke_simulator_start`, `test_continuous_system` and `test_lsode`.

That snapshot is useful because it proves the presets exist and the smoke suite is intentionally small. It is not a substitute for the full build and test execution history recorded in the status documents.

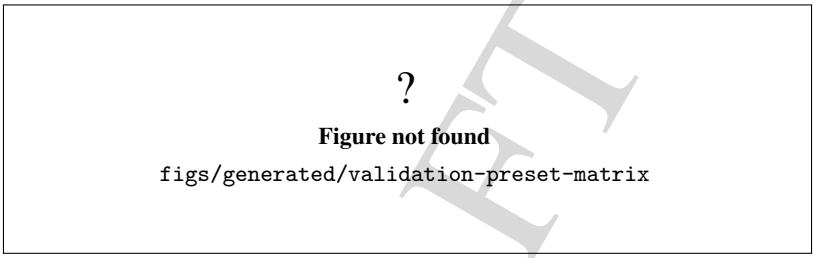


Figure 30.2: Current test preset matrix and build-tree separation.

Figure 30.2 should make the separation between presets and build trees explicit. The chapter intentionally treats the presets as separate surfaces because a change in one tree must not silently alter another.

30.3 What the suites cover

The current suite coverage is intentionally uneven, and the chapter should say so directly instead of smoothing it over.

The unit/regression surface currently includes focused executables such as:

- `genesys_test_tools_legacy_solver_regression`;
- `genesys_test_search_remove_runtime`;
- `genesys_test_queue_statistics_lifecycle`;
- `genesys_test_station_statistics_lifecycle`;
- `genesys_test_delay_statistics_lifecycle`;
- `genesys_test_resource_accounting_lifecycle`;
- `genesys_test_simulator_plugin_completion_lifetime`;
- `genesys_test_optimizer_ownership_contract`.

The smoke surface currently includes:

- `smoke_simulator_start`;
- `test_continuous_system`;
- `test_lsode`.

The status ledger still records the latest exact core inventory as:

- registered: 1,721;

- executed/passed: 1,717;
- failed: 0;
- disabled: 4 historical duplicate Search/Remove blocks.

That exact baseline is a historical current-state record, not an eternal promise. It should be refreshed when the production test graph changes.

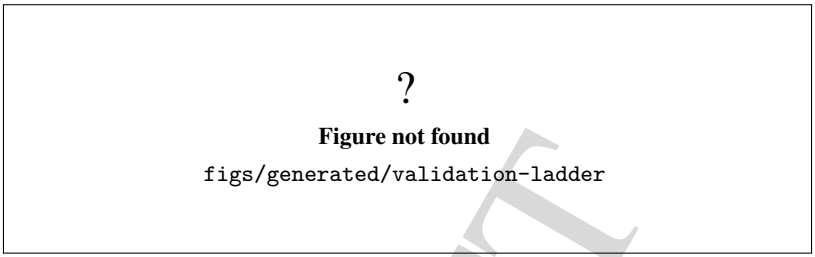


Figure 30.3: Current validation ladder and claim boundaries.

Figure 30.3 should show that a user-visible startup claim is weaker than a runtime workflow claim, and that both are weaker than a scientific validity claim.

30.4 Focused sanitizer path

The repository does not expose a universal sanitizer preset. Instead, it has a focused sanitizer-enabled executable in `source/tests/CMakeLists.txt`. The current contract is:

- target: `genesys_test_simulator_plugin_completion_lifetime`;
- compile options under GNU or Clang: `-fsanitize=address` and `-fno-omit-frame-pointer`;
- link option: `-fsanitize=address`;
- runtime environment: `ASAN_OPTIONS=detect_leaks=1:halt_on_error=1` and `LSAN_OPTIONS=exitcode`

The purpose of this target is narrow: it exercises the temporary plugin-model completion path and checks that the bounded ownership/lifetime slice exits cleanly. It proves that the focused path is clean. It does not prove global leak freedom, whole-repository UB freedom, or any broader scientific claim.

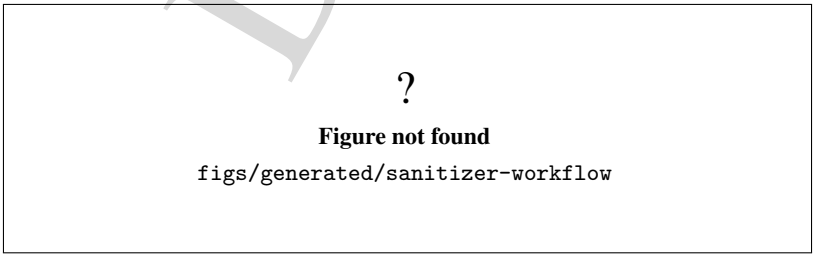


Figure 30.4: Focused sanitizer workflow for the plugin-completion lifetime test.

Figure 30.4 should make the narrowness of the path explicit. The diagnostic is intentionally strong on one slice and intentionally silent about broader repository guarantees.

30.5 Interpreting results

The chapter should interpret results conservatively:

- configure success means the current preset graph is coherent;
- build success means the affected targets compiled in the current tree;
- CTest inventory means the current tree exposes the expected tests;
- CTest execution means the registered tests actually passed in that run;
- sanitizer success means the focused path did not trip the checked class of memory or UB defect;
- scientific validation requires domain-specific evidence, not just a green build or a green smoke suite.

The most important negative rule is the one that prevents overclaim: startup evidence is not functional correctness, and functional correctness is not scientific validity.

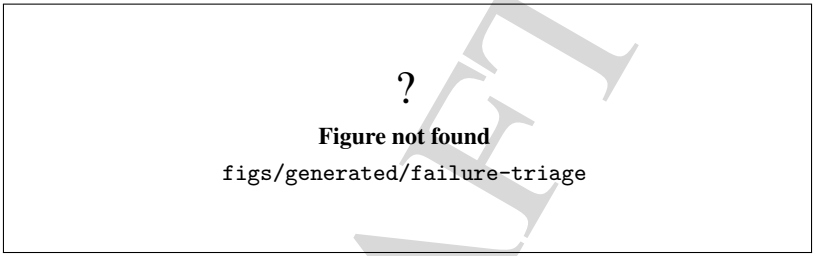


Figure 30.5: Current failure triage and stop-gate behavior.

Figure 30.5 should emphasize the stop rules. If the code and the canonical documentation diverge, if a required target is missing, if a scientific claim lacks a reference, or if a required validation path is not available, the correct action is to stop and report the blocker rather than to inflate the claim.

30.6 Current evidence snapshot

The local documentation pass for this chapter executed the following commands on 2026-07-23 at commit `7097bc395d11b269692e62d39021cce220e0b1ad`:

- `cmake -preset tests-unit;`
- `cmake -preset tests-kernel-unit;`
- `cmake -preset tests-smoke;`
- `cmake -build -preset tests-unit -parallel "$(nproc)";`
- `cmake -build -preset tests-kernel-unit -parallel "$(nproc)";`
- `cmake -build -preset tests-smoke -parallel "$(nproc)";`
- `ctest -preset tests-unit -N;`
- `ctest -preset tests-kernel-unit -N;`
- `ctest -preset tests-smoke -N.`

The configure commands completed successfully. The build commands exercised the current kernel, plugin, and tools graph and then exposed a linker failure in `genesys_test_optimizer_ownership_contract` during both `tests-unit` and `tests-kernel-unit`. The failure was an undefined-reference link error involving `SamplerDefaultImpl1`, `CollectorDefaultImpl1`, and `StatisticsDefaultImpl1`. The smoke tree had already built through its three executables. The inventory commands returned the fresh-tree results listed above.

The current historical evidence ledger also records the following bounded conclusions for the broader repository:

- ordinary unit, kernel, and smoke baselines passed for the recorded historical head;
- a focused ASan/LSan run reduced the exercised leak surface to zero for the plugin-completion lifetime path;
- those results are bounded to their recorded commit, environment, and scope.

Those historical results belong in the status ledger and validation ledger, not as a blanket claim that every future change will behave the same way.

30.7 Practical reading rule

When a developer reads this chapter, the safest interpretation is:

1. use the smallest preset that covers the change;
2. prefer focused regression tests over broad guesses;
3. use the focused sanitizer only for ownership and lifetime questions it was designed to answer;
4. consult the status ledger when you need the last exact inventory or the historical core baseline;
5. stop when the repository evidence and the chapter disagree.

That rule keeps the chapter aligned with the repository's current evidence discipline and prevents a validation story from drifting into assumption.

31. Packaging and CI

This chapter is part of the new editorial skeleton for the developer manual. It keeps the packaging chapter in place while the automation story is rewritten.

Objective. Describe packaging, CI, and the release-oriented automation used by the project.

Audience. Developers and maintainers who need to build, package, or inspect automated pipelines.

Scope. Focus on build artifacts, packaging flow, CI topology, and the current release boundaries.

Status. Temporary skeleton only; the final packaging and CI narrative will be added later.

32. Governance and Contribution

This chapter is part of the new editorial skeleton for the developer manual. It preserves the governance chapter while the contribution policy is reorganized.

Objective. Explain the governance documents, contribution expectations, and review boundaries.

Audience. Contributors, maintainers, and reviewers who need to align their changes with project policy.

Scope. Cover the canonical documents, branch and review conventions, and the current human-decision boundaries.

Status. Temporary skeleton only; the final contribution guide will be expanded later.

Editorial Migration Table

The table below records a first-pass mapping from the legacy chapter set to the new editorial tree. It is intentionally conservative: each legacy chapter is mapped to the most relevant new destination without claiming that the migration is already complete.

Old chapter	Old section	New destination	Status
chapter_genesys_overview.tex	Whole chapter	chapter_application_overview.tex and chapter_main_graphical_application.tex	mapped
chapter_installation_and_build.tex	Whole chapter	chapter_user_installation.tex and chapter_development_environment_and_build.tex	mapped
chapter_graphical_interface_first_steps.tex	Whole chapter	chapter_main_graphical_application.tex and chapter_first_model.tex	mapped
chapter_example_models.tex	Whole chapter	chapter_first_model.tex and chapter_model_specific_and_advanced_examples.tex	mapped
chapter_execution_observation_simulang.tex	Whole chapter	chapter_running_simulations.tex, chapter_results_traces_and_reports.tex, and chapter_model_language_for_users.tex	mapped
chapter_source_organization_architecture.tex	Whole chapter	chapter_repository_architecture.tex	mapped
chapter_kernel_overview.tex	Whole chapter	chapter_kernel_lifecycle_and_managers.tex and chapter_event_scheduling_and_replications.tex	mapped
chapter_components_model_data_plugins.tex	Whole chapter	chapter_model_components_and_data_definitions.tex and chapter_plugin_registration_and_architecture.tex	mapped
chapter_parser_expressions_language.tex	Whole chapter	chapter_parser_architecture.tex and chapter_expression_language_reference.tex	mapped
chapter_applications_tools_tests_evolution.tex	Whole chapter	chapter_application_architecture.tex, chapter_tools_and_algorithms.tex, chapter_testing_and_validation.tex, and chapter_governance_and_contribution.tex	mapped

II Legacy Chapters (Temporary)

33. What is GenESyS

33.1 Introduction

GenESyS is a systems simulator designed to serve, at the same time, as a practical modeling and simulation tool and as a software base for expansion and research. In this book, however, these two dimensions will not be treated simultaneously. The order was deliberately reversed in favor of usage. First, the reader will learn how to work with *GenESyS* as a simulator user. Then, already familiar with the application and models, you will begin to study it as a software system and as an extensible platform.

This choice is important. A common mistake when approaching an academically rich simulation is to start with its most internal aspects: classes, subsystems, parser, *plugins*, execution infrastructure. Although all of this is essential, this is not the best entry point for those who want to really put the system to work. The most productive path begins with use: understanding what the simulator offers, how the user sees it, how models are opened, executed and observed, and how the interface organizes this work.

This chapter, therefore, fulfills a guidance function. It introduces *GenESyS* as a working environment for modeling and simulation, defines the role of the graphical interface as the main point of user interaction, and explains how the rest of the manual is organized. The objective is not yet to teach how to install the system or use its menus in detail, but to establish a sufficiently clear vision so that the next chapters make sense as a natural continuation.

33.2 Purpose of GenESyS

GenESyS should be understood, firstly, as an environment for working with simulation models. This means that its value to the user is not just in "running" a program, but in enabling a complete cycle of working with models: opening, examining, parameterizing, executing, observing, comparing, recording and improving.

From a practical point of view, this environment needs to support a set of activities that the user performs recurrently:

- load existing models;
- inspect its structure;
- change parameters and properties;
- run the simulation;
- observe the dynamics of the model;
- interpret initial results;
- save modified versions of the work.

GenESyS was designed exactly for this type of use. Instead of being just an isolated program, it presents itself as a workspace where the model is the central object of interaction. The user does not only deal with abstract commands, but with concrete entities from the simulation domain: components, flows, supporting data, simulated time, events, results and complementary representations of the same model.

33.2.1 GenESyS as a modeling and simulation tool

When seen from the user's perspective, *GenESyS* is a modeling and simulation tool. This means that the main question stops being "how was the software implemented?" and becomes "how is the model built, executed and observed within the software?".

This shift in perspective is essential to the first part of this manual. The user wants to know:

- how to get the simulator;
- how to get it up and running;
- how to open example models;
- how to use the graphical interface;
- how to run simulations;
- how to understand what the system is showing.

It is exactly this path that will be followed in the first chapters.

33.2.2 GenESyS as an evolving system

Although this chapter treats *GenESyS* primarily as a user tool, it is important to note at the outset that it is also an evolving software project. This means that, behind the graphical application and the models that the user manipulates, there is an internal infrastructure made up of a kernel, parser, *plugins*, auxiliary applications and test engines.

This dimension, however, should not interfere with the reader's initial path. The fact that the system can also be studied and improved as software does not change the learning order proposed in this book. First, the reader will use the simulator. Afterwards, you will study its implementation.

33.3 How the User Works with GenESyS

The user's work with *GenESyS* can be described as a relatively clear flow, although composed of several complementary steps. In a typical situation, the user:

1. obtains the project and prepares the environment;
2. compiles and starts the application;
3. opens the graphical interface;
4. load an already saved model or start a new one;
5. inspects model elements;
6. runs the simulation;
7. observe your behavior and results;
8. saves the work and its variations.

This flow is more important than it might seem at first glance. It defines not only the way the user works with the simulator, but also the organization of the first half of the book. Each of the next chapters will delve deeper into one of these steps.

Table 33.1: User Workflow in GenESyS.

Step	**Practical goal**
Get the system	Access the code, dependencies, and required environment
Compile and start	Run the graphical application
Open a model	Work on a concrete case within the simulator
Explore the GUI	Understand how the interface organizes the model and simulation
Run the model	Observe simulated system dynamics
Inspect results	Read early signals of model behavior and performance
Save and Continue	Preserve work and prepare new iterations

The most important aspect here is that *GenESyS* should not be used as an isolated command system, but as an iterative work environment. The user opens models, explores, modifies, executes, compares and reexecutes. It is this iterative nature that makes the GUI so important in the initial use of the tool.

33.4 The Graphical Interface as the Main Point of Use

In this first part of the manual, the graphical interface will be treated as the main way of using *GenESyS*. This decision is not just editorial; it arises from the expected usage profile of the simulator user. Those who are working with models, especially in the initial learning phase, need a more direct, more visible and more guided form of interaction than the mere manipulation of internal project artifacts.

The GUI precisely fulfills this role. It organizes the model as a visual and operational object. Through it, the user can:

- open saved models;
- observe its structure;
- browse elements and properties;
- trigger simulation commands;
- monitor system behavior;
- access complementary views of the model, including its textual description.

That's why this book won't start with terminal applications or examples implemented in `C++`. These artifacts are important, but they belong to the developer's field. For the user, the most natural path is the graphical interface and the already saved models.

33.4.1 Why GUI comes before language

Another important decision is the order between GUI and language. The *GenESyS* language, presented to the user through the *Simulang* tab, will only make sense after the reader has already developed a concrete notion of the model in the graphical interface. If the language text is introduced too early, it tends to be perceived as an opaque formalism. On the other hand, after the user has seen the model in the GUI, run its simulation and observed its elements, the textual description starts to function as a second way of reading the same system.

This order is pedagogically superior:

1. first, the user sees and manipulates the model;
2. he then notes how this model can also be described verbatim.

Thus, language enters as a deepening and not as an initial obstacle.

33.5 Example Models as a Gateway

Initial contact with a simulator is rarely more productive when starting from scratch. For most users, the best way to get into the system is to open an existing model, look at how it is organized, and use that case as a basis for exploration. *GenESyS* follows exactly this logic.

Here, example models play a didactic and operational role at the same time. They are didactic because they help the user to see concrete cases of using the tool. And they are operational because they can already be opened, executed, inspected and modified within the GUI.

By starting with a saved model, the user does not have to face all the initial difficulties simultaneously:

- learn the interface;
- choose components;
- set parameters;
- organize the flow;
- check whether the model is coherent.

Instead, he starts working with a model that already exists and can focus his attention on understanding the simulator. This will be the strategy adopted in the next chapters.

33.6 What the Reader Will Find in This Manual

This book is organized into two clearly distinct parts.

The first part is the *User Manual*. In it, the reader will learn how to:

- get the project;
- prepare the environment;
- compile the application;
- open the GUI;
- work with example models;
- run simulations;
- observe model results and representations.

The second part is the *Developer Manual*. In it, the focus shifts from simulator operation to internal structure. The reader will study:

- the organization of the source code;
- the simulator kernel;

- components and *data definitions*;
- the *plugins* system;
- the parser and internal language;
- auxiliary applications, tools and tests.

Table 33.2: Overall structure of the GenESyS manual.

Book part	**Main purpose**
User Manual	Teach how to use GenESyS as a modeling and simulation application, with an emphasis on the GUI and saved models
Developer Manual	Teach how to understand, extend and improve GenESyS as a software project

This division should not be seen as mere editorial convenience. It corresponds to two real ways of working with the system. First, the reader needs to be able to use the simulator. Then, if you wish, you can understand and modify its implementation.

33.7 How to Read This Book in the Most Beneficial Way

There are at least two main reader profiles for this manual.

The first is the reader who just wants to use *GenESyS* as a simulator. In this case, the natural reading is linear throughout the first part of the book, with special attention to the chapters on installation, GUI, example models, execution and observation of results.

The second is the reader who, in addition to using the system, intends to study it and improve it as a developer. For this profile, the best strategy is to first also go through the user part. This prevents the internal architecture from being studied too abstractly. A developer who has already seen the GUI in operation and opened real models then better understands the role of the kernel, the parser and the *plugins*.

In other words, the first part of the book is not just preparatory for the user; it also prepares the future developer to read the system from the correct point of view.

33.8 Final Considerations

This chapter introduced *GenESyS* as a modeling and simulation tool, emphasizing the user perspective. The most important point to remember is that the system must initially be understood as a work environment and not as an object of architectural analysis. For the user, the natural path begins with the graphical interface, saved models and the execution of concrete cases. Only then does it make sense to delve deeper into the textual language of the model and, further, into the internal structure of the software.

This order does not reduce *GenESyS*'s wealth; on the contrary, it makes it more accessible. By starting with use, the reader builds a concrete basis for everything that comes later. With this established, the next natural step is to prepare the working environment, that is, obtain the project, install the dependencies, compile the application and achieve the first functional execution of the GUI.

Test your understanding of this content by answering the following questions:

1. Why does this book start by treating *GenESyS* as a tool before treating it as a software project?
2. What is the advantage of adopting the GUI as the main entry point for the user?
3. Why should the *Simulang* tab and template language be presented only after the graphical interface?
4. Why are example models the best gateway for the first practical contact with the simulator?

34. Download, installation and compilation

34.1 Introduction

After understanding the role of *GenESyS* and the way this manual is organized, the reader needs to put it to work. That is the objective of this chapter. Instead of treating compilation as a central theme of the book, it will be treated here as what it really is for the user: a necessary step to achieve graphical application and start working with models.

It is important to note that *GenESyS* is a broader software project than a single application. There are kernel, parser, *plugins*, tests, auxiliary applications and other internal elements. However, this complexity does not need to be absorbed by the user right from the start. The practical goal of this step is simple: obtain the project, prepare the environment, compile the GUI and validate the first execution of the graphical application. If this is achieved safely, the reader will already have what is necessary to advance to the phase of effective use of the simulator.

34.2 Getting the project

GenESyS is available as a source code repository. Therefore, the first step is to obtain a local copy of the project. In an environment with `git` properly installed, this procedure is straightforward.

```
1 git clone https://github.com/rlcancian/Genesys-Simulator.git
2 cd Genesys-Simulator
```

Listing 34.1: Initial Clone of the GenESyS Repository

This command produces a local copy of the repository. From then on, the user will have on their machine the set of files necessary to prepare the work environment. It is advisable, at this point, to carry out an initial inspection of the root of the project, just to recognize its presence and structural volume.

```
pwd
ls
```

Listing 34.2: Initial Inspection of the Repository Root

The user does not yet need to interpret in depth everything that appears in this list. Just recognize that there is a source tree, there is a templates folder, and there are configuration files that support the build and the rest of the project.

34.3 What the user really needs at this stage

A frequent difficulty in academic and research projects is assuming that the user needs to understand the entire internal organization before being able to use them. For *GenESyS*, this strategy would be bad. The user of this first part of the book does not need to immediately study the kernel structure, *plugins* or the parser. He only needs enough to:

- prepare a coherent environment;
- compile the graphical application;
- start the GUI;
- confirm that the interface is functional.

Therefore, the organization of the repository should be read, at this point, with only a practical concern. The `source/` folder contains application and system code. The `models/` folder will be important in the next chapters, as it is where the example models used by the user will come from. Other elements of the project may remain in the background for now.

34.4 Environment dependencies

The current version of *GenESyS* uses `CMake` as the main configuration and build system. This means that the user's build environment must offer a minimum set of tools that are compatible with that organization. In addition to `CMake` itself, the project depends on a modern `C++` compiler and a working installation of Qt for the graphical application.

In the documented baseline, the build uses `CMake` 3.24 or newer, the `Ninja` generator, the `C++23` language standard and the Qt6 development packages.

From a practical point of view, it is worth thinking about the environment in three layers:

1. build tools;
2. graphics and support libraries;
3. compiler and general development utilities.

In the first layer, the central element is `CMake`. In the second, the most important point is Qt, especially the modules associated with the graphical interface. The third includes the `C++` compiler and the usual development tools for the chosen platform.

In the current build baseline, the GUI is expected to be compiled against `Qt6`. For the user, this means that the environment needs to make the Qt6 development packages available in a coherent way, together with the build system and compiler required by the project.

34.4.1 Minimum environment checklist

Before starting to configure the build, the user must confirm that they have, at least:

- `git`;
- `cmake`;
- `Ninja`;
- the modern `C++` compiler;
- a working installation of `Qt6`;
- basic terminal and file system tools.

This checklist does not replace specific instructions for each platform, but it prevents the compilation process from starting in a clearly incomplete environment.

34.5 Compiling the graphical application

GUI compilation must be conducted in a build directory separate from the source tree. This practice is recommended for two reasons. The first is to keep the repository clean, without mixing generated files with the original project files. The second is to make it easier to repeat the process in case of adjustments, cleaning or reconfiguration of the environment.

With the build tree kept under the repository root, the user can configure the project. Since the main path of this first part of the book is the GUI, the configuration must explicitly enable the graphical application. A typical initial configuration is:

```
cmake --preset gui-app
```

Listing 34.3: Initial GUI Build Configuration

This preset configures the `build/gui-app` tree, enables the graphical application and selects the current `CMake/Ninja` baseline expected by the project. Depending on the platform and local environment, the user may want to adjust other options. However, as an initial guideline in the manual, the essential point is to ensure that the GUI is included in the build process. After that, the compilation itself can be carried out.

```
cmake --build --preset gui-app --parallel "$(nproc)"
```

Listing 34.4: GUI Build Compilation

The build preset already drives the aggregate GUI target `genesys_gui`, which depends on the application target `genesys_gui_application`. That application target produces the runtime executable named `genesys-gui`.

34.5.1 Why is the GUI explicitly enabled

It is important to understand the logic of this configuration. The project contains more than one possible application, but the user manual will not be guided by all of them. The target of this first half of the book is the graphical interface. Therefore, the GUI is explicitly enabled in the build configuration. This

decision reduces ambiguities and aligns the technical process with the actual way the user will work with the system.

34.6 How to interpret configuration and compilation

At first glance, the textual output of `CMake` and the compilation process may appear excessively detailed. However, the user does not need to interpret everything. There are just a few essential signs to look out for.

In configuration:

- Qt must be localized correctly;
- the process must not end with a fatal error;
- the build directory should be generated normally.

In compilation:

- the process must complete the generation of the GUI executable;
- linking failures or missing libraries should not appear;
- the target of the graphical application must be produced at the end.

This selective reading of build output is important. The user does not yet need to analyze the entire internal chain of project targets. He only needs to confirm that the path to the graphical application was successful.

34.7 What to do in case of an error

Compilation problems are part of working with software, especially when the project is evolving. This does not mean, however, that the user needs to immediately enter into architectural debugging of the system. The most useful approach is to start with the simple causes.

In case of error, first check:

- whether Qt was installed correctly;
- whether `cmake` is available and updated;
- whether the `C++` compiler is correctly configured;
- if the build directory was created clean;
- that there are no residues of previous incompatible configurations.

A particularly useful practice is to remove the `build` directory and repeat the process in a clean environment when inconsistencies that are difficult to interpret appear. In many cases, this eliminates configuration waste or previous poor choices.

34.8 First run of the GUI

Once the compilation is complete, the next objective is to start the graphical application. The generated executable corresponds to the `genesys_gui_application` target, and its runtime name is `genesys-gui`. From the preset build directory, it is available at `build/gui-app/source/applications/gui/genesys/genesys-gui`.

```
./build/gui-app/source/applications/gui/genesys/genesys-gui
```

Listing 34.5: Generic Example of the First GUI Run

The exact path may vary, but the intention is always the same: to reach the main application window and confirm that the GUI is functional.

34.8.1 What to validate on the first run

When starting the application for the first time, the user must check some very specific points:

- the main window opens correctly;
- menus and visible areas of the interface appear coherently;
- there is no immediate structural failure when application begins;
- the GUI appears ready to open or create templates.

There is no need to open models or run simulations yet. The goal of this step is more modest, but fundamental: to confirm that the prepared environment actually leads to a working graphical application.

34.9 First useful observations about the interface

Once the GUI is open, the user can read the application very briefly, without trying to use it in depth. It is worth noting:

- the presence of a menu bar;
- the central work area;
- the existence of panels, trees or side flaps;
- the availability of commands linked to model, editing and simulation.

This observation does not replace the following chapter, which will deal specifically with the graphical interface. However, it helps to transform the first execution into a more concrete experience than simply “the program opened”.

34.10 Preparing the next step

If the reader managed to get this far, then they already have the most important element of this initial phase: a functional installation of *GenESyS* ready for use. This is enough to move forward. It is not necessary, at this point, to delve into the details of the code structure or explore all possible build options. From here, the manual moves from the technical problem of preparing the environment to the practical problem of using the simulator.

In other words, the reader is now in a position to start working with the graphical application itself.

34.11 Final Considerations

The purpose of this chapter was to take the reader from obtaining the project to the first functional execution of the *GenESyS* GUI. Although this step involves downloading, dependencies, configuration, and compiling, its role within the book remains instrumental. The objective was not to make the build the center of the manual, but to remove the technical barrier necessary for the use of the simulator to actually begin.

With the graphical application in operation, the next natural step is to better understand the main window and its organizational logic. Therefore, the following chapter starts to deal with the graphical interface as a user’s working environment, showing how the GUI organizes menus, editing areas, simulation commands, navigation panels and different views of the model.

Test your understanding of this content by answering the following questions:

1. Why is GUI compilation treated in this manual as a means to achieve simulator use, and not as the book's organizing axis?
2. What minimum dependencies does the user environment need to provide before configuring the build?
3. Why is it recommended to build the project in a separate build directory?
4. What signs show that the first run of the graphics application was successful?

35. First steps in the graphical interface

35.1 Introduction

Once the *GenESyS* graphical application is running, the user actually enters the simulator. It is from that moment on that the system stops being just a compiled project and becomes a work environment. This chapter has precisely this function: transforming the first execution of the GUI into a guided first contact with the interface.

The proposal here is not yet to model in detail or run complete examples. Before that, the reader needs to recognize the logic of the main window, understand which areas of the interface are most important, understand where the essential commands are and build a sufficiently clear notion of the workspace that *GenESyS* offers. In other words, this chapter teaches the user how to locate themselves within the application.

This step is more important than it may seem. A rich interface, with multiple menus, tabs, trees, panels and simulation commands, can generate the impression of excessive complexity when the user tries to understand it at once. The best strategy is different: first recognize the global structure of the application, then identify its functional groups and, only then, start working with concrete models. It is exactly this route that will be followed here.

35.2 The Main Window as a Workspace

GenESyS's graphical interface is organized around a main window that centralizes modeling, inspection, simulation, debugging, and reporting operations. This means that the user is not faced with an application with a single panel or a minimal set of isolated commands. Instead, the GUI was designed as a complete work environment in which different tasks can be performed through different views of the same model.

From a usage point of view, this main window should be understood as the space where five fundamental dimensions of working with the simulator are articulated:

1. model management;
2. graphic editing;
3. parameterization and inspection;
4. running the simulation;
5. observation of results and system behavior.

This organization is especially important because the user does not work with the model just before execution or just after it. The model is continually built, adjusted, inspected, and reevaluated. The GUI was structured to support exactly this cycle.

35.3 Opening the Application and First Visual Reading

When starting the graphical application, the user must resist the temptation to immediately click on all available menus and commands. The best way is a guided visual reading of the main window. This reading can be done in very simple steps.

First, look at the menu bar and notice that it organizes work into functional categories. In general, model actions, editing, visualization, simulation, tools and auxiliary information are present. Second, notice the central area of the interface, which is the main graphical representation space of the model. Third, note the existence of tabs, side or bottom panels and navigation trees, which complement the central view. Fourth, note that the application was not designed just to edit a diagram, but to monitor the entire life of the model, including its simulation and inspection.

This first reading is enough to draw an important conclusion: the *GenESyS* GUI is not a simple graphical editor. It is an integrated work interface.

35.4 Interface Functional Groups

The most productive way to understand the GUI is to divide it into functional groups. Instead of memorizing isolated commands, the user should ask: what kind of work does each part of the interface organize?

35.4.1 Template Commands

The first functional group is model commands. They are what allow you to start a new job, open an already saved model, save the current model, close it, check its consistency and obtain information about it. These commands structure the most basic cycle of using the tool.

In practice, this means that the user must recognize at the outset where the actions responsible for:

- create a new model;
- open an existing model;
- save work;
- close the current model;
- check the model before simulating.

These commands will be used repeatedly throughout the first part of the book.

35.4.2 Editing Commands

The second group is editing commands. This includes actions such as undo, redo, copy, paste, delete, group, ungroup and, in some cases, organization and alignment commands. This set reveals that the interface was designed for effective editing of the model, and not just for passive observation.

Even if the user is not yet building new models, it is important to realize right away that the GUI supports modification of graphic and structural elements. This will be useful when the example models start to change, even slightly.

35.4.3 View Commands

Another important group is view commands. They include adjustments such as zoom, grid, guides and other features that facilitate the visual organization of the graphic space. On first contact, the user does not need to explore all these resources in detail, but must recognize them as tools to support reading and editing the model.

In particular, zooming is often useful right from the start, because different models may require different observation scales.

35.4.4 Simulation Commands

A central group of the GUI is made up of commands linked to running the simulation. These typically include starting, pausing, resuming, stopping, step-by-step, and configuring the simulation. These commands show that the application not only organizes the model as a structure, but also monitors its execution.

In this chapter, the reader must only recognize the existence and location of these commands. Its effective use will be deepened when we start working with concrete models. Still, it's important to stick to the idea that the same GUI used to open and edit the model will also be used to conduct your simulation.

35.4.5 Inspection and Support Commands

In addition to the previous groups, the interface also offers inspection and support mechanisms: property panels, component trees and *data definitions*, debug areas, report tabs and other auxiliary views. The presence of these resources reinforces an essential point: *GenESyS* was not designed just to design models, but to study them in execution.

Table 35.1: Main Functional Groups of the GenESyS GUI.

Functional Group	**Role in the User's Work**
Template	Create, open, save, close, and check models
Editing	Change and rearrange model elements
View	Adjust scale, guides, and graphic layout
Simulation	Start, pause, resume, stop, and monitor execution
Inspection and support	Inspect properties, elements, results, and helper views

35.5 The Graphics Area of the Model

The graphics area is the visual center of working with *GenESyS*. It is where the model appears in a diagrammatic form and it is where most of the user's attention tends to focus when reading and editing the models. At first glance, this area should be understood as the main space where the model's flow becomes visible.

This is important because the user needs to start reading the simulator by the way it shows the model, and not just by the way the program organizes its menus. The graphic area allows you to view:

- the components present in the model;
- the connections between these components;
- the general logic of the modeled flow;
- the visual distribution of elements within the workspace.

Even before opening a concrete model, the user should already perceive this area as the core of the simulator's visual experience.

35.5.1 Graphic area is not just drawing

An important point is that the graphics area should not be confused with a simple drawing editor. What is organized there are not just geometric shapes, but the representation of the model that will later be simulated. In other words, graphic space is also semantic space. Each visible element there tends to correspond to a relevant modeling entity.

This observation helps the user to look at the interface in the correct way. Instead of thinking that he is just manipulating visual objects, he must understand that he is interacting with the operational representation of the model.

35.6 Trees, Panels, and Helper Editors

In addition to the central graphics area, the *GenESyS* GUI offers other elements that are equally important for everyday work. These elements complement the visual reading of the model and make it possible to inspect it in a richer way.

35.6.1 Navigation Trees

Navigation trees help the user locate system and model elements. Depending on the context, they may show:

- *plugins*;
- model components;
- *data definitions*;
- other auxiliary structures associated with the current work.

At first glance, these trees should be seen as guidance mechanisms. They help answer the question: What elements are available or present in the model I'm working with?

35.6.2 Property Editor

The property editor is one of the most important tools in the GUI. It is through this that the user stops just observing the existence of an element and starts dealing with its attributes, parameters and configuration options. In other words, this is where the template starts to become effectively editable.

In practice, this means that the selection of an element in the graphics area or in a navigation tree

tends to be associated with the display of corresponding properties. The next chapters, this mechanism will appear continuously, because almost every relevant modification of a model goes through some type of parameterization.

35.6.3 Complementary tabs and views

The presence of central tabs and panels organized by tabs shows that the GUI was built to support multiple perspectives on the same model. Some of these perspectives are visual, others are textual, others are focused on execution, debugging, or presenting results. The user does not need to master all of this now, but must retain the idea that the model is not observed only through a single visual surface.

This multiplicity of views will be especially important when, later on, we introduce the *Simulang* tab as a complementary textual representation to the graphical model.

35.7 Minimal User Guidance Flow in the GUI

In order to familiarize yourself with the GUI to be productive, it is advisable to adopt a short guideline. Instead of randomly exploring the main window, the user can follow this sequence:

1. find template commands;
2. identify the central graphics area;
3. locate auxiliary trees and panels;
4. find the property editor;
5. recognize basic simulation commands;
6. note the existence of additional tabs.

This script has an important advantage: it prevents the richness of the interface from being perceived as clutter. By following a simple order, the user transforms the main window into something intelligible and progressively explorable.

35.8 What Is Not Necessary to Master Now

A successful first contact with the GUI does not require mastering the entire application. It is not necessary, for example:

- use all editing commands;
- understand all debug panels;
- interpret all interface trees;
- know in depth all types of available elements;
- also explore all of the model's languages and representations.

On the contrary, trying to master everything immediately tends to produce noise. The most efficient learning occurs in layers. First, the user understands the general structure of the interface. Then, he starts working with concrete models. Then, it begins to report the GUI with execution, results and textual language. This is the route adopted in this manual.

35.9 Good Practices When First Encountering the Interface

There are some simple practices that make the initial experience with the GUI much more beneficial.

The first is to always work with a concrete objective. Instead of "exploring the interface", try to

locate something specific: the open model command, the graphics area, the properties panel or the simulation commands.

The second is not to confuse quantity of resources with difficulty of use. The *GenESyS* interface is broad because it organizes several phases of working with models. This does not mean that all resources need to be used simultaneously.

The third is to look at the interface as an integrated environment. Menus, graphics area, trees, properties, simulation and reports are not independent parts. They form a single workflow.

The fourth is to accept that some parts of the GUI will only make sense when a model is actually loaded. That's why the next chapter will go directly into the example models.

35.10 Final Considerations

This chapter introduced the *GenESyS* graphical interface as the user's main working environment. Instead of treating the GUI as a dispersed set of commands, the text sought to show its organizational logic: the main window brings together resources to manage models, edit them, parameterize them, simulate them and observe them from different perspectives. This understanding is essential because it prevents the user from reducing the application to a simple graphical editor or, at the opposite extreme, perceiving it as an excessively complex and cluttered interface.

With this guidance in place, the next natural step is to start working on concrete models. This is when the graphical interface really takes on operational meaning. Therefore, the following chapter will deal with the example models already available in the project and their initial use in the GUI, allowing the reader to open, inspect, execute and modify real simulation cases.

Test your understanding of this content by answering the following questions:

1. Why should the *GenESyS* GUI be understood as an integrated work environment, and not just as a graphical editor?
2. What are the most important functional groups of the interface in a first contact?
3. What is the function of the model's graphical area within the simulator usage experience?
4. Why isn't it necessary to immediately master all the application's menus, trees and panels?

36. Example models and initial use of the simulator

36.1 Introduction

Once the *GenESyS* graphical interface is up and running and the reader has an initial view of your organization, the next natural step is to start working with concrete models. This is the point at which the simulator stops being just an application open on the screen and starts to become an effective modeling and experimentation environment. To achieve this, the most appropriate path is not to start immediately by creating a complete model from scratch, but to use the example models already available in the project.

This decision is important from both a didactic and practical point of view. From a didactic point of view, an example model already offers a coherent structure, with interconnected elements, completed properties and expected behavior. This reduces the initial cognitive load on the user, who does not need to simultaneously learn the interface, the semantics of the components, the flow construction logic and the simulation execution mechanism. From a practical point of view, example models allow you to check early on whether the system installation is functional and whether the GUI can open, display, interpret and execute real models.

In this chapter, the objective is to teach the reader how to use the models saved in the `models/` folder as a gateway to the simulator. At the end of reading, the user should know how to locate an example model, load it into the graphical interface, read its visual structure, identify the main flow of the modeled system, run the simulation and make small controlled modifications to observe the effect of these changes.

36.2 Why start with example templates

Rich simulations tend to offer many job possibilities from the first contact. This wealth, although valuable, can become an obstacle when a good entry strategy is not chosen. In the case of *GenESyS*, using example templates solves exactly this problem.

When opening an already saved model, the reader has immediate access to an artifact that:

- is already organized coherently;
- already contains a flow logic or modeling structure;
- can now be inspected by the GUI;
- can now be executed;
- can now be used as a basis for tracked changes.

This allows learning to occur in layers. First, the user learns to recognize the model as a visual object within the interface. Then learn how to execute it. Then, begin relating the graphic elements to their effects in the simulation. Only later did it begin to produce its own models with greater autonomy.

In pedagogical terms, the example model fulfills a role similar to that of an experiment already set up in a laboratory: it does not replace active learning, but creates a stable initial context in which the user can begin to observe, formulate hypotheses, modify parameters and interpret results.

36.2.1 The mistake of starting from scratch too soon

There is a natural temptation, especially in technically curious readers, to try to build their own model during their first interactions with the system. Although this initiative may seem productive, it often introduces unnecessary difficulties.

When the user starts too early from an empty space, he needs to decide at the same time:

- which elements to insert;
- in what order to connect them;
- which properties to adjust;
- what supporting data is needed;
- how to check if the model is coherent;
- what to expect from execution.

This accumulation of decisions makes it more difficult to distinguish what is a difficulty inherent to the modeling and what is simply an initial lack of knowledge of the tool. The example model avoids this noise. It separates the problem of using the interface from the problem of full model authoring.

36.2.2 Example templates as reading material

In addition to serving as executable cases, the example templates should also be read. This reading, however, is not the same as reading a text or a source code. It involves:

- observe the visual arrangement of the model;
- perceive the main flow sequence;
- identify entry and exit points;
- recognize auxiliary elements that support execution;
- report visual structure to observed behavior.

With this, the model stops being a file and becomes an object of study within the GUI.

36.3 The `models/` folder as a reference for use

For the *GenESyS* user, the `models/` folder should be understood as one of the most important areas of the project. This is where the saved models that will be used as an entry point for exploring the simulator are located. In this first part of the book, this folder has much higher priority than any source code directory.

The role of this folder is twofold. First, it concentrates concrete material for using the simulator. Second, it provides the user with a repertoire of examples from which to learn how to open, read, run, and change models. This means that for initial learning purposes, the user does not need to worry about software implementation details. Your main universe of work is precisely in these model files.

36.3.1 What to look for in the `models/` folder

When browsing the `models/` folder, the user must look for models that preferably satisfy three criteria:

1. are visually simple or moderately simple;
2. have an easily identifiable main flow;
3. allow an execution whose behavior can be observed without great interpretative effort.

This doesn't mean that larger or fancier models aren't valuable. It just means that, when entering the simulator, it is advisable to prioritize cases that favor clarity and readability.

36.3.2 How to select good first examples

As a first approximation, the best examples tend to be those where the user can answer the following questions relatively quickly:

- where does the flow begin?
- what main elements does it go through?
- where does it end?
- Are there queues, resources, attributes, or auxiliary data that support this behavior?
- what do I expect to see when this model is run?

If these questions can be answered without great difficulty, the model is probably a good candidate for initial exploration.

36.4 Opening a sample model in the GUI

With the application already started, the next step is to open a real model. The operation of opening a model file should not be treated as a simple technical detail, because it inaugurates the phase in which the GUI starts to operate effectively on a concrete work object.

The general procedure is as follows:

1. start the *GenESyS* GUI;
2. find the menu or command corresponding to opening the model;
3. navigate to the `models/` folder;
4. select an appropriate template file;
5. confirm the opening and observe the interface update.

After that, the central graphics area should start displaying the model, and the GUI panels will tend to reflect the loaded elements, allowing for more detailed inspection.

36.4.1 What to confirm immediately after opening

Once the template is loaded, the user must confirm a few simple but important points:

- the graphics area has been filled with the model structure;
- there are visible elements and connections;
- the interface seems to recognize the model as open;
- inspection and simulation commands became usable;
- the user can visually navigate the loaded content.

This confirmation is useful because it distinguishes two very different states of the application:

- the GUI is empty or without an active model;
- the GUI effectively working on a model.

Only in the second case does it make sense to proceed to reading and execution.

36.4.2 When the model appears “too big”

In some cases, the first model opened may appear visually too dense. If this occurs, the user should not immediately interpret this sensation as a problem with the simulator or model. In general, this just indicates that:

- zoom needs to be adjusted;
- the chosen model was not the most suitable for the first exploration;
- reading must start with a main region, and not with the entire scene.

In this situation, the best practice is to return to a simpler example or restrict the analysis to the most easily identifiable main flow.

36.5 How to read a model in the graphics area

Once the model is loaded, the user’s next task is to read it. This reading is neither purely visual nor purely technical. It is a reading guided by questions about behavior and structure.

The best initial reading order is:

1. find the beginning of the flow;
2. follow the main path of the model;
3. identify the end of the flow;
4. observe support elements;
5. only then examine local properties and details.

This order is important because it prevents the user from getting lost in details before understanding the central logic of the model.

36.5.1 Identifying the main flow

The main flow is the most important structure of the model for first contact. It corresponds to the essential path taken by the main entities, events or transformations represented in the system. In many cases, this flow can be recognized by a sequence of components connected in a relatively linear way or by a visual arrangement that clearly suggests the progression of the modeled process.

The question that should guide the user is:

What is the minimal story this model is telling?

When this question begins to be answered, the model stops being a set of elements on the screen and starts to be perceived as an organized system.

36.5.2 Distinguishing main flow from auxiliary elements

A careful reading of the model depends on distinguishing what constitutes its central path from what supports it. Often, in addition to the main flow, the model also involves auxiliary data and structures: queues, resources, statistics, attributes, variables, counters or other definitions.

The user should not ignore these elements, but he should not start with them either. The correct procedure is:

1. understand the flow first;
2. then ask what support elements allow this flow to work;
3. only then observe specific properties.

This order greatly improves the readability of the model.

36.6 Running the model for the first time

After opening and reading the model, the decisive moment comes: running the simulation. It is at this stage that the user checks whether the observed visual structure actually corresponds to understandable dynamic behavior.

On first use, execution should be conducted with exploratory intent and not in a rush. The objective is not just to "run" the model, but to understand:

- if the simulation starts correctly;
- whether the model's behavior makes sense in relation to its structure;
- whether the GUI provides sufficient signals to track progress;
- whether the model reaches a final state or evolves coherently.

36.6.1 What to watch out for while running

When running a model for the first time, the user should avoid trying to read all output and all panels at the same time. It's best to focus on a few core aspects:

- there is a clear indication that the simulation has started;
- the model appears to visually evolve or produce coherent signs of progress;
- the observed behavior is compatible with the previously identified main flow;
- there is no obvious structural flaw in the execution process.

In other words, the first run serves to connect the static model to its dynamics.

36.6.2 First validation of behavior

A well-done initial validation is modest but sufficient. The user must be able to respond:

- did the model open correctly?
- Could the simulation be started?
- does the observed behavior seem plausible?
- Did the execution finish or progress without obvious inconsistencies?

If these answers are positive, then the first practical contact with the simulator has already been successful.

36.7 Modifying an example model in a controlled way

After opening and running a model, the next most productive step is to make small, controlled modifications. This step is pedagogically very important, because it transforms the example model into an active learning instrument.

The best initial modifications are those that:

- do not completely change the structure of the model;
- can be easily reversed;
- produce an observable effect;
- help the user understand the function of properties or parameters.

Examples of appropriate changes at this stage include:

- adjust a simple property;
- change a name or label;
- change a numerical value associated with a component;
- save a modified copy of the template.

36.7.1 Why small changes are better

If the first modification is too big, the user loses the ability to report cause and effect. It is difficult to know whether the observed change in behavior is due to a specific parameter, an extensive structural change or an inconsistency introduced accidentally.

Small changes, on the contrary, allow the user to experience the simulator logic almost like in a laboratory:

1. observe the original state;
2. change a specific point;
3. rerun;
4. compare the effect.

This methodology is much more useful than making extensive modifications at the beginning.

36.8 Best practices for exploring example models

A few simple practices make initial use of the example templates much more beneficial.

36.8.1 Start with smaller cases

Smaller models generally allow for clearer reading of the interface, flow, and behavior. They also facilitate the association between visual structure and observed execution.

36.8.2 Save separate copies

When modifying a sample template, it is recommended that you save a new copy rather than immediately overwriting the original file. This preserves the reference example and makes later comparisons easier.

36.8.3 Use execution as a reading tool

Often, the model only becomes fully intelligible when it is executed. Therefore, execution should not be seen just as a final step, but as part of the process of understanding the example itself.

36.8.4 Return to structure after running

A very useful practice is to re-read the model visually after the first run. Observed behavior often sheds new light on the function of its elements, connections, and supporting data.

Table 36.1: Recommended workflow for exploring example models in GenESyS.

Step	Goal
Open the model	Load a working case in the GUI
Read the main flow	Understand the central logic of the modeled system
Recognize supporting elements	Understand what data and structures support the flow
Run the simulation	Compare visual structure with observed behavior
Change something simple	Turn the example into an active learning object
Rerun and compare	Observe the effect of the modification made

36.9 Final Considerations

In this chapter, example models have been presented as the user's main entry point into practical work with *GenESyS*. Instead of starting with the creation complete of a new model, the reader was instructed to open already saved cases, read them visually, identify their main flows, execute them and make small controlled changes. This strategy allows learning to happen in a progressive and concrete way, without requiring the user, right from the beginning, to fully master the entire tool.

The essential point to remember is that an example model should not be treated as a simple passive demonstration. It is, at the same time, an object of reading, an object of execution and an object of experimentation. From here, the user is not only familiar with the GUI as a structure, but begins to use it effectively on real models. The next natural step is to deepen the relationship between graphical interface, execution, behavior observation and textual representation of the model, which will be done in the next chapter by introducing the *Simulang* tab, the execution observation mechanisms and the first procedures for analyzing the system's behavior.

Test your understanding of this content by answering the following questions:

1. Why are example templates a better entry point than immediately creating a complete template from scratch?
2. What is the most productive order to read a model loaded in the GUI?
3. What should the user validate when first running a sample model?
4. Why is it important to initially modify only small aspects of the model?

37. GUI, Simulang, model execution and observation

37.1 Introduction

In the previous chapters, the reader got *GenESyS* up and running, got to know the general logic of the graphical interface and started working with example models already saved in the project. From this point forward, it becomes possible to take an important step: move away from just a structural reading of the models and start observing them in execution, following their dynamics and relating the graphical representation to the textual representation of the system.

This chapter has exactly that purpose. It delves deeper into GUI usage in three complementary directions. First, it shows how the execution of the model should be followed by the user, not as an opaque operation, but as an observable process. Second, it introduces the *Simulang* tab as a textual form of representation of the same model that has already been seen in the graphical interface. Third, it guides the observation of the first results, traces and signs of the system's behavior, so that the user begins to build a more informed reading of the simulation.

The importance of this stage is great. A really useful simulator is not only able to open models and run them, but to allow the user to understand what is happening. The *GenESyS* GUI was designed to offer different perspectives of the same work: a graphical perspective, a textual perspective and a dynamic perspective, linked to execution. From here, the reader will begin to articulate these three layers.

37.2 From static structure to model dynamics

When the user opens a model and looks at its graphical structure, he sees a static description of the system. The components, connections, supporting data and graphic elements indicate how the simulation was organized, but do not yet show, by themselves, the effective temporal behavior of the model. Running the simulation transforms this static structure into a dynamic process.

This point is central to understanding the simulator. A model is not just a well-organized diagram; he is a behavior machine. The function of the simulator is precisely to activate this machine, driving the advancement of simulated time, the processing of events, the circulation of entities and the updating of the system's relevant information.

For the user, the main consequence is this: execution should not be seen as just "pressing the start button". It should be understood as the passage from the model description to its operational dynamics.

37.2.1 What changes when the model starts running

When the simulation starts, the user starts working with a new type of reading. Before, the main question was:

How is this model organized?

During execution, this question changes to:

How does this model behave over time?

This means observing:

- whether entities are being created, transformed or removed;
- if the flow actually goes through the elements that the model suggests;
- whether resources, queues or supporting data appear to be used coherently;
- if the global behavior of the simulation makes sense in relation to the previously read structure.

37.3 Execution commands and their usage logic

The *GenESyS* GUI provides a set of commands directly linked to the simulation. Among them, the commands to start, pause, resume, step by step, stop and configure stand out. For the user, these commands should not be perceived just as operational buttons, but as different ways of relating to the dynamics of the model.

37.3.1 Start the simulation

The simulation start command is the transition point between the static model and observed behavior. Before activating it, the user must ensure that:

- the correct model is loaded;
- the visual structure makes sense;
- the system appears to be ready to run;
- the objective of the observation is minimally clear.

This last observation is important. It is better to run the simulation with some reading hypothesis, for example observing a specific flow or the effect of a certain property, than simply starting the execution without knowing what you want to track.

37.3.2 Pause, resume and advance step by step

One of the great advantages of a rich GUI is that it allows different rates of observation of the simulation. In some situations, the user just wants to check the overall behavior. Other times, you want to look at specific parts of the stream more closely. It is in this context that commands such as pause, resume and step by step become particularly valuable.

These commands transform the simulation into an inspection object. Instead of just letting the system run to completion, the user can:

- temporarily stop execution;
- analyze the current state;
- resume simulation;
- or advance in a controlled manner.

This mode of use is especially useful in learning models, in initial debugging and when reading the behavior of elements whose function is not yet completely clear to the user.

37.3.3 Stop and configure

The stop and configuration commands also deserve attention. Stopping the simulation is not just ending a process, but stopping the evolution of the model in a controlled way. Configuring the simulation means adjusting execution conditions before starting the experiment.

Although the depth of these parameters depends on the model and the context, the user must remember that the execution is not completely rigid: it can be prepared, started, observed and stopped according to different needs.

37.4 Observing model behavior during execution

Observation of execution must be conducted in a guided manner. If the user tries to simultaneously follow all the GUI signals, the entire reading will become fuzzy. The ideal is to observe the model in levels.

37.4.1 First level: the global flow

At the first level, the user must observe the behavior of the model globally. The most important questions are:

- did the simulation start correctly?
- does the main flow seem coherent with the model structure?
- does the system evolve plausibly?
- does termination of execution make sense?

This is a macroscopic reading. She doesn't look for fine details, but overall consistency.

37.4.2 Second level: specific elements

After observing global behavior, the user can focus on specific elements of the model. This may include:

- a component whose function is not yet clear;
- a point in the flow where the behavior seems particularly important;
- a relevant queue, resource or supporting data;
- a property whose effect has recently been modified.

This localized reading is more productive after the general dynamics have already been minimally understood.

37.4.3 Third level: comparison between runs

An especially useful practice is to compare the model run before and after small changes. This procedure helps the user understand not only how the system works, but also how it will react to changes. The GUI, in this sense, is not just an observation interface, but also a comparison laboratory.

37.5 The *Simulang* Tab as a Textual Representation of the Model

After the reader has already seen the model in the GUI and observed it in execution, the textual representation starts to make much more sense. This is exactly why the *Simulang* tab appears now, and not before.

The function of the *Simulang* tab is not to replace the GUI, but to offer a second way of reading the same model. In this way of reading, the user stops seeing only the graphical organization of the system and starts seeing a corresponding textual description. This is valuable for several reasons:

- makes the model more formally explicit;
- allows you to compare the visual representation with the textual representation;
- helps consolidate understanding of the model;
- paves the way for later chapters on language and development.

37.5.1 How to Read the *Simulang* Tab

Reading the *Simulang* tab must be done with the same didactic caution used in the rest of the first part of the book. The user does not need, at this point, to master the entire syntax or semantics of the language. What he needs is to accomplish that:

- the textual description corresponds to the model you have already seen in the GUI;
- visual elements of the model have a textual expression;
- the language offers a more formal view of the modeled system.

The most productive way to start this reading is to compare excerpts from the textual representation with already known elements of the graphical interface. Instead of trying to decipher the entire text at once, the user should ask:

> What part of the graphical model does this region of the text seem to describe?

37.5.2 Why textual representation is useful to the user

Even if the user continues to work primarily through the GUI, the textual representation offers concrete benefits. It helps to:

- consolidate understanding of the model;
- make the system structure more explicit;
- understand the model beyond its visual layout;
- start thinking about the relationship between interface and language.

This experience is particularly valuable because it shows, from an early stage, that *GenESyS* does not treat the model just as a drawing, but also as a formal description.

37.6 Tracing, breakpoints and initial observability

An important feature of a well-structured simulator is to offer mechanisms for observing the behavior of the running system. In *GenESyS*, this is expressed through different means available in the GUI, including

traces, events, reports, and controlled stopping or tracking mechanisms.

At this stage of the manual, the reader does not yet need to deeply master these mechanisms. But you need to understand their general function: they make the simulation more observable.

37.6.1 Tracing as reading behavior

Tracing should be seen as a way of recording what happens in the simulation. It helps the user follow the model not only through the visual perception of the interface, but also through an operational narrative of what is happening.

This is especially useful when the model's behavior is not immediately evident from the graphics area alone. In such cases, tracing answer helps:

- what events are occurring;
- in what order;
- with what apparent effects on the state of the system.

37.6.2 Breakpoints as a controlled observation tool

Breakpoints, when used, allow you to interrupt the simulation at points of interest. For the user, this turns the execution into an examineable object. Instead of letting the model evolve completely without intervention, you can stop at relevant states and take a closer look at what is happening.

This tool is particularly useful when:

- if you want to study the behavior of a specific part of the model;
- if you want to validate the effect of a modification;
- if you want to better understand the dynamics of a specific element.

37.7 First reading of the results

In addition to observing the execution in progress, the user needs to start developing the habit of interpreting the results produced by the simulator. Here again, the guidance must be progressive.

The first reading of results should not be statistically sophisticated or overly ambitious. She must answer simple questions:

- did the simulation behave as expected?
- were there coherent signs of progression and termination?
- Did simple changes to the model produce observable effects?
- Do the reports or signals presented by the GUI seem compatible with the model structure?

Only after this initial stage does it become useful to move on to more technical readings.

37.7.1 Results as continuation of model reading

An important point is that the results should not be treated as something separate from the model. They are a continuation of reading the model by other means. The graphical model shows structure, the execution shows dynamics, and the results show observable consequences of that dynamic. These three dimensions must always be articulated.

37.8 Best practices for observing running models

Some practices make observing the much simulation more beneficial.

37.8.1 Set an observation focus

Before starting execution, try to decide what to observe first: the global flow, a specific component, an effect of a certain change, or the relationship between GUI and Simulang. This prevents dispersion.

37.8.2 Switch between views

The strength of the *GenESyS* GUI is in offering multiple perspectives on the same model. Switch between:

- graphic structure;
- observed execution;
- traces or events;
- textual representation in *Simulang*.

This alternation often produces deeper understanding than any single view.

37.8.3 Make small changes and compare

Whenever possible, make small, controlled changes to the model and compare the behavior before and after. This strategy turns observation into a guided experiment.

37.8.4 Don't try to exhaust everything in a single run

A simulation rarely reveals everything the model can teach in a single run. It is better to read successively, each with a clearer focus, than to try to absorb everything at once.

Table 37.1: Recommended Strategy for Observing Running Models in GenESyS.

Step	**Goal**
Observe the graphical structure	Understand the static organization of the model
Start simulation	Transform structure into dynamic behavior
Monitor global flow	Check overall consistency of execution
Explore tracing and events	Make behavior more observable
Read the *Simulang* tab	Relate the graphic model to its textual description
Compare runs	Understand the effect of small model changes

37.9 Final Considerations

This chapter deepened the use of the *GenESyS* GUI in an essential direction: the articulation between the model's visual structure, simulation execution, behavioral observability and textual representation in *Simulang*. The main idea that the reader should remember is that these dimensions do not compete with each other. They complement each other. The graphical model allows you to see the organization of the system, the simulation allows you to observe its dynamics, the tracing and pausing mechanisms make this behavior more readable, and the *Simulang* tab offers a textual way to consolidate this understanding.

With this, the first part of the manual begins to gain real density of use. The reader no longer just opens the GUI and loads a model; he began to observe the simulator in operation and report its

various representations. The natural next step from here is to consolidate the use of the system with more guided examples and prepare the transition to a more complete understanding of the relationship between modeling, simulation and the internal structure of GenESyS.

Test your understanding of this content by answering the following questions:

1. Why should the execution of a model be understood as a transition from the static structure to the system dynamics?
2. What is the use of commands such as pause, resume and step-by-step during the simulation?
3. Why should the *Simulang* tab only be introduced after the user has already seen and executed the model in the GUI?
4. How do tracing, breakpoints and results complement visual observation of model execution?

38. Source Code Organization and Overall Architecture

38.1 Introduction

From this point on, the book explicitly enters its second half: the developer's manual. Changing perspective is important. So far, the focus has been on using *GenESyS* as a modeling and simulation tool, with an emphasis on the GUI, example models and observation of system behavior. Now, the focus becomes understanding the software itself: its internal organization, its central subsystems, the way the code is distributed and the points from which a developer can begin to study, modify and expand it.

Before studying the core of the simulator or discussing *plugins*, parser and applications, it is necessary to build an overview of the repository organization. Without this vision, the developer tends to get lost in files and classes without realizing the architectural function of each region of the code. The goal of this chapter is precisely to avoid this problem. Instead of immediately presenting local details, it organizes the overall project map.

The central idea is simple. *GenESyS* should not be read as an amorphous set of `C++` files, but as a system divided into areas with distinct responsibilities. Some of these areas underpin the core of the simulation. Others organize the extensibility mechanism. Still others implement applications, tools and tests. Understanding this division is the first step to a careful technical reading of the project.

38.2 From Simulator Application to Software System

In the first half of the book, *GenESyS* appeared to the reader mainly as an application. The user opened the GUI, loaded models, ran simulations, and observed results. For the developer, however, this vision is just the surface of a broader system. The graphic application does not exhaust the project; it is one of its forms of presentation.

This observation is important because it helps to correctly reposition the object of study. The developer does not work just on a graphical interface, nor just on a set of models. He works on software that:

- maintains a simulation kernel;
- represents models internally;
- manages events, entities and simulated time;
- interprets expressions and simulation language;
- loads and integrates *plugins*;
- supports different applications on the same basis;
- provides infrastructure for testing and evolution.

Therefore, the starting point of the second half of the book is not the GUI itself, but the system of which the GUI is a part.

38.3 The Overall Logic of the Directory Tree

The *GenESyS* repository should be read as a tree organized by technical responsibility. Not all directories have the same importance for the developer, and a good initial reading consists precisely in distinguishing what is structural from what is just contextual.

Broadly speaking, there are three main areas of interest:

1. the main source code zone;
2. the zone of usage models and artifacts;
3. the development, build and testing support area.

Within the first of these zones, the `source/` folder contains the effective core of the software. It is the one that starts to dominate the reading from this chapter forward. The `models/` folder, which in the first half of the book served as the main entry point for the user, remains important, but now changes its role: it is no longer just material for use but also becomes material for architectural observation, as it helps the developer to relate the representation of the model to the behavior of the code.

38.3.1 Why the `source/` folder is the center of reading

For the developer, the `source/` folder is the heart of the project. This is where the fundamental subsystems of *GenESyS* are located:

- kernel;
- parser;
- *plugins*;
- applications;
- tools;
- tests.

This division already suggests a first architectural reading of the system. The kernel represents the central infrastructure of the simulation. The parser takes care of the language and expressions. The *plugins* mechanism materializes the extensibility mechanism. The applications expose concrete ways of using the infrastructure. The tools complement the ecosystem. Tests organize the validation of software

behavior.

38.3.2 Why `models/` remains relevant to developers

In the first part of the book, the models were treated as material for using the simulator. Now, they also start to serve as software reading instruments. This is because the developer constantly needs to report two things:

- the way the system presents itself to the user;
- the way the system is implemented internally.

The models in `models/` help exactly with this bridge. They show the type of artifacts that the application manipulates and help to report the structure of the models to the set of kernel classes and mechanisms.

38.4 The large areas of `source/`

Reading `source/` must start with its large areas, and not with isolated files. This approach reduces noise and helps build an architectural view before reading detailed classes.

38.4.1 `source/kernel`

The `source/kernel` folder contains the simulator's central infrastructure. It is there that the developer finds the most fundamental elements of the system: model representation, entities, events, simulation mechanisms, utilities and statistics. This is, without a doubt, the most important region of the project for anyone who wants to understand GenESyS in depth.

From an architectural point of view, the `kernel` folder organizes what the simulator core does independently of any specific interface. Graphical applications, terminal applications, tools and other modules can only exist because this core provides them with a stable base.

38.4.2 `source/parser`

The `source/parser` folder brings together the infrastructure associated with interpreting the simulator's language and expressions. It is especially important because many model parameters are not just literal values, but textual expressions that need to be interpreted within the context of the model and simulation.

In the first part of the book, this dimension appeared indirectly in the *Simulang* tab. Here, it becomes clear that the `source/parser` folder should be understood as a software subsystem responsible for connecting text, model semantics, and execution.

38.4.3 `source/plugins`

The `source/plugins` folder organizes one of the most characteristic aspects of GenESyS: its extensibility. Instead of concentrating all functionality in a closed core, the project foresees the incorporation of specialized modules that expand the components, data and other capabilities of the system.

For the developer, this means that an important part of the simulator's evolution happens on this border between kernel and *plugins*. Many of the modeling elements most visible to the user are provided precisely by this mechanism.

38.4.4 `source/applications`

The `source/applications` folder contains applications built on top of the system core. This is where GenESyS stops appearing just as infrastructure and starts to take on concrete forms of interaction, such as the GUI and other auxiliary applications. This layer is important because it shows a valuable architectural separation: the simulation core is not confused with a single application.

This means that the software can be read at two levels:

- as infrastructure;
- as a set of applications on this infrastructure.

38.4.5 `source/tools`

The `source/tools` folder brings together tools that support the use, development or evolution of the system. Although they are not always the first reading priority, these tools can become important in debugging, support, integration, or maintenance contexts.

38.4.6 `source/tests`

The `source/tests` folder represents the project validation dimension. It is especially important for the developer because it offers a concrete path to verify expected behavior, prevent regressions and provide security for changes to the system.

In evolving projects, the existence of an organized test tree is one indication of disciplined development. In GenESyS, this is particularly relevant because the software has multiple layers and extensibility mechanisms.

38.5 A Layered Reading of the System

A very useful way to understand the general architecture of *GenESyS* is to imagine the system in layers.

At the deepest layer is the kernel. It is what defines the fundamental structures and services of the simulation. On top of this layer, language and extensibility mechanisms appear, such as parser and *plugins*. Further up, concrete applications emerge, such as the GUI, which present the user with a way of interacting with this infrastructure. In parallel, tests and tools help support the maintenance and evolution of the project.

This layered reading helps the developer avoid a common mistake: confusing what is structural with what is just surface use. The GUI is very important, but it is not the heart of the system. Likewise, a plugin may be central to a specific modeling domain, but still depends on the kernel to exist.

Table 38.1: Layered Reading of the Overall GenESyS Architecture.

Layer	**Main Function**
Kernel	Core simulation infrastructure, model representation, events, time and statistics
Parser and extensibility	Expression interpretation, simulator language and extension by *plugins*
Applications	Concrete interaction forms, such as the graphical interface
Tools and Tests	Support for development, validation, maintenance and evolution

38.6 The Role of the `Simulator` Class in the System View

The `Simulator` class provides a particularly useful synthesis of this overview. It presents itself as the highest level class of the simulation kernel and exposes several central system managers. These include `PluginManager`, `ModelManager`, `TraceManager`, `ParserManager`, and `ExperimentManager`.

This observation is valuable because it shows, in a very concrete way, that the GenESyS architecture is not organized around a single file or a single operational class, but around a core that coordinates multiple specialized subsystems. In other words, the developer must understand `Simulator` as an access point to the simulator ecosystem, and not just as an isolated technical object.

38.6.1 Why This Matters for Reading the Code

If the developer enters the project only through the GUI or a specific modeling component, they run the risk of reading the system in a fragmented way. The presence of the `Simulator` class as an aggregator of managers helps to refocus the reading: software is better understood when seen as a set of coordinated subsystems, and not as a collection of independent parts.

This insight will be particularly important in the next chapter, when the focus shifts to the kernel and classes that effectively control the life of the model in simulation.

38.7 Misreadings This Chapter Avoids

This chapter aims to avoid some common misreadings in projects like GenESyS.

The first is to imagine that the GUI is the entire system. It is fundamental for the user, but it is just an application on a broader infrastructure.

The second is to imagine that the kernel can be read without relation to the parser, *plugins* and applications. Although the kernel is the foundation, it does not live in isolation.

The third is to start studying the code with random files. This practice often produces local familiarity but not architectural understanding.

The fourth is to treat the `models/` folder as something irrelevant to the developer. In fact, it is important because it helps to relate the implementation to the simulator's real work object.

38.8 Recommended Reading Strategy for the Code

The most productive way to study the *GenESyS* source code is progressive.

First, build an overview of the project tree and the responsibilities of its major areas. Then, enter the kernel and study the central classes that articulate the model, simulation, events and observability. Then move on to the component base classes, *data definitions*, parser, and *plugins*. Only then does it make sense to delve deeper into specific applications, tools and tests.

This order is not arbitrary. It follows the software's own logic:

1. first, the infrastructure;
2. then, the expansion mechanisms;
3. then, the concrete applications;
4. finally, the maintenance and validation instruments.

38.9 Final Considerations

This chapter reorganized the reader's perspective on *GenESyS*. From this point onward, the project is no longer seen just as a graphical application and instead becomes a software system distributed across areas with different responsibilities. The most important point to remember is that the GenESyS architecture must be read in layers: kernel, parser, *plugins*, applications, tools, and tests. Without this overall view, the study of individual classes tends to become fragmented and unproductive.

With this base established, the next step is to enter the most important region of the project: the core of the simulator. It is here that the model, the simulation, the events, the simulated time, the list of future events and the central mechanisms for observing the system's behavior are defined. Therefore, the following chapter will focus on the kernel itself, with an emphasis on the classes that articulate the dynamics of the simulation.

Test your understanding of this content by answering the following questions:

1. Why should the `source/` folder be treated as the center of the project's technical reading?
2. What is the advantage of interpreting GenESyS in layers, rather than reading it as a scattered set of files?
3. Why shouldn't the GUI be confused with the entire system?
4. What role does the `models/` folder still play for the developer, even in the second half of the book?

39. Simulator Kernel

39.1 Introduction

If the previous chapter organized the general map of the project, this chapter goes into the most important region of that map: the core of the simulator. This is where *GenESyS* stops being seen just as an application and starts to be understood as a system that represents models, schedules events, controls repetitions, maintains simulated time and coordinates the observability of the experiment.

The study of the kernel is decisive for a simple reason: everything the user does in the GUI and everything the developer adds via parser, *plugins* or applications ultimately depends on the infrastructure defined here. It doesn't matter whether the model was opened visually, described textually or constructed by code. In all cases it will need to be represented, verified, executed and observed by core kernel classes.

In this chapter, the focus will be on a set of particularly important classes and structures:

- `Simulator`;
- `Model`;
- `ModelSimulation`;
- `OnEventManager`;
- list of upcoming events;
- current entities and events;
- simulation cycle control mechanisms.

The goal is not yet to exhaust all kernel classes, but to build a firm understanding of the core dynamics of GenESyS.

39.2 The Kernel as Simulation Infrastructure

The *GenESyS* kernel should be understood as the layer that makes it possible for the simulator to exist regardless of the interface used. The GUI, terminal application, textual templates, *plugins* and tools may vary; the kernel, however, is the basis that supports the representation of the model and the execution of the simulation.

In architectural terms, this means that the kernel needs to solve at least four fundamental problems:

1. how to represent a simulation model;
2. how to represent the execution state of this model;
3. how to advance in simulated time through events;
4. how to make the simulation observable and controllable.

These four problems appear clearly in the project's central classes. That's why studying the kernel is not studying "another package" of the system, but the part that defines the essential behavior of the simulator.

39.3 The `Simulator` class as kernel entry point

The `Simulator` class occupies a special position in the kernel view. It presents itself as the main class of the simulation system and provides access to important managers such as `PluginManager`, `ModelManager`, `TraceManager`, `ParserManager`, and `ExperimentManager`. This organization already reveals something important: GenESyS was not structured around a single monolithic class, but around a nucleus that coordinates specialized subsystems.

From a developer's perspective, the `Simulator` class is useful for two reasons. First, because it offers a synthetic view of the project's architecture. Secondly, because it works as a high-level access point to the other mechanisms of the system. Instead of entering the software in isolated fragments, the reader can start by realizing that there is a central class that articulates the simulator's ecosystem.

39.3.1 What This Class Suggests About the Architecture

The mere presence of managers accessible from `Simulator` already indicates five architectural pillars:

- the system manages models;
- the system manages *plugins*;
- the system has an explicit tracing infrastructure;
- the system has a parser subsystem;
- the system provides support for experiments.

Even before reading the details of each manager, the developer already realizes that the GenESyS core was designed as a coordinating infrastructure and not as a single execution block.

39.4 The class `Model` as the center of the simulation model

The `Model` class deserves special attention because it represents, in the kernel, the simulation model itself. Its importance is already explained in the header documentation: it is described as probably the most important class in the GenESyS kernel. This formulation makes sense. It is where components, *data definitions*, list of future events, controls, responses, persistence, parser, consistency check, entity management and access to the simulation state are articulated.

In practical terms, `Model` is the class in which the model stops being just an idea or diagram and becomes an object that can be manipulated by the software.

39.4.1 Why `Model` is so central

The centrality of `Model` arises from the fact that it articulates several essential mechanisms at the same time:

- insertion and removal of model elements;
- creation and removal of entities;
- sending entities between components;
- model consistency check;
- persistence and loading;
- access to parser, tracing and simulation;
- maintenance of the list of upcoming events.

This means that the `Model` class is not just a component container. It is the organizing center of the model's life within the simulator.

39.4.2 Model as a Combination of Components and Data

The class documentation itself indicates that the model is mainly represented by:

- a collection of model components, appropriately configured and connected;
- a collection of *model data definitions*, which supports the model infrastructure.

This distinction is architecturally fundamental. It anticipates a principle that will return in later chapters: the model is not just made of flow blocks. It also depends on a layer of data and definitions that supports the simulation.

39.4.3 The Future Event List

Among the attributes of `Model`, one deserves special mention: the list of upcoming events. The code's own documentation describes it as one of the most important elements of an event-driven simulation system. This is easily justified. It is from this list that the simulation advances chronologically, always taking the next event to be processed.

For the developer, this is essential. The simulated time does not advance as a simple arbitrary incremental loop. It progresses by orderly firing of scheduled events, and the list of future events is the mechanism that supports this process.

39.5 The class `ModelSimulation`

If `Model` represents the model, `ModelSimulation` represents the execution of that model. This distinction is extremely important. The model and its simulation are related but not identical objects. The first describes the structure of the modeled system; the second controls its execution cycle.

`ModelSimulation`'s documentation is particularly helpful. It describes the current simulation execution layer, based on repetitions, replication length, *warm-up* configuration, and event processing. It also emphasizes that although there are higher abstractions of experimentation in development, `ModelSimulation` remains the execution core used today. This observation is useful for the developer, because it indicates which operational class concentrates the current behavior of the simulation.

39.5.1 Responsibilities of `ModelSimulation`

`ModelSimulation`'s responsibilities can be organized into a few main groups:

- start, pause, step by step and stop the simulation;

- control repetitions;
- maintain the current simulated time;
- manage replication length and period of *warm-up*;
- check termination conditions;
- manage breakpoints;
- trigger reports;
- coordinate event processing.

Note that this class is not a mere operational detail. It concentrates the effective logic of model execution and, therefore, must be studied carefully by any developer who intends to change the central dynamics of the simulator.

39.5.2 The Simulation Cycle

Reading the `ModelSimulation` class allows you to see a clear logical structure of the simulation cycle:

1. simulation initialization;
2. initializing a replication;
3. sequential event processing;
4. checking breakpoints and termination conditions;
5. replication shutdown;
6. simulation termination.

This cycle is very important for understanding the system. It shows that execution is not a simple monolithic "run" call, but a process structured into phases, each with its own responsibilities.

39.5.3 Simulated Time, Replications, and *Warm-up*

One of the merits of the `ModelSimulation` class is that it makes explicit that model execution involves more than simply passing events. There is also the management of:

- current simulated time;
- number of repetitions;
- replication length;
- *warm-up* period;
- base time unit;
- termination condition.

These elements reveal that the class was designed to support simulation-oriented experimentation and analysis, and not just ad hoc execution of a flow of events.

39.6 Events, the Current Event, and Sequential Processing

Discrete event-driven simulation depends on a central idea: the system evolves from event to event. In *GenESyS*, this logic appears very clearly in the relationship between `Model`, `ModelSimulation`, `Event` and the list of future events.

During the simulation, there is always a current event being processed. This event defines the current simulated instant and the context of the ongoing operation. When processed, it can produce effects on entities, components, model data and the system's own future agenda.

This organization has two important implications for the developer:

1. the system state is not updated diffusely, but at discrete processing points;
2. the semantics of the components and most of the model's actions depend on the current event.

This observation connects directly to the parser, entities, components and various expressions linked to the model state, which will be discussed in later chapters.

39.7 The `OnEventManager` class and core observability

If `ModelSimulation` controls execution, `OnEventManager` makes that execution observable through high-level events. This is a very elegant architectural point of GenESyS and deserves special attention.

The `OnEventManager` class allows you to register handlers for different types of occurrences in the model and simulation life cycle. Among them are:

- model check successful;
- model loading and saving;
- replication initiation and termination;
- event processing;
- creating, moving and removing entities;
- start, pause, resume and end of the simulation;
- breakpoint triggering.

This infrastructure shows that GenESyS treats observability as a constitutive part of the kernel, and not as a mere peripheral interface resource.

39.7.1 Why this is architecturally important

`OnEventManager` elegantly separates two things:

- the simulation execution logic;
- the logic for reacting to relevant simulation events.

This separation is valuable because it allows applications, tools, reporting, and debugging mechanisms to plug into kernel behavior without having to mix their logic directly with central processing. It is, in essence, an observation and reaction mechanism based on events in the simulator itself.

39.7.2 ModelEvent e SimulationEvent

The presence of the auxiliary classes `ModelEvent` and `SimulationEvent` reinforces this architecture. They function as context objects for registered handlers, carrying information about the model, the simulated time, the current event, the replication, the paused or stopped state, the created entity and the target component, among other relevant data.

With this, `OnEventManager` not only announces that “something has occurred”, but provides enough context for the reaction to that event to be technically useful.

39.8 Breakpoints and Fine-Grained Execution Control

The `ModelSimulation` class also makes explicit the existence of breakpoint lists by time, component and entity. This feature is particularly relevant because it shows that the kernel was designed not only to run models to completion, but to allow controlled observation of the simulation.

From the user’s perspective, this appears in the GUI as the ability to pause or advance step by step. From the developer’s point of view, this reveals that fine control over execution is not added by the interface in a superficial way. It is provided in the kernel infrastructure itself.

This is an important architectural decision. It means that debugging, inspection, and controlled

experimentation do not depend exclusively on the graphical application; they are based on the simulation core.

39.9 An integrated reading of the kernel

The best way to understand the core of *GenESyS* is to integrate the functions of the classes studied so far.

- `Simulator` coordinates access to central subsystems.
- `Model` represents the model, its components, data and future events.
- `ModelSimulation` controls the execution cycle for this model.
- `OnEventManager` makes relevant events observable and responsive.

This integration helps to realize that the kernel is not a set of parallel classes without articulation. It is a coherent infrastructure in which each class occupies a specific role within the same simulation dynamics.

Table 39.1: Roles of the core classes in the GenESyS kernel.

Classe	**Papel principal**
`Simulator`	Highest-level kernel class, coordinating access to central managers
`Model`	Representation of the simulation model, its components, data, entities and future events
`ModelSimulation`	Control of execution, repetitions, simulated time, termination conditions and events
`OnEventManager`	Registration and triggering of handlers associated with relevant model and simulation events

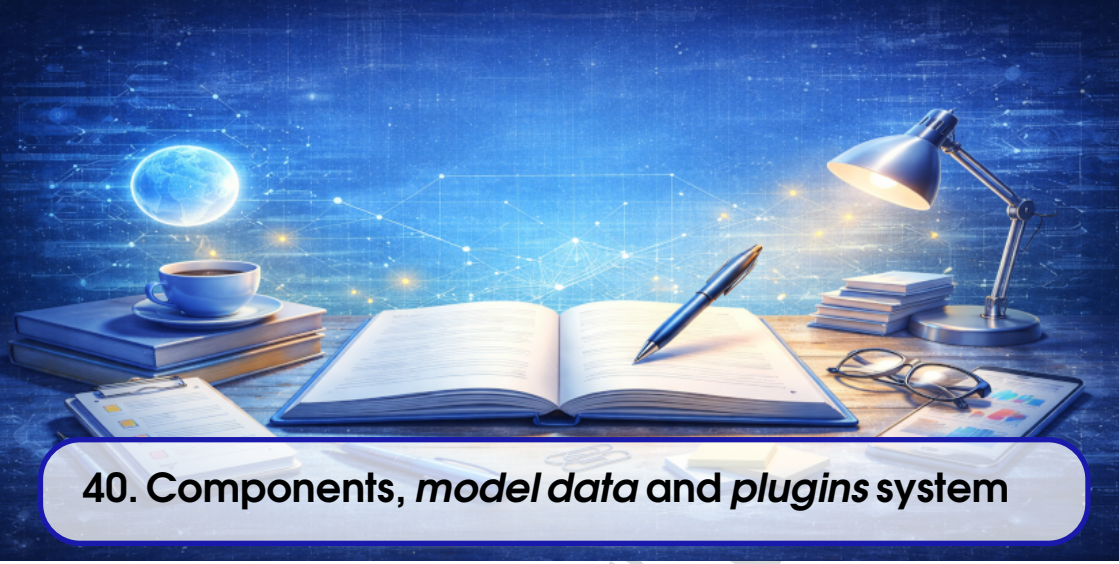
39.10 Final Considerations

This chapter presented the core of the simulator based on its most central classes. The main point to remember is that *GenESyS* organizes its simulation infrastructure clearly: there is a high-level class that coordinates subsystems, a class that represents the model, a class that controls the execution of that model, and an explicit event-driven observability mechanism. This organization is robust enough to support applications, parser, *plugins*, tracing, breakpoints and future extensions.

With this foundation established, the next natural step is to move on to the structure of the model elements themselves. In other words, if this chapter showed how the system represents and executes a model, the following chapter will show what types of objects this model is made of: components, *data definitions* and extensibility mechanisms by *plugins*.

Test your understanding of this content by answering the following questions:

1. What is the conceptual difference between `Model` and `ModelSimulation` in the GenESyS kernel?
2. Why is the list of future events such an important structure in event-driven simulation?
3. What role does `OnEventManager` play in system observability?
4. Why is the presence of breakpoints in the kernel a relevant architectural decision and not just an interface detail?



40. Components, *model data* and *plugins* system

40.1 Introduction

After studying the core of the simulator at its most central level, the next step is to understand what concrete objects the model is made of. In other words, if the previous chapter answered the question "how does GenESyS represent and execute a model?", this chapter answers another equally important question: "what are the elements that make up this model within the software?".

In GenESyS, this answer is not reduced to a single class type. The model is built on a fundamental architectural distinction between:

- template components;
- *model data definitions*.

This distinction is especially important because it organizes the entire way the simulator represents the main flow of a system and the data infrastructure that supports it. Furthermore, it is from here that the extensibility mechanism via *plugins* gains clarity. New components and new model data can be introduced to the system without the core having to be rewritten as a monolithic block.

This chapter delves deeper into this architecture. First, it discusses the `ModelDataDefinition` base class. Then, it shows how `ModelComponent` specializes it to represent behavior in the model flow. It then discusses the importance of internal and attached data, the relationship between composition and aggregation, and finally situates the *plugins* mechanism as a way to expand the system in a consistent manner.

40.2 `ModelDataDefinition` as model base class

The `ModelDataDefinition` class occupies a structural role in GenESyS. Its documentation describes it as the basis for any model data, such as queues, resources, variables, and other elements, and also for any model components. This means that it defines a common infrastructure that will be shared both by elements that support the simulation and by elements that directly participate in the model flow.

This design decision is extremely important. Instead of completely separating flow components and supporting data into unrelated hierarchies, GenESyS makes them both share a common foundation. With this, the software gains:

- a uniform identification and naming mechanism;
- common persistence support;
- common verification support;
- common infrastructure for creating internal and attached data;
- standardized tracing mechanisms and simulation controls.

In architectural terms, this makes the model more coherent. Instead of two uncommunicating universes, there is a unified hierarchy in which components and data are different species of model elements.

40.2.1 Core responsibilities of `ModelDataDefinition`

Reading the class shows some particularly relevant responsibilities:

- maintain element identity and name;
- offer persistence infrastructure;
- allow local verification of the element;
- sustain internal and attached data;
- offer integration with parser and simulation controls;
- provide tracing mechanisms;
- defines extension point for creating related data.

This means that `ModelDataDefinition` is not just a convenient abstract superclass. It is a real block of infrastructure on which the rest of the model rests.

40.2.2 Internal Data and Attached Data

One of the most interesting aspects of the `ModelDataDefinition` class is the explicit distinction between *internal data* and *attached data*. This distinction, which has already been important in previous discussions of the book, appears here as part of the system's basic infrastructure.

- **Internal data** corresponds to elements internal to the object, maintained by composition.
- **Attached data** corresponds to referenced data, maintained by aggregation.

This distinction is architecturally very valuable. It allows the software to clearly represent both elements that structurally belong to a component and elements that are only associated with it. In other words, GenESyS does not treat all relationships between model objects in the same way, and this greatly improves the clarity of the system's internal semantics.

40.2.3 Persistence and Verification as Base Responsibilities

Another important aspect is that persistence and verification already appear in the base class. This shows that the system expects model elements to be:

- loaded;
- saved;
- validated;

- reconstituted;
- prepared for simulation.

By concentrating this infrastructure at the base, GenESyS avoids unnecessary repetition in subclasses and imposes consistent discipline on model extensions.

40.3 `ModelComponent` as behavioral specialization

The `ModelComponent` class inherits from `ModelDataDefinition` and adds a decisive dimension to it: behavior guided by entity arrival and event triggering. Its documentation describes it as a block that represents a specific behavior to be simulated and that is activated when an entity arrives at the component.

This formulation is extremely important because it defines the place of the components in the GenESyS architecture. A component is not just a visual object nor just a nominal block of the model. It is a unit of behavior within the flow of the modeled system.

40.3.1 The Idea of Flow Between Components

A model built with `ModelComponent` is, in essence, a set of interconnected components. The global behavior of the model emerges from the path of entities through this flow. This explains why the class maintains a `ConnectionManager` and provides connection operations between components.

This is one of the big differences between `ModelComponent` and a generic *model data definition*:

- a component directly participates in the flow logic;
- model data can support this flow without itself being part of the main path.

40.3.2 Event dispatch and `_onDispatchEvent`

The `ModelComponent` class makes it clear, through its interface, that each concrete component must implement a specific dispatch behavior. This appears in the protected and pure virtual method `_onDispatchEvent(Entity*, unsigned int)`. It is at this point that the specific semantics of each concrete component comes into existence.

This architectural choice is particularly good because it separates:

- the common infrastructure of the components;
- the particular behavior of each type of component.

With this, GenESyS can have a well-defined base class, but still maintain highly specialized subclasses.

40.3.3 Components as Specialized Model Data

The fact that `ModelComponent` inherits from `ModelDataDefinition` has profound consequences:

- a component also has a name, identity and persistence;
- a component can also have internal and attached data;
- a component participates in the common verification infrastructure;
- a component naturally integrates with the rest of the model's semantics.

This avoids an artificial separation between data structure and behavioral structure. Instead, the system recognizes that a component is also an element of the model, just endowed with a specific behavioral role.

40.4 Main Flow and Supporting Data

One of the most important ideas that the developer needs to consolidate is the difference between:

- the main flow of the model;
- the supporting data that supports this flow.

In practice:

- the main flow is composed of connected components;
- supporting data includes queues, resources, attributes, variables, collectors, counters and other related objects.

This distinction is central because it guides both the modeling and extension of the software. Many design errors occur precisely when trying to treat a support element as if it were a component of the flow, or vice versa.

40.4.1 Why This Distinction Improves the Architecture

By separating main stream and supporting data, GenESyS gains clarity:

- procedural behavior is concentrated in components;
- support objects are represented as model data;
- the relationship between the two can be expressed in an explicit and controlled way;
- the system becomes more extensible.

This also improves the readability of the software. The developer can more easily identify whether a new element they want to implement should be a component, model data, or a combination of the two by composition or aggregation.

40.5 Composition, Aggregation, and the Internal Model Design

The infrastructure of `ModelDataDefinition` and `ModelComponent` reveals that GenESyS takes the distinction between composition and aggregation seriously.

In composition, the internal element structurally belongs to the object that contains it. In practical terms, this means that it is created, maintained, and destroyed as part of the main object's lifecycle. In aggregation, on the contrary, there is only a reference or link to existing data in the model, without absorbing its life cycle.

This distinction is particularly useful when a component:

- internally maintains its own counters or statistics;
- or references external objects such as queues and resources.

Without this separation, the system would run the risk of producing ill-defined links between model elements, making persistence, removal, verification and reuse difficult.

40.5.1 Internal data as a composition relationship

When a component or model data maintains *internal data*, it is explaining a composition relationship. This is often suitable for elements that:

- exists only to support the internal functioning of that object;
- don't make sense as model-independent objects;
- should disappear along with the main object.

40.5.2 Attached data as an aggregation relationship

When an object maintains *attached data*, what exists is an aggregation relationship. The referenced data continues to exist in the model as its own object, and can be shared, referenced by other elements and persisted independently.

This separation between composition and aggregation makes the GenESyS architecture much cleaner and more semantically expressive.

40.6 The *plugins* system

The presence of *plugins* is one of the most important features of GenESyS as extensible software. The `Simulator` class itself already makes this evident when exposing a `PluginManager`. In the current repository, the plugin implementations are built as static libraries and linked into the simulator, while the connector and manager layers organize metadata, discovery and instantiation. This shows that the *plugins* mechanism is neither improvised nor peripheral; it is a structural part of the system, but not a set of independent runtime packages.

For the developer, the consequence is clear: expanding GenESyS does not necessarily mean directly modifying the kernel. In many cases, the most appropriate strategy is to introduce new capabilities through *plugins*. This is especially true for:

- new components;
- new *data definitions*;
- new language extensions;
- new specialized behaviors.

40.6.1 Why *plugins* are important

Without *plugins*, the system's growth would depend on increasingly profound changes to the core. This would tend to make the software more rigid and more difficult to maintain. With *plugins*, the project gains:

- modularity;
- extensibility;
- domain specialization;
- greater separation between stable infrastructure and additional functionality.

This is an especially appropriate architectural decision for a simulator that aims to be generic and expandable. At the current stage, that extensibility is expressed at build and link time, not as a separate deployable plugin runtime. Any future ABI boundary belongs to a later architectural decision, not to the present contract described here.

40.6.2 Components and Data as Natural Extensions

From the model's point of view, many *plugins* precisely materialize as new components and new data. This means that the architecture discussed so far, `ModelDataDefinition`, `ModelComponent`, internal data, and attached data, is not just a matter of internal elegance. It is the foundation on which the system's extensibility rests.

In other words, understanding the architecture of the model elements is also understanding the architecture of *plugins*.

40.7 How to Think About a New Extension

A developer looking to extend GenESyS should start with a very simple question:

> What exactly am I trying to introduce into the system?

From this question, it is usually possible to distinguish some cases:

- a new behavior in the model flow;
- new data supporting the model;
- a new combination of behavior and internal data;
- a new language or parser needs;
- a new application or tool.

If the need is for new behavior in the flow, the extension will probably be in the form of new `ModelComponent`. If it is a new backing object, it will probably take the form of new `ModelDataDefinition`. If it involves both, it will be necessary to think carefully about the composition, aggregation and internal relationships of the new element.

40.8 Misreading This Chapter Avoids

This chapter seeks to avoid a very common misreading: imagining that the GenESyS model is composed only of visual boxes in the GUI. In reality, the model is a much richer structure, supported by a hierarchy of classes and a clear architectural distinction between behavior and data.

It also seeks to avoid another error: imagining that *plugins* are just dynamic loading details or independent runtime packages. In GenESyS, they are part of the system's growth logic and, in the current implementation, are aggregated through static libraries and connector-managed metadata. Extensibility is not an appendix; it is an architectural principle.

Table 40.1: Central Distinctions for Understanding the Model Structure in GenESyS.

Distinction	**Architectural Meaning**
`ModelComponent` vs. `ModelDataDefinition`	Difference between in-stream behavior and model data/infrastructure
Main stream vs. supporting data	Difference between procedural path and elements that support execution
Internal data vs. data	Difference between composition and aggregation within the model
Kernel vs. *plugins*	Difference between core infrastructure and specialized extensions

40.9 Final Considerations

This chapter showed that the GenESyS model is structured based on a coherent and extensible architecture. The `ModelDataDefinition` class provides the base infrastructure for model elements. The `ModelComponent` class specializes this base to represent behavior in the flow. The distinction between main flow and supporting data, as well as between composition and aggregation, clearly organizes the internal semantics of the system. Finally, the *plugins* mechanism appears as a natural consequence of this architecture, allowing the simulator to be expanded within the current static-linking and metadata-based infrastructure without losing coherence.

With this, the reader already has the necessary basis to understand how new elements can be introduced into the system. The next natural step is to study the subsystem that connects the language, expressions and semantics of the model: the GenESyS parser and its relationship with the simulator's internal language.

Test your understanding of this content by answering the following questions:

1. Why does `ModelComponent` inherit from `ModelDataDefinition`?
2. How does the distinction between main stream and supporting data help you understand the structure of the model?
3. What is the difference between *internal data* and *attached data* in the internal design of GenESyS?
4. Why should the *plugins* system be understood as a structural part of the architecture and not just as an optional extension mechanism?

41. Parser, Expressions, and Internal Language

41.1 Introduction

Throughout the first half of this book, the *GenESyS* language appeared to the user mainly through the *Simulang* tab. At that stage, the central interest was in realizing that the graphic model also has a textual representation. For the developer, however, this observation is just the starting point. The real problem now is different: how the system interprets expressions, how the language is linked to the model state and how new syntactic and semantic capabilities can be introduced into the simulator.

This chapter deals exactly with this subsystem. Its focus is not just on the generic notion of "parser", but on the concrete way *GenESyS* organizes the evaluation of expressions, the handling of model symbols, the integration with stochastic functions, the Bison/Flex grammar, and the extension points that allow the system to grow.

The importance of this topic is great. In a simulator like *GenESyS*, language is not ornament. Many model decisions involve expressions: parameters, formulas, attribute references, statistical queries, simulation controls and probabilistic functions. The parser, therefore, is not just a textual convenience; it is part of the operational semantics of the simulator.

41.2 The Role of the Parser in GenESyS

The *GenESyS* parser should be understood as the infrastructure that makes it possible to interpret expressions and functions in model context. This means that it doesn't just operate on abstract text, but on text that needs to be related to concrete elements of the simulator:

- attributes of the current entity;
- simulation controls and responses;
- accountants and statistical collectors;
- variables, formulas, queues, resources and other model objects;
- mathematical and probabilistic functions;
- information about the current state of the simulation.

In other words, the parser works as a bridge between language and the internal state of the system. This is why it is so important in the GenESyS architecture.

41.2.1 Parser as a Semantic Subsystem

It is useful to avoid a limited reading of the parser as a simple parser. In *GenESyS*, it does more than recognize a well-formed expression. It also needs:

- resolves symbol names;
- identify the nature of these symbols;
- get values associated with the current state of the model;
- integrate stochastic functions supported by a sampler;
- propagate useful error messages.

This means that the correct reading of the parser in GenESyS is semantic and not just syntactic.

41.3 The `Parser_if` interface

The `Parser_if` interface offers a very clear summary of the role of the parser in the project. It defines a minimal contract for evaluating stochastic expressions and functions, making some central responsibilities explicit:

- evaluate expressions and return a numeric value;
- offer a variant that reports success and an error message without throwing an exception;
- make the last error message available;
- associate the parser with an instance of `Sampler_if`;
- expose the internal parser driver for advanced scenarios.

This interface is important because it shows that the parser is seen, architecturally, as a simulator service and not as an accidental implementation detail.

41.3.1 Expression Evaluation

The most obvious point of the interface is the expression evaluation method. The existence of two main forms of `parse` is particularly revealing:

- a way that can throw exceptions;
- a way that avoids exceptions and explicitly reports success and error messages.

This decision improves the integration of the parser with different layers of the system. In some contexts, the developer may prefer exception handling. In others, especially in validation and user interface, it is more convenient to receive the success state explicitly.

41.3.2 Integration with `Sampler_if`

The interface also makes it clear that the parser is not just a parser of deterministic expressions. It integrates stochastic functions supported by `Sampler_if`. This is important because it reinforces the specific nature of the GenESyS language: it is not just a mini-arithmetic language, but a simulation-oriented language.

In practical terms, this means that probabilistic functions of the language depend on the presence of a sampler associated with the parser. As a result, the semantics of expressions naturally incorporate random generation mechanisms.

41.4 The `Model` class as parser user

The role of the parser within the kernel is especially clear in the `Model` class. It exposes methods like `parseExpression` and `checkExpression`, which shows that the model uses the parser directly to:

- evaluate expressions defined in the model context;
- validate expressions entered by the user;
- check references to *data definitions* in the text of expressions.

This detail is architecturally very important. It means that the parser is not isolated in an external layer; he participates in the model's life.

41.4.1 Why does `Model` invoke the parser

The presence of these methods in `Model` reveals an important principle: expressions are part of the model's semantics, and not just the interface. A model component or data that stores an expression needs to be able to validate it and, in many cases, evaluate it within the current context of the simulation.

This architectural choice is very good because it keeps the language connected to the object where it really belongs: the model.

41.4.2 Expressions and Consistency Checking

It's also important to note that the `Model` class offers `checkExpression`. This shows that the parser participates not only in the execution, but also in the model checking phase. In other words, the validity of an expression is treated as part of the integrity of the model, and not as a problem to be discovered only during an execution that has already started.

This decision greatly improves the robustness of the system and helps explain why model checking is such an important step in the use and development of GenESyS.

41.5 The GenESyS Bison/Flex Grammar

The concrete implementation of the parser in `source/parser/parserBisonFlex` shows a well-defined implementation of this subsystem. The `bisonparser.yy` file defines a C++ grammar based on `lalr1.cc`, with its own parser class, typed tokens, localizations, support code, and explicit integration with `genesyspp_driver`. This shows that the parser was designed as an explicitly documented component of the simulator infrastructure, and not as an improvised textual routine.

Grammar covers, among other elements:

- numbers;
- relational and logical operators;

- trigonometric and mathematical functions;
- probabilistic functions;
- functions associated with the kernel;
- template symbols;
- extensions introduced by *plugins*.

This scope is enough to show that the parser is one of the subsystems with the highest semantic density in the project.

41.5.1 A Language Truly Integrated with the Model

Reading the grammar shows something particularly relevant: many tokens and productions do not just refer to abstract syntactic constructions, but to real objects in the model. There are rules for attributes, simulation responses, simulation controls, and elements such as variables, formulas, queues, and resources. Functions associated with statistics, counters and simulation status also appear.

This reinforces an essential point:

> The GenESyS language is not external to the simulator; it is a textual projection of the semantics of the model and simulation.

41.5.2 Probabilistic Functions and Sampling

The grammar makes explicit the presence of probabilistic functions such as exponential, normal, uniform, Weibull, lognormal, gamma, Erlang, triangular and beta, among others. In all of them, the operational semantics depend on the `Sampler_if` associated with the parser.

This is very important for the developer, because it shows that:

- the simulator language directly incorporates stochastic sampling;
- the parser implementation needs to maintain a consistent contract with the statistical subsystem;
- error handling in these functions is not only syntactic but also operational.

41.6 Model Symbols and Dependence on the Current State

One of the most interesting features of the GenESyS parser is its explicit dependence on the current state of the simulation. The grammar shows, for example, that access to attributes depends on the current entity associated with the current event. Likewise, functions such as `TNOW`, `TFIN`, number of repetitions and identifiers of the current entity depend on the state maintained in `ModelSimulation` and the current event.

This feature is architecturally important because it reveals that the interpretation of expressions in GenESyS is contextual. The same expression can have different meanings depending on:

- the loaded model;
- the current entity;
- the event being processed;
- the simulated instant;
- the model data available in that context.

41.6.1 Parser como consumidor do estado do kernel

This means that the parser directly consumes information from the kernel. It queries:

- the model;

- the simulation;
- the current event;
- or *data manager*;
- collectors and counters;
- controls and responses.

This dependence is not a defect. Rather, it is part of the nature of the problem. A parser for a simulation language needs to know the universe of symbols and states of the simulator.

41.7 Extension points by *plugins*

The grammar of *GenESyS* very clearly reveals the existence of regions designed for extension. The blocks marked by structured comments for *plugins* inclusions, tokens, types, and productions show that the system is designed for growth.

This aspect is particularly important for the developer. It shows that expanding the language does not necessarily depend on completely rewriting the base grammar. There are architecturally planned points for new elements to be introduced.

41.7.1 Syntactic and Semantic Extension

When a *plugin* extends the language, it doesn't just introduce new tokens. In general, it also introduces:

- new symbols;
- new functions;
- new interpretation rules;
- new relationships between text and model data.

This means that the parser extension is simultaneously syntactic and semantic. This is precisely why the `ParserChangesInformation` and `ParserManager` infrastructure exists in the project.

41.8 The class `ParserManager`

The `ParserManager` class makes explicit GenESyS's architectural ambition to support language evolution. Its interface provides:

- generation of a new parser based on changes;
- connecting a new parser to the system.

Although the header itself indicates that important parts of this flow are still under development, the existence of this class is already architecturally relevant. It shows that GenESyS does not treat grammar as an entirely static artifact. There is, in the system design, space for explicit management of the evolution of the parser.

41.8.1 What This Means for the Developer

For the developer, this means that working with language in GenESyS is not limited to modifying a Bison or Flex file. The project foresees, at an architectural level, a subsystem responsible for:

- describe changes;
- generate parser artifacts;
- connect new parsers to the simulator.

Even though this infrastructure is still under development, it already guides a sensible way of thinking about the evolution of language.

41.9 Erros, mensagens e robustez

Another important aspect of parser implementation is error handling. The grammar shows explicit concern for useful messages, including in the case of illegal tokens, missing literals, and failures in probabilistic functions. This is important because simulation language, when misinterpreted or misdiagnosed, quickly becomes a source of frustration for user and developer.

The robustness of the parser therefore depends on three factors:

- correct syntactic recognition;
- correct semantic integration with the model;
- producing sufficiently informative error messages.

This triad is especially important in a system where expressions can appear at many points in the model.

41.10 How to Study and Modify the Parser Productively

The most productive way to study the GenESyS parser is in layers.

First, understand the `Parser_if` interface and the way `Model` invokes the parser. Then, study the Bison/Flex grammar and the way it accesses model symbols. Then identify the extension points introduced by *plugins*. Only then does it make sense to propose more significant changes, especially when they involve new functions, new symbols or changes to grammar.

It is also important to remember that changes to the parser are not just textual changes. They can affect:

- model checking;
- component semantics and *data definitions*;
- integration with sampling;
- error messages;
- compatibility with *plugins*.

Table 41.1: Reading Layers of the GenESyS Parser.

Camada	**Pergunta principal**
Interface	What service does the parser provide to the rest of the system?
Model Integration	How does the model use this service in simulation?
Concrete grammar	How is language recognized and evaluated?
Extensibility	How do new symbols and functions enter the system?
Robustness	How are errors detected and reported?

41.11 Final Considerations

This chapter has shown that the *GenESyS* parser is much more than an auxiliary text layer. It is an integral part of the simulator semantics. The `Parser_if` interface, the direct integration with `Model`, the

Bison/Flex grammar, the use of model and simulation symbols, the dependence on a sampler, and the explicit extension points by *plugins* reveal an architecturally dense and strategically important subsystem for the project.

With this, the reader already has the main elements to understand how *GenESyS* links language, model and execution. The next step is to broaden the focus again, leaving the parser and returning to the project's broader ecosystem: applications, tools, C++ examples, tests and software evolution strategies. This is exactly what will be covered in the next chapter, which closes this version of the developer manual.

Test your understanding of this content by answering the following questions:

1. Why should the *GenESyS* parser be understood as a semantic subsystem and not just a syntactic one?
2. How important is it for the `Model` class to expose methods like `parseExpression` and `checkExpression`?
3. How does the *GenESyS* grammar show dependence on the current state of the model and simulation?
4. Why are extension points by *plugins* essential to the evolution of the simulator language?

42. Applications, Tools, C++ Examples, Tests, and Project Evolution

42.1 Introduction

This chapter wraps up the main structure of this developer manual by broadening the focus again. In the previous chapters, the attention fell on the central architecture of the system: organization of the source code, kernel, structure of model elements, *plugins*, and parser. Now the goal is to examine how this infrastructure projects into concrete applications, helper tools, programmatic examples in C++, and testing mechanisms. In other words, this chapter is about the development ecosystem that forms around the *GenESyS* core.

The importance of this closure is great. An extensible simulator doesn't just live off its core classes. It also depends on applications that expose it to the user, on tools that support its use and maintenance, on examples that show how the library can be used programmatically and on tests that support the safe evolution of the system. Without these elements, the core of the software could exist, but the project as a development platform would be impoverished.

Therefore, this chapter should be read as a final synthesis of the second half of the book. It shows how GenESyS transitions:

- from the kernel to applications;
- of the architecture for programmatic use;
- functional extension for technical maintenance;
- of stable infrastructure for the future evolution of the project.

42.2 Applications as Projections of the Kernel

The `source/applications` folder is the place where GenESyS stops appearing just as infrastructure and starts to take on concrete forms of interaction. This observation is essential for the developer. It shows that the project was not designed to exist just as an abstract library, but to support real applications at its core.

Architecturally, this is very valuable. By separating the kernel from the applications, the system gains:

- modularity;
- core reuse on different interfaces;
- lower coupling between simulation logic and form of interaction;
- greater capacity for evolution.

In practice, this separation allows the GUI, Shell application, model-specific applications, and other future layers to exist as distinct projections of the same set of core services.

42.2.1 The GUI as an Application on the Core

In the first half of this book, the GUI was treated as the main way of using the simulator. Now, it starts to be seen in another way: as an application built on the GenESyS infrastructure. This shift in perspective is especially important for the developer.

The GUI is not the core. It is an application that:

- consumes the model;
- consumes the semantics of components and data;
- connects to simulation engines;
- exposes these capabilities to the user through a visual interface.

This separation is architecturally correct and explains why the system can, in principle, support other applications in addition to the graphical interface.

42.2.2 The Shell and Model-Specific Applications as Alternative Forms of Use

The current build tree of the project shows that the command-line applications are handled in a particularly structured way. The `CMakeLists.txt` of `source/applications/shell` defines the `genesys_shell` target, while `source/applications/modelSpecific` defines `genesys_modelspecific_app` and lets the build select a concrete model-specific example file to compose the executable.

This is extremely important for the developer. It shows that the command-line path is not just an old auxiliary tool or a marginal way of use. It is an effective application of GenESyS and, more than that, a valuable environment for:

- study the programmatic use of the simulator;
- build compact examples;
- develop lightweight applications based on the library;
- check behaviors without GUI mediation.

The *Worker* service remains outside this application path as a local or private component only; this chapter does not provide public deployment instructions or imply that public exposure is approved.

42.3 The examples in C++ as developer material

In the first half of the book, the examples in C++ of the command-line applications were deliberately left out, because the user manual should not start with code. Now, however, they begin to occupy a central place. This is because these examples show one of the most important capabilities of GenESyS as software: its use as a simulation library.

The model-specific application's `CMakeLists.txt` explicitly shows the existence of build-selectable examples, including *smart* examples and more elaborate teaching examples, which confirms the role of these files as study material and architectural demonstration.

42.3.1 Why These Examples Are So Valuable

The examples in C++ are valuable because they allow you to observe the simulator from another angle. In the GUI, the model appears as a visual and operational artifact. In the programmatic examples, it appears as an object explicitly constructed by the developer. This helps reveal:

- how *Simulator* is instantiated;
- how the model is built by code;
- how components and *data definitions* are created and related;
- how execution is triggered;
- how the simulator library can be reused in new applications.

42.3.2 The bridge between examples in C++ and saved models

One of the most didactically rich features of the project is the possibility of relating:

- examples built programmatically in C++;
- models saved in the `models/` folder.

This bridge is extremely useful for the developer because it shows the same system from two perspectives:

- as library object;
- as a modeling and usage artifact.

This correspondence helps to consolidate the understanding of GenESyS as a platform, and not just as a ready-made application.

42.4 Tools as a Support Layer

The `source/tools` folder represents an important region of the project, although it is often less discussed than the kernel, parser, and *plugins*. Its general function is to support system use, development, integration, or maintenance.

From a developer's point of view, tools are relevant for three reasons:

- help make the simulator ecosystem more productive;
- offer points of support for observation or automation;
- can serve as a basis for new integrations.

In architectural terms, the existence of `tools` shows that the project is not limited to the "kernel versus application" dichotomy. There is also space for intermediate instruments, utilities and support mechanisms.

42.4.1 When to study `tools`

Tools should not always be the first reading priority. In general, it makes more sense to study them after the developer already understands:

- the simulator core;
- the *plugins* system;
- the parser;
- at least one concrete application, such as the GUI or the Shell.

After that, reading the tools tends to be much more productive, because the developer can better see their role in the project ecosystem.

42.5 The Importance of Tests

In a project of the size and ambition of *GenESyS*, the existence of tests is an essential part of the software's evolutionary architecture. This not only means that the project "has tests", but that the build system itself already treats them as a structured part of development.

`CMakeLists.txt` of `source/tests` explicitly shows the division between unit tests and *smoke tests*, with an aggregated target `genesys_tests` and the possibility of enabling or disabling the construction of smoke tests by build option

This organization is particularly valuable because it gives the developer a way to think about validation in layers:

- unit tests for local behavior and minor invariants;
- *smoke tests* for broader system health.

42.5.1 Why Tests Are Part of the Architecture, Not Just the Routine

It is common to think of testing only as an activity external to the main project. In GenESyS, however, they must be read as part of the system's evolutionary infrastructure. This is particularly important because the software has:

- relatively dense kernel;
- extension mechanisms;
- multiple applications;
- integration between parser, model, events and observability.

In a context like this, changing code without testing support tends to be risky. Therefore, the developer must see the `source/tests` folder as a direct ally in project maintenance.

42.5.2 How the Developer Should Use the Tests

A disciplined development practice in GenESyS must consider testing in at least three moments:

1. before the change, to understand the current basis of behavior;
2. during the change, to avoid local regressions;
3. after the change, to validate integration and general sanity.

This discipline is especially important when the change affects the kernel, parser, *plugins* or applications.

42.6 Build, Applications, and Behavior Selection

Even though the build is not the guiding axis of the book, it continues to offer the developer an important set of signals about the project organization. The Shell and model-specific build files, for example, show that the system was designed to allow configurable selection of the behavior of the command-line executable, either by a fixed Shell entry point or by concrete examples under `source/applications/modelSpecific`.

This reveals something important about the project:

> GenESyS is not just a program; it is a platform upon which application behaviors can be chosen and assembled.

This observation is of great interest to the developer because it reinforces the modularity of the system and helps to think about future applications about the core.

42.7 Project Evolution and Contribution Paths

The combined reading of kernel, parser, *plugins*, applications, tools and tests shows that GenESyS is a project in continuous evolution. This evolution can occur on several fronts:

- kernel improvements;
- expansion of the set of components and *data definitions*;
- evolution of the language and parser;
- strengthening the GUI and other applications;
- expansion of auxiliary tools;
- consolidation of the test base.

From the developer's point of view, the most useful question is no longer:

> Where can I move?

and becomes:

> At which layer of the system does my change make the most sense architecturally?

This change in perspective is what distinguishes an opportunistic modification from a technically grounded contribution.

42.7.1 How to Choose an Entry Point for Contribution

The choice of entry point depends largely on the type of contribution desired.

- If the intention is to change the central dynamics of the simulation, the study must begin in the kernel.
- If the intention is to expand language and expressions, the study must begin with the parser.
- If the intention is to add modeling elements, reading should focus on *plugins*, components and *model data*.
- If the intention is to improve the user experience, the GUI and applications become central.
- If the intention is to increase robustness, testing and tracing become a priority.

This way of thinking greatly improves the quality of the contribution to the project.

42.7.2 The Role of Examples and Tests in Evolution

Examples and tests play complementary roles in software evolution.

- Examples show como usar the system and help you explore new capabilities.
- Tests help verify se o sistema continua correto when being modified.

A GenESyS developer needs to value both. Without examples, the system loses its ability to demonstrate and learn. Without testing, you lose evolutionary security.

42.8 An Integrated Final View of GenESyS

Coming to the end of this version of the developer manual, it is useful to return to the integrated view of the project.

GenESyS can be understood as:

- a simulation kernel;
- a hierarchy of objects representing model, components and data;
- a language system and parser;
- an extensibility infrastructure by *plugins*;
- a set of applications built on top of the kernel;
- an ecosystem of examples, tools and tests that supports use and evolution.

This integrated view is important because it avoids too partial readings. GenESyS is not just GUI, not just parser, not just *plugin framework*, not just simulation library. It is the articulation of all these dimensions.

Table 42.1: Main Dimensions of GenESyS as a Software Platform.

Dimension	**Role in the Project**
Kernel	Central Simulation Infrastructure
Parser	Language, expressions and textual semantics of the model
Plugins	Simulator extensibility and expansion of modeling elements
Applications	Concrete forms of use, such as the GUI and Shell
Tools	Support for use, integration and maintenance
Tests	Evolutionary security, validation and regression prevention
Examples	Material for demonstration, study and library use

42.9 Final Considerations

This chapter has closed the main structure of the developer's manual by showing that *GenESyS* should not only be read for its central classes, but also for its ecosystem of applications, tools, examples and tests. The GUI now appears as an application on top of the core. The Shell application and the model-specific examples in C++ reveal the use of the simulator as a library. Tools expand the development environment. The test tree provides the basis for safe evolution. And the general organization of the project shows that GenESyS should be thought of as a software platform, and not just as a ready-made simulator.

With this, the reader now has a sufficiently broad and technically grounded view to use, study and expand GenESyS in a careful way. From here, further development can follow different paths: review and improvement of the GUI, more detailed study of components and *plugins*, evolution of the parser, strengthening of the testing infrastructure, development of new applications or creation of new modeling domains on the same core. In all these cases, the logic remains the same: understand the correct layer, respect the system architecture and evolve the simulator based on its own structural principles.

Test your understanding of this content by answering the following questions:

1. Why should the GUI be understood, from the developer's point of view, as an application on the core and not as the entire system?
2. What is the architectural value of the examples in `C++` of the Shell and model-specific applications?
3. Why should the project's test base be treated as part of the software evolution infrastructure?
4. How does the distinction between kernel, parser, *plugins*, applications, tools, examples and tests help the developer to better choose the entry point for a contribution?